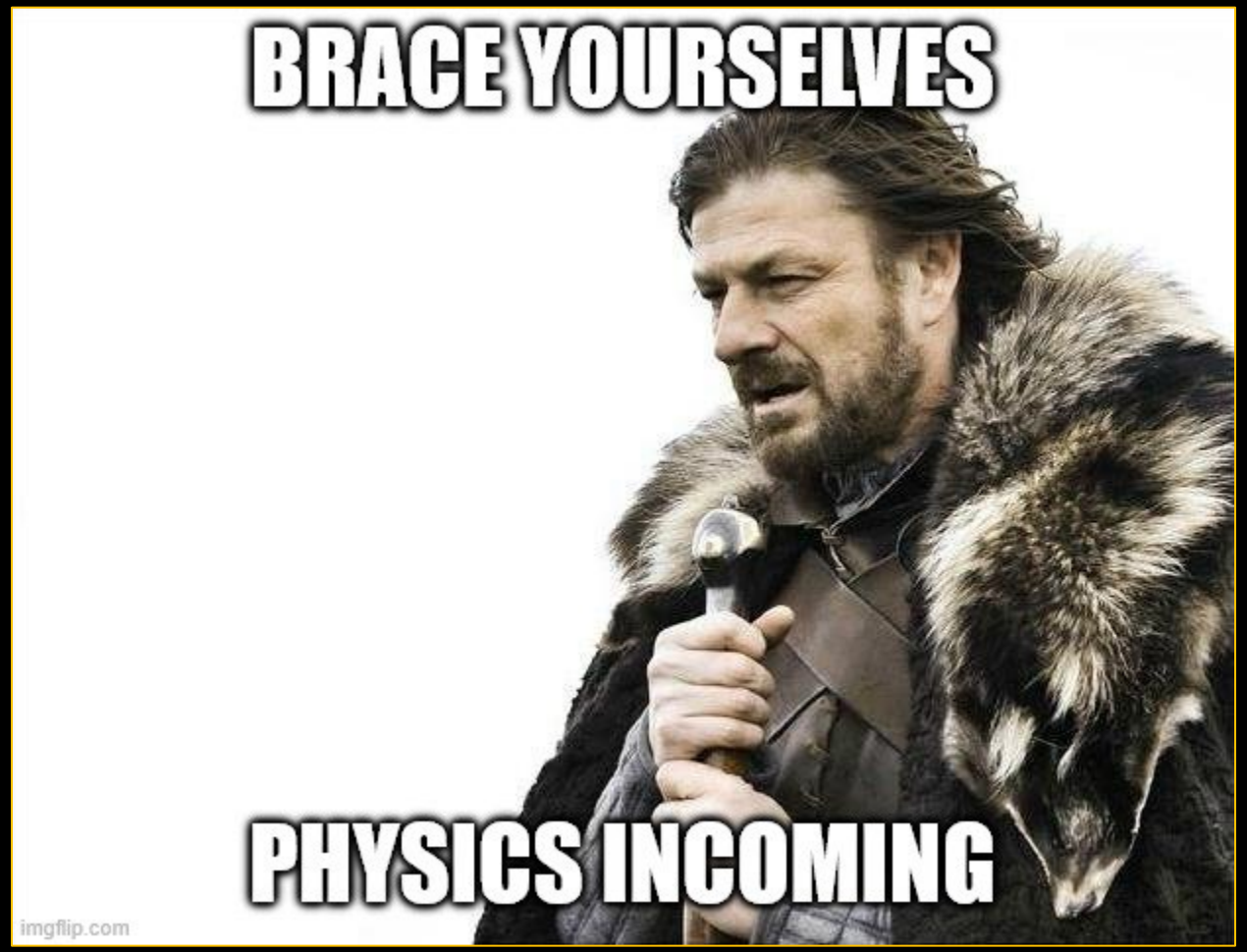
GRAPHICS PROGRAMMING I

SHADING & DIRECT ILLUMINATION

THEORIA


BRACE YOURSELVES
PHYSICS INCOMING
imgflip.com

ACT I

Light & Matter

ACT II

From Physics to Model

ACT III

Inside the Equation

ACT IV

BRDFs

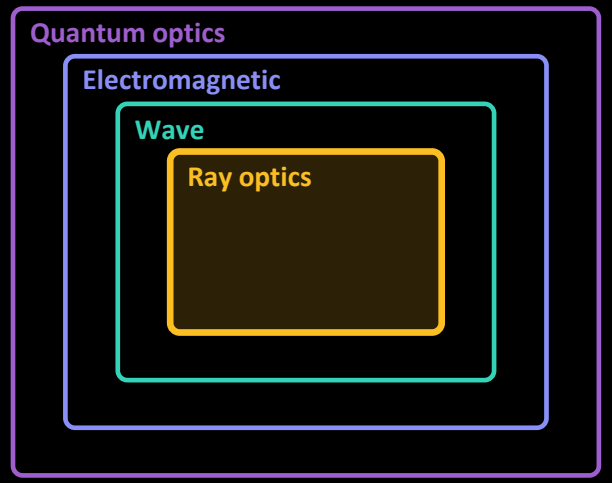
ACT V

Surface Variation

PHYSICS OF LIGHT | MODELS

Physicist built ever-more complete models of light. Each explains more than the last. For real-time graphics, the simplest (ray optics) is often accurate enough. Though it is interesting to explore some of the underlying physics, as these fundamentals do have their use in the computer graphics field. These are the currently accepted models, and their associated phenomena:

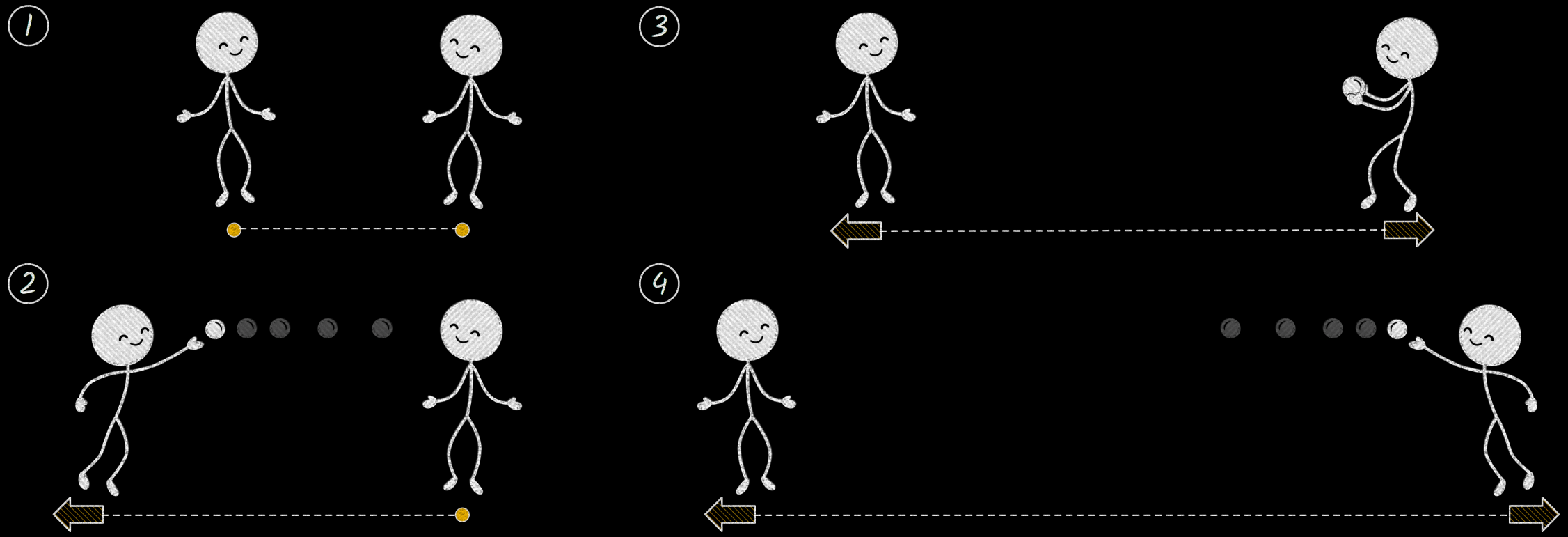
- **Ray Optics** [Reflection | Refraction | Transmission]
- Wave Optics [Diffraction | Interference]
- Electromagnetic Optics [Polarization | Dispersion]
- Quantum Optics [Fluorescence | Phosphorescence]





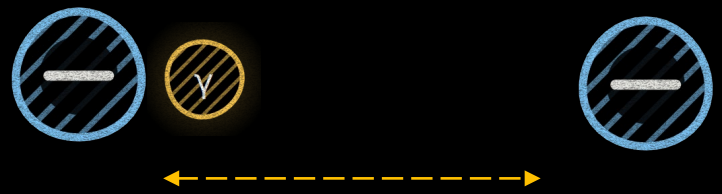
PHYSICS OF LIGHT | ELECTRIC CHARGE

Imagine two people throwing a ball in space. At first both are stationary, but when the first person throws the ball, the recoil pushes him backward. When the second person catches it, the ball's momentum transfers to her, pushing her in the same direction the ball was traveling. Now both are drifting apart, or in other words, repelling.



PHYSICS OF LIGHT | ELECTRIC CHARGE

At the quantum scale, particles such as electrons can also exchange momentum. In quantum physics, we describe their interaction using virtual photons. The ball example helps illustrate this momentum transfer, but it is only an analogy. Virtual photons belong to the mathematical description of the interaction, rather than being ordinary light particles thrown between electrons.



For two stationary electrons, the repulsive force follows an inverse square law: the closer they are, the stronger the force. Halving the distance makes the force four times stronger, while doubling it makes the force four times weaker.

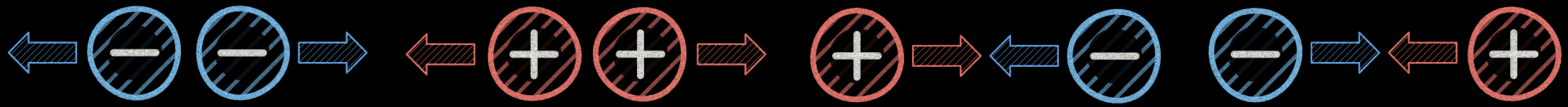


PHYSICS OF LIGHT | ELECTRIC CHARGE

Many other particles exchange virtual photons, and their tendency to interact this way is called electric charge. The electron has an electric charge of $-1e$, where e is the basic unit of electric charge. A particle can have multiple of these basic units!

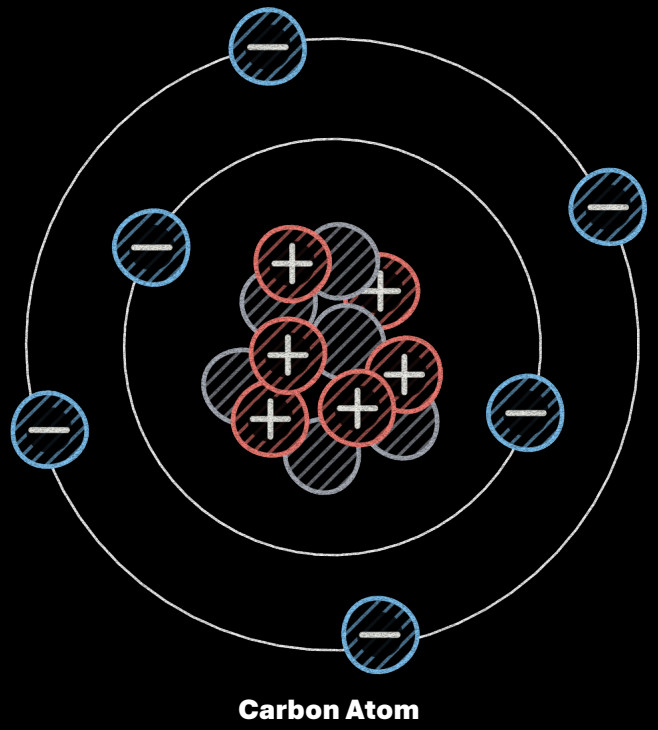
We also have protons, which interact in the same way as electrons but with opposite sign, so their charge is positive: $+1e$. We often represent the electron and proton with their electric charge sign.

Another interesting thing is that two particles with the same type of charge (positive or negative) repel each other, while opposites attract each other.

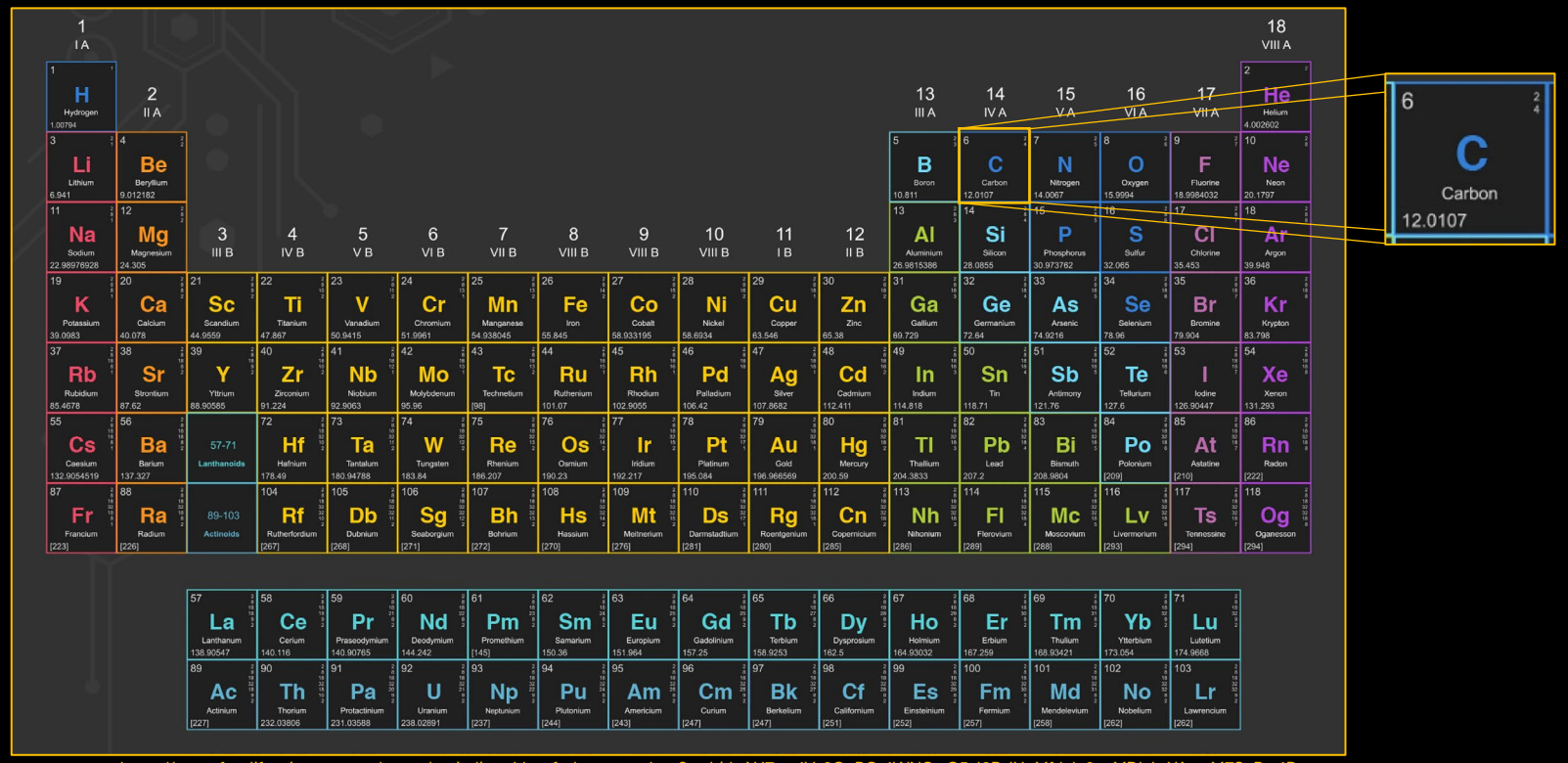


PHYSICS OF LIGHT | ELECTRIC CHARGE

To connect this to chemistry, the periodic table is more than a list of elements. Each element's atomic number is its proton count, and in a neutral atom the electrons match: same number. Take carbon (atomic number 6): 6 protons and 6 electrons. Its period (the row) tells us how many shells those electrons fill. Carbon sits in row 2, so 2 shells: 2 electrons in the inner one, 4 in the outer.



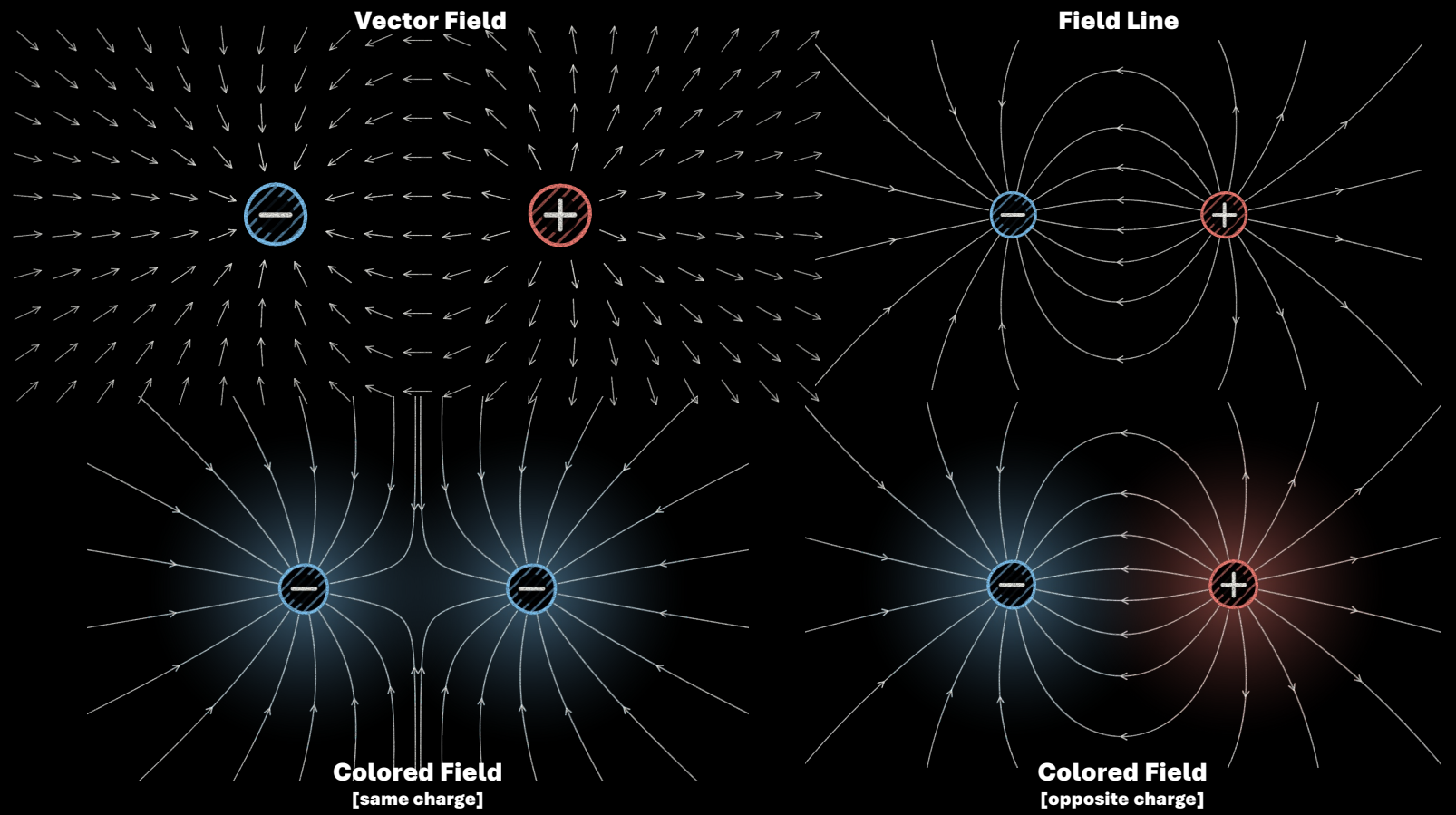
Carbon Atom



https://www.foxxlifesciences.com/pages/periodic-table-of-elements-chart?srsltid=AU7gw4Vy9OpBGr4WNCwO5xl6PzIUoMfdclx6ceMDlvLvYAmgM7SnPw4D

PHYSICS OF LIGHT | ELECTRIC FIELD

When we take all interactions, repelling and attracting, into account, we introduce the electric field. We can picture it as a vector field, field lines, or a colored field, where blue is negative and red is positive.



PHYSICS OF LIGHT | SPECIAL RELATIVITY

A metal wire contains stationary positive ions and drifting electrons, whose motion produces an electric current. In this idealized model, positive and negative charge per unit length balance, making the wire neutral, and the electric field outside is zero. A proton at rest beside it therefore feels no electric force and, because it is stationary in this frame, no magnetic force.

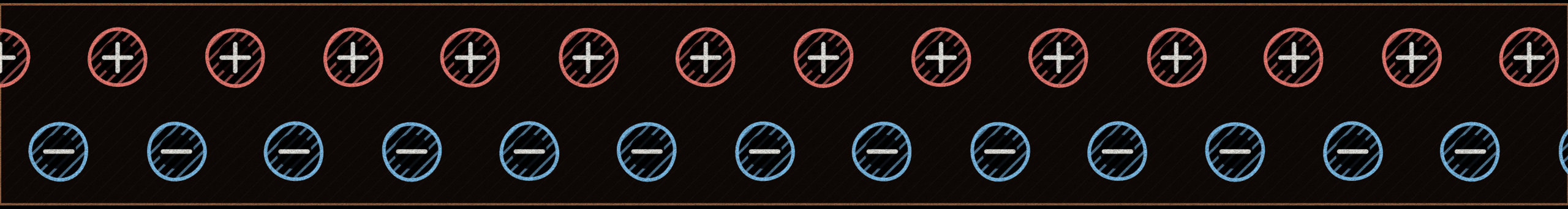
Observer's frame of reference (at rest)



PHYSICS OF LIGHT | SPECIAL RELATIVITY

Now the proton moves along the wire at the same speed and direction as the electron drift, opposite to conventional current. The wire remains neutral, but its magnetic field pushes the moving proton away from the wire, deflecting it downward. In this frame, we describe this sideways push as a magnetic force.

Observer's frame of reference (moving charge)



PHYSICS OF LIGHT | SPECIAL RELATIVITY

Now we move with the proton's initial velocity. The proton is momentarily at rest, and the electrons have no net drift, while the positive ions move the other way. If we ignored relativity, we would expect the wire to remain neutral and incorrectly predict no force. Yet both observers must describe the same deflection.

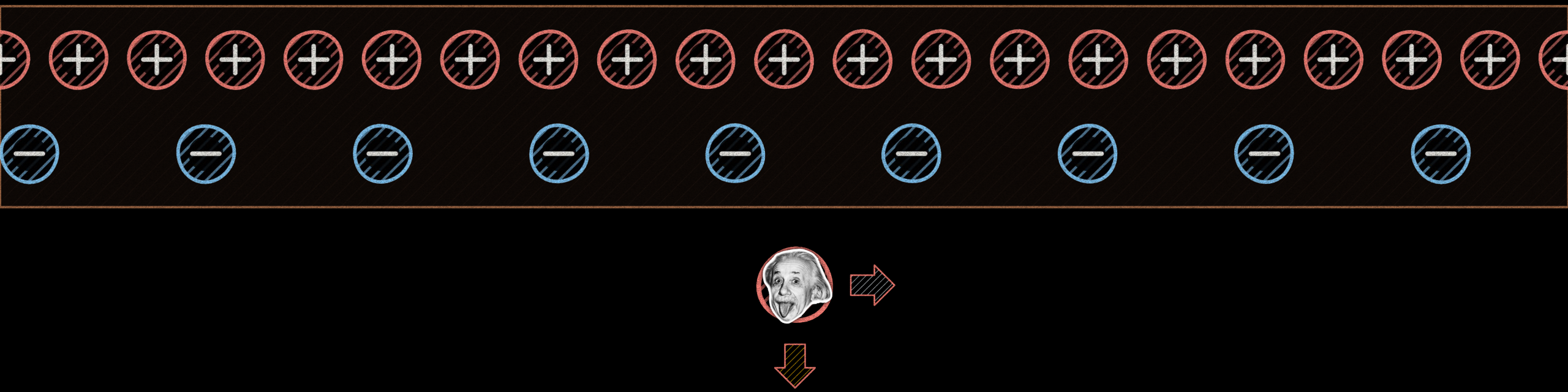
Object's frame of reference



PHYSICS OF LIGHT | SPECIAL RELATIVITY

Special relativity changes the measured spacing between charges. In the proton's frame, moving ions are closer together and electrons farther apart than in the wire's frame. The resulting net positive charge repels the proton through an electric force, explaining the magnetic force seen in the wire's frame. **Magnetism is electricity plus relativity**.

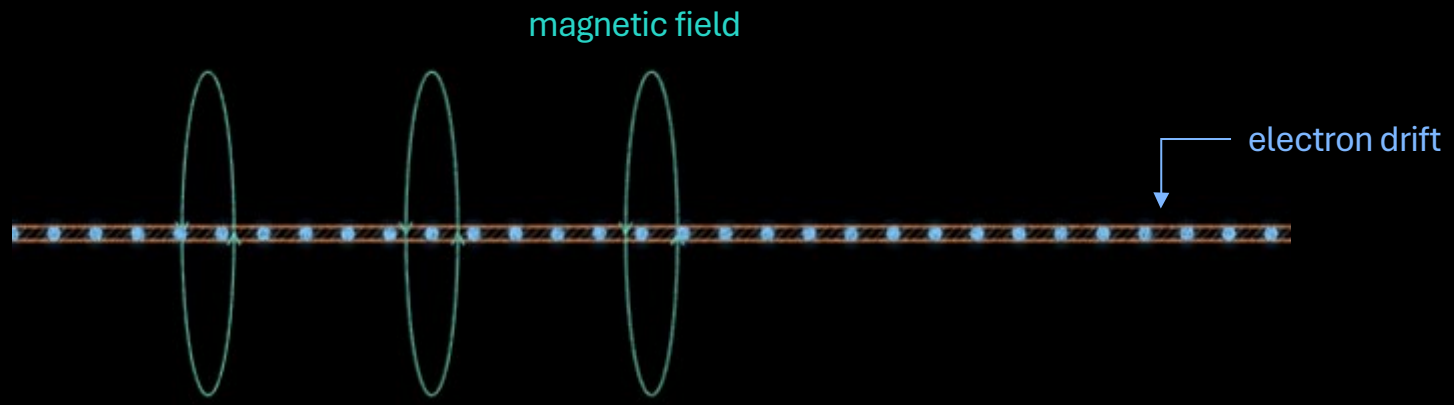
Special relativity



PHYSICS OF LIGHT | MAGNETIC FIELD

To reiterate, the electric field of a moving charge differs from that of a stationary charge, as described by special relativity. A charge moving relative to an observer also produces a magnetic field, which determines the magnetic force it exerts on other moving charges.

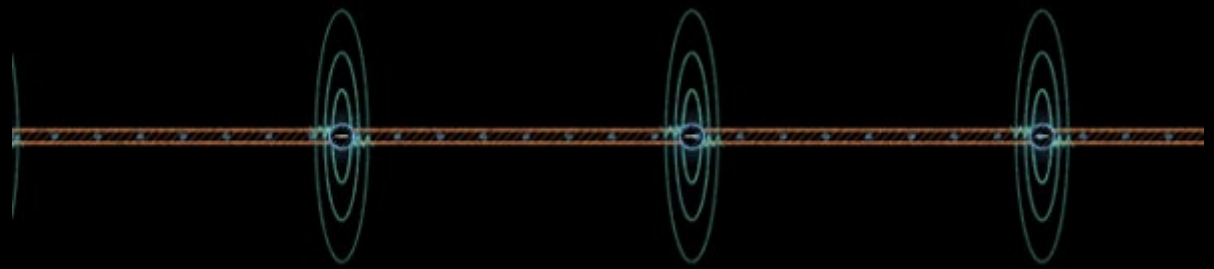
We can represent the charge's motion with a direction vector and visualize its magnetic field as circular field lines around its line of motion, with arrows indicating the field's direction.

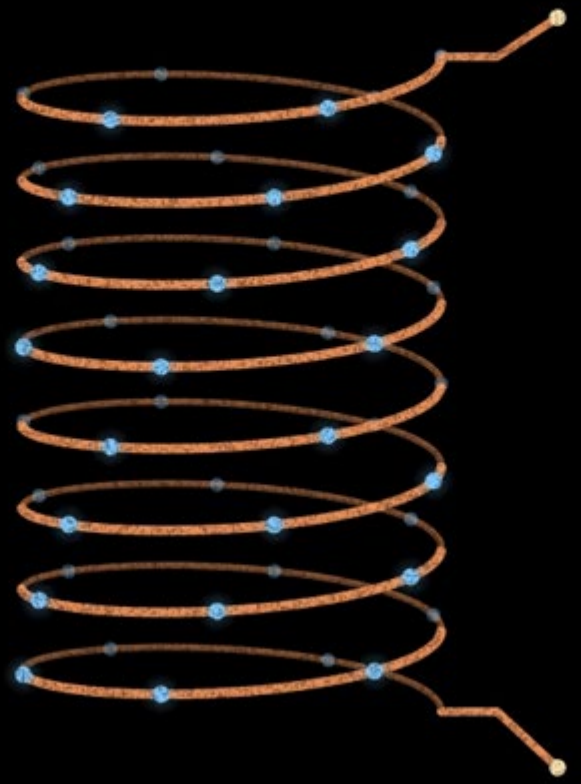


PHYSICS OF LIGHT | MAGNETIC FIELD

Let’s get a more intuitive understanding of this. Imagine winding a wire into a series of loops. This creates a coil, also called a solenoid. When connected to a power source, electrons drift through the wire, following its turns. This movement produces an electric current.

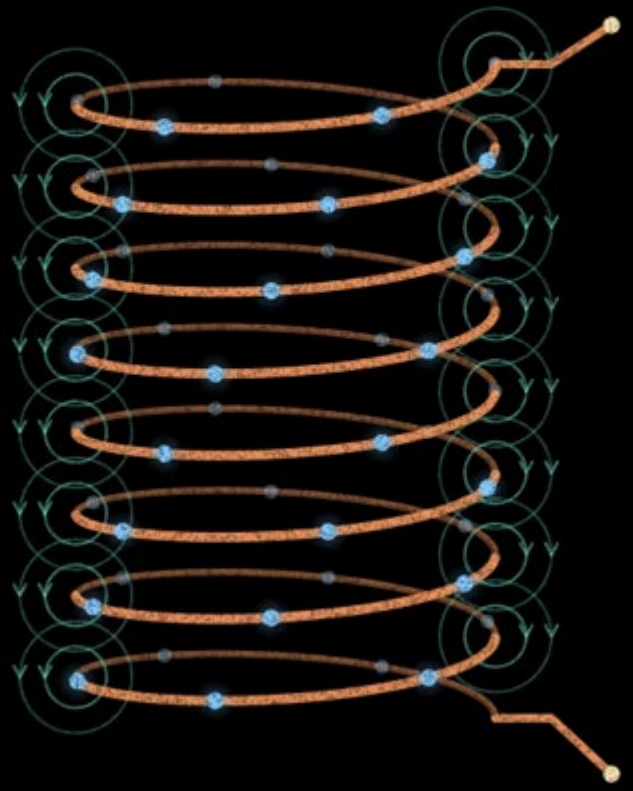
Each moving electron contributes a magnetic field around its direction of motion, and thus itself. We represent this field using circular lines as before, with the arrows showing the field’s direction. The electrons themselves continue along the wire. As the wire bend around the coil, the direction of their motion changes, and so does the orientation of these circular fields.





PHYSICS OF LIGHT | MAGNETIC FIELD

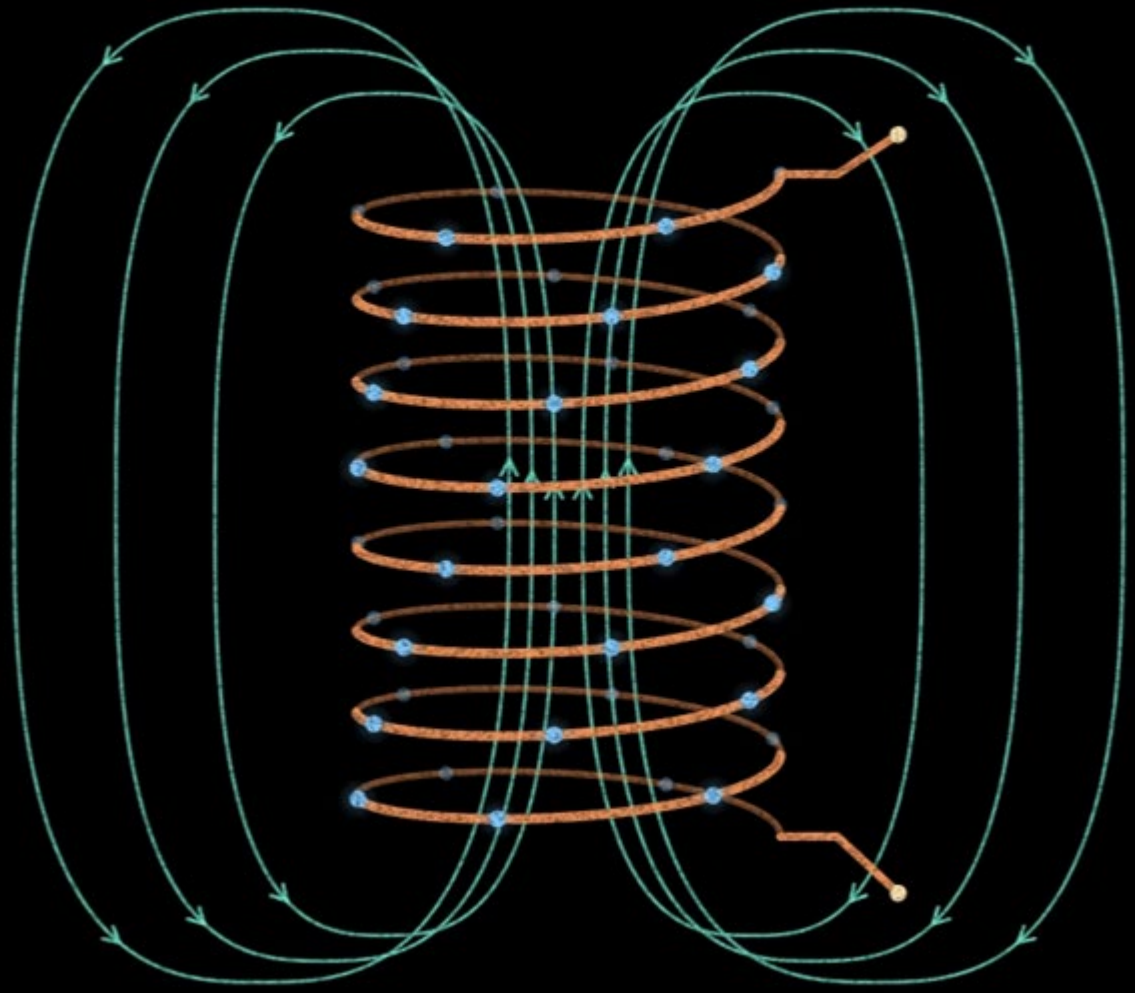
All these magnetic fields are present at the same time, and their contributions add together. Where they point in the same direction, they reinforce each other. Inside the coil, contributions from the different turns line up, producing a stronger field along the coil's axis.



PHYSICS OF LIGHT | MAGNETIC FIELD

All these magnetic fields are present at the same time, and their contributions add together. Where they point in the same direction, they reinforce each other. Inside the coil, contributions from the different turns line up, producing a stronger field along the coil's axis.

Looking at the coil as a whole, we see one combined magnetic field extending through and around it. Near the center of a long coil, this field is approximately uniform and parallel to its axis. Outside, the field lines curve around and return, forming closed loops.

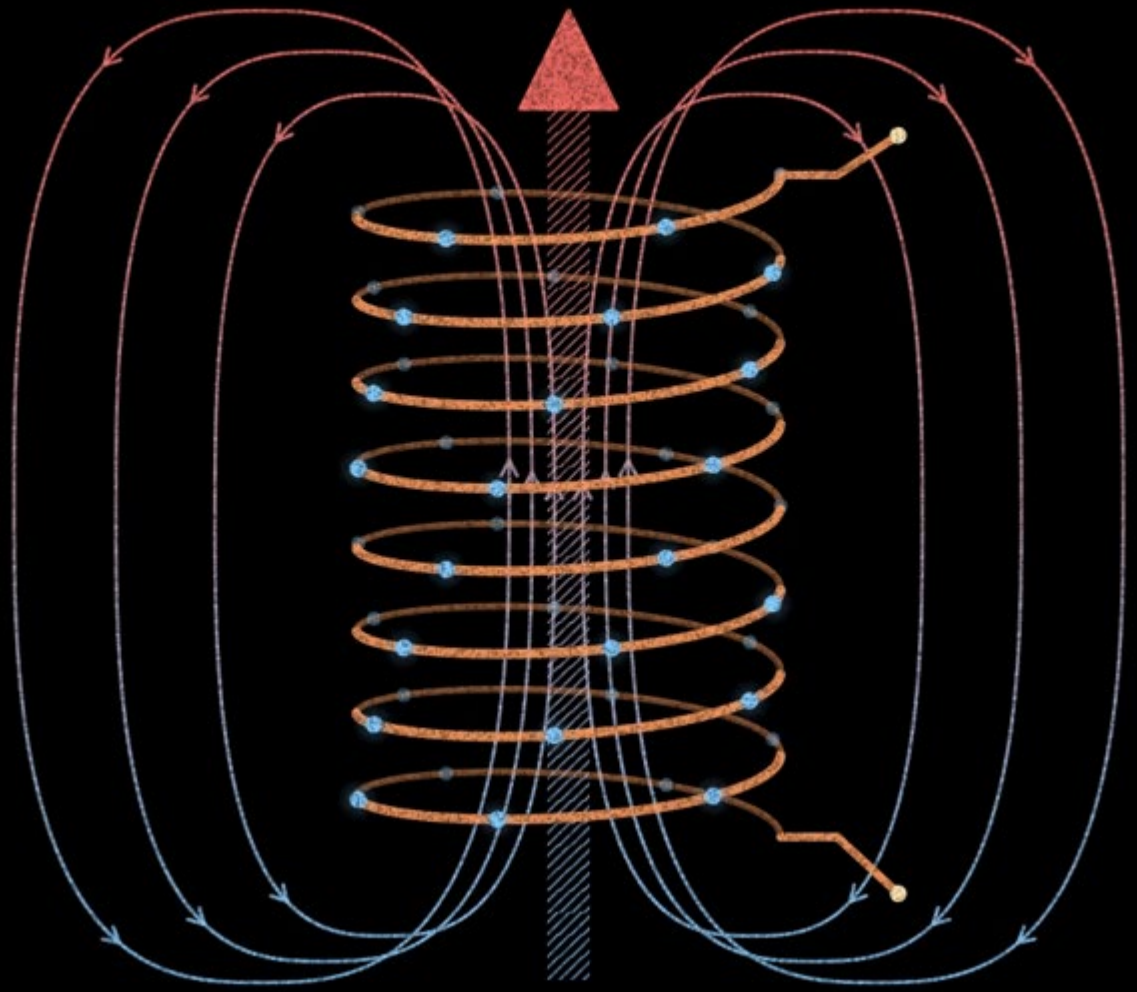


PHYSICS OF LIGHT | MAGNETIC FIELD

All these magnetic fields are present at the same time, and their contributions add together. Where they point in the same direction, they reinforce each other. Inside the coil, contributions from the different turns line up, producing a stronger field along the coil's axis.

Looking at the coil as a whole, we see one combined magnetic field extending through and around it. Near the center of a long coil, this field is approximately uniform and parallel to its axis. Outside, the field lines curve around and return, forming closed loops.

The coil now behaves like an **electromagnet**, with a north pole and a south pole. Reversing the current would reverse the magnetic field and swap the poles. The electrons keep moving, but with a steady current the overall magnetic field remains constant in time.

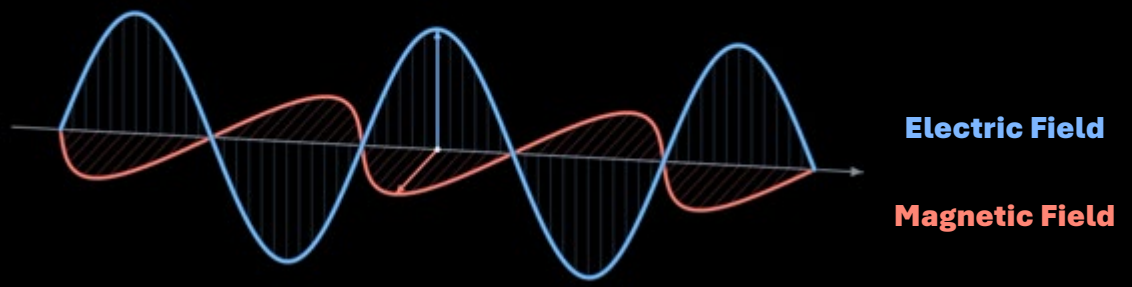


PHYSICS OF LIGHT | ELECTROMAGNETIC FIELD

This means, the electric field and magnetic fields are inseparable, hence why they are often united in a single concept called the electromagnetic field. The field behaves and evolves in a well-define way which can be summarized in four fundamental principles called Maxwell's equations:

1. Electric charges create an electric field, spreading out of positive charges and into negative ones.
2. There are no lone magnetic poles, so field lines always loop back and north and south come together.
3. A changing magnetic field creates an electric field, e.g. driving a current through a nearby coil.
4. A changing electric field creates a magnetic field.

The last two can be combined into the important and most relevant part. Because a changing electric field makes a magnetic field and a changing magnetic field makes an electric field, the two keep regenerating each other and travel off together as an **electromagnetic wave**.



PHYSICS OF LIGHT

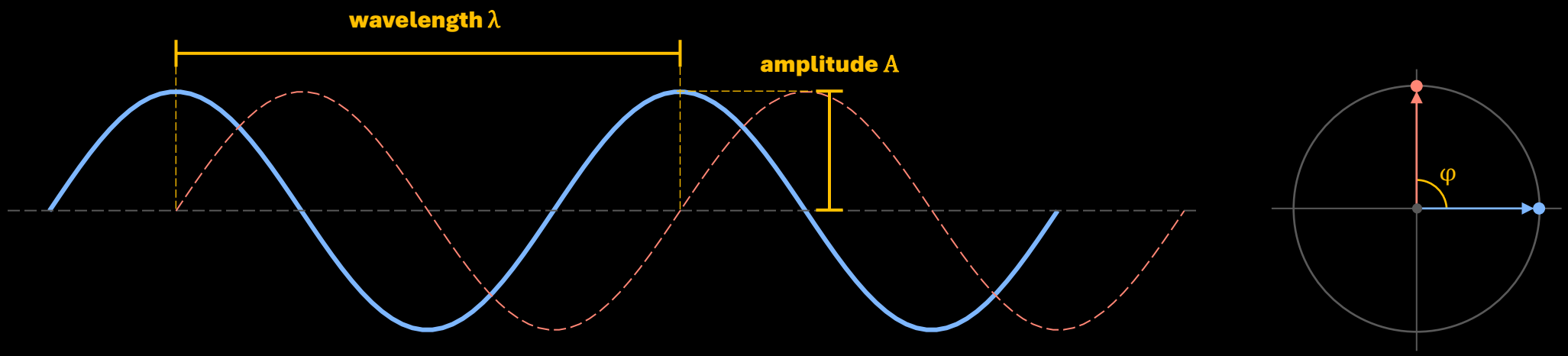


LIGHT IS AN ELECTROMAGNETIC WAVE

PHYSICS OF LIGHT | WAVE CHARACTERISTICS

Waves have some interesting characteristics:

- Wavelength (λ): the distance between two consecutive speaks.
- Amplitude (A): the maximum displacement from the resting position.
- Period (T): time taken for one cycle, measured in seconds (s).
- Frequency (f): the number of complete cycles (oscillations) per second, measured in hertz (Hz).
- Phase (φ): the position within a cycle at a given moment, measured as an angle (in radians $[0, 2\pi]$).

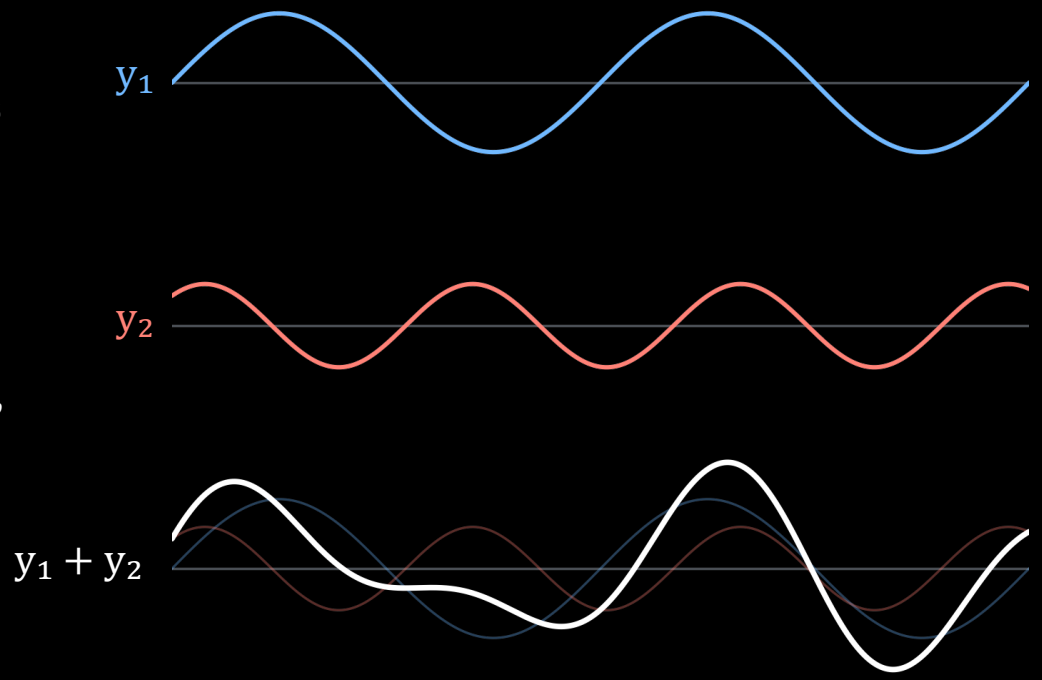


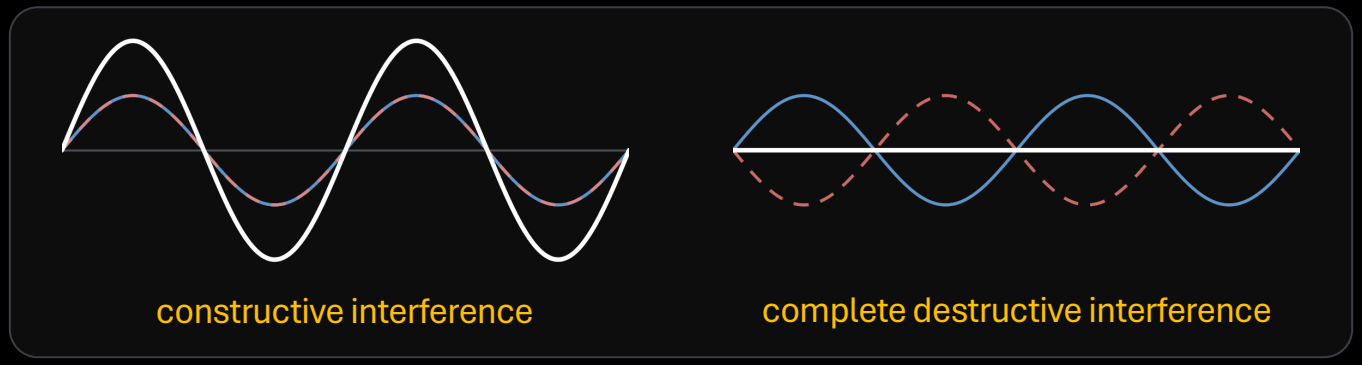
PHYSICS OF LIGHT | INTERFERENCE

When two waves meet, they add up. This is the principle of superposition. At every point, the total displacement is the sum of the individual displacements:

$$y(x, t) = y_1(x, t) + y_2(x, t)$$

Interference refers to what happens when waves overlap, including constructive reinforcement and destructive cancellation. Thus, the resulting wave is the sum calculated by superpositions and exhibits the interference.

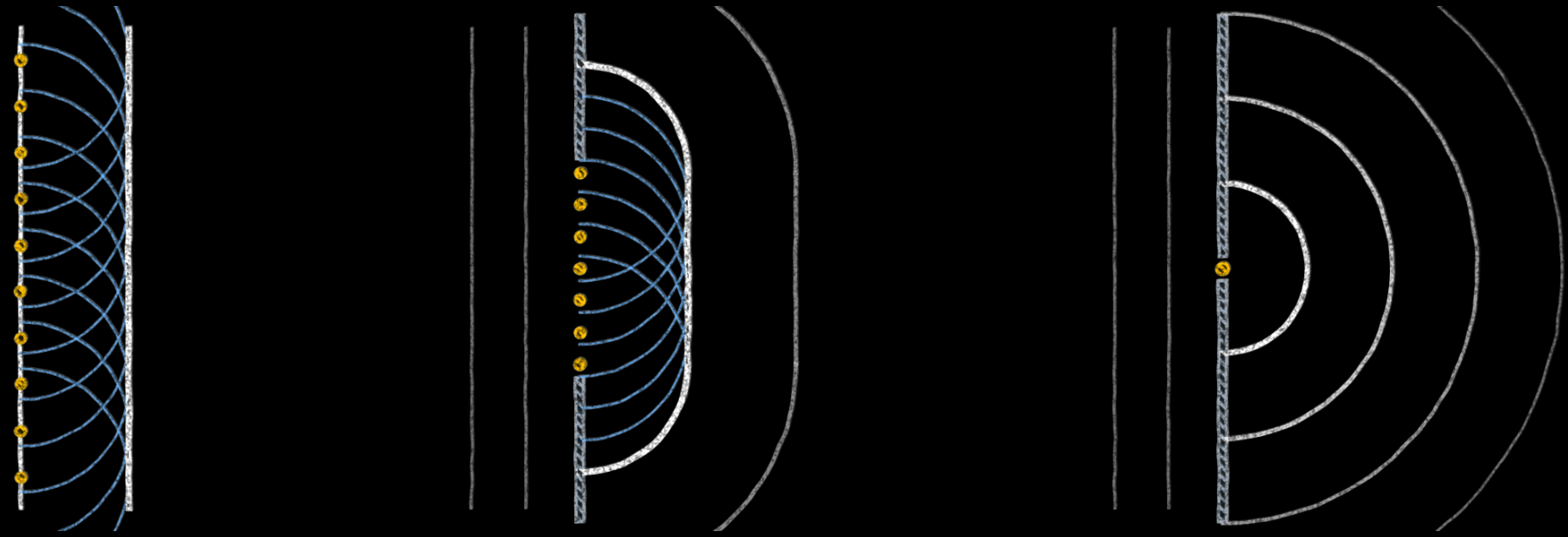




PHYSICS OF LIGHT | DIFFRACTION

A wave passing through a small opening or around an edge bends and spreads out. This is diffraction.

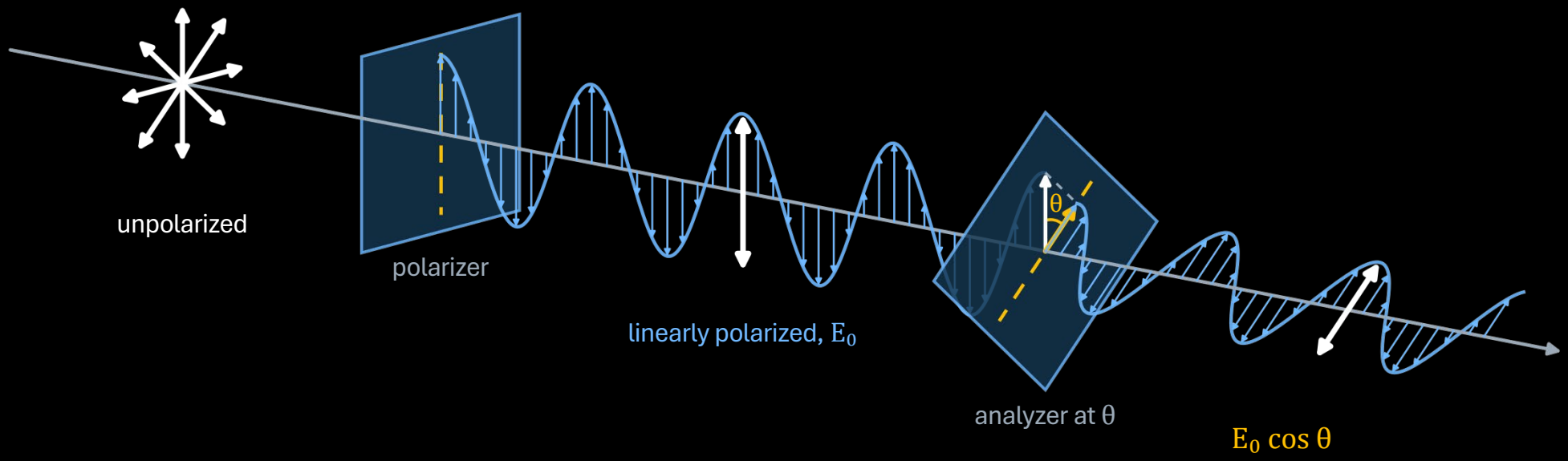
Huygens' principle explains why. Every point on a wavefront emits a small circular wavelet, and together they form the next wavefront. At an opening, the wall blocks some wavelets, so the wave curls around the edges. The wavelets that pass through interfere, creating bright and dark bands.



PHYSICS OF LIGHT | POLARIZATION

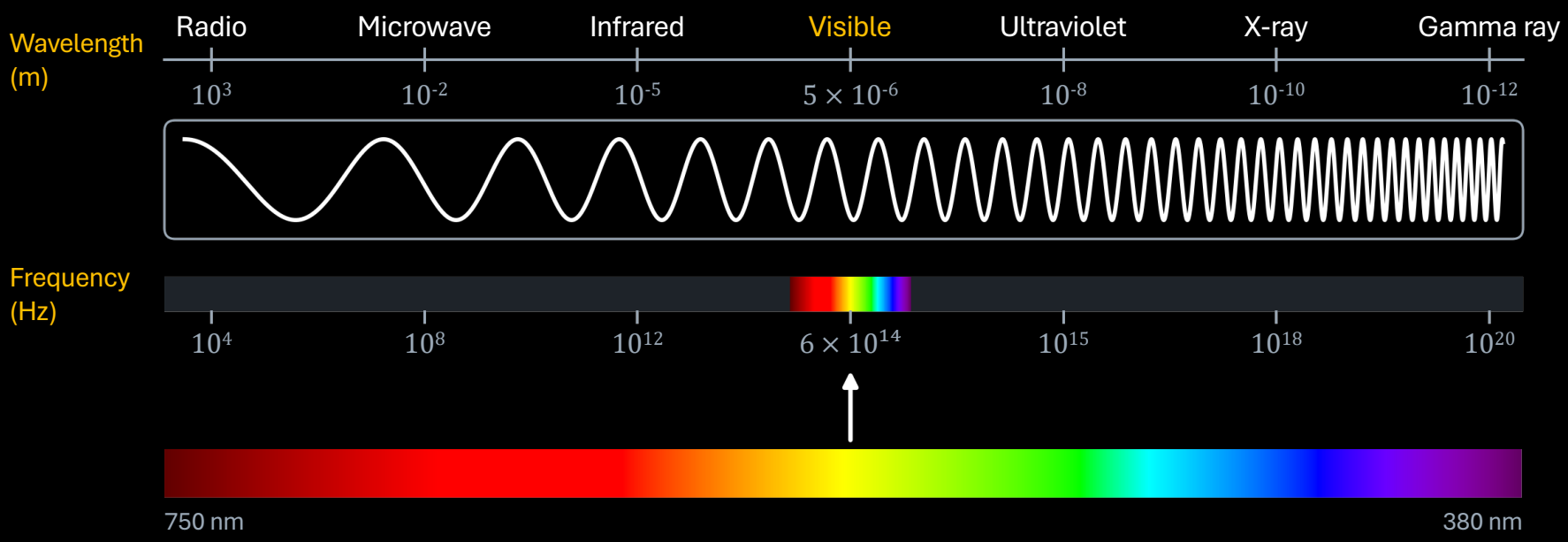
Light is a transverse wave. Its electric field oscillates perpendicular to the direction of travel, and the direction of that oscillation is the polarization.

Sunlight and light bulbs are unpolarized, meaning the direction changes randomly. A polarizing filter only lets one direction through, producing linearly polarized light. Reflection also polarizes light partially, which is why polarized sunglasses can block the glare from water.



PHYSICS OF LIGHT | SPECTRUM

Light is a mix of waves with different frequencies (and so different wavelengths, since $c = \lambda \cdot f$, where c is speed of light, λ is the wavelength and f the frequency), and a spectrum shows how much power each frequency contributes. The full electromagnetic spectrum runs from radio waves (low frequency) to gamma rays (high frequency), and visible light is only the narrow band in between, roughly 380–750 nm, that our eyes can detect. Within that band, frequency is what we perceive as color, from red at the low-frequency end to violet at the high-frequency end.

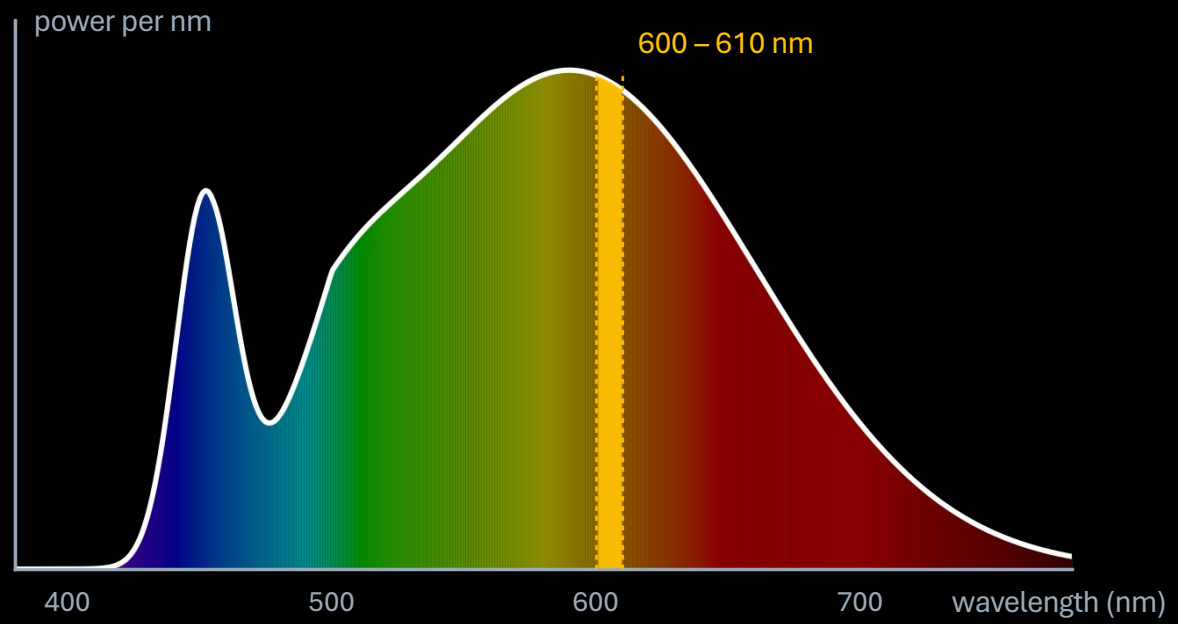


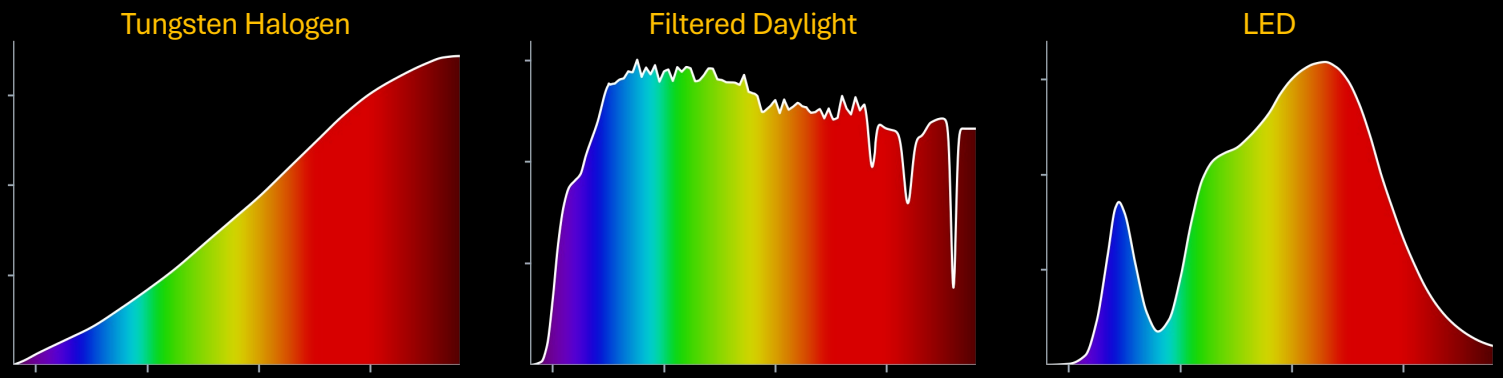
PHYSICS OF LIGHT | DISTRIBUTION

As seen in previous slide, a spectrum describes how light is distributed across wavelengths.

The spectral power distribution (SPD) gives the power per wavelength interval (in $\mathrm{W/nm}$).

The height at a wavelength tells how much power sits around that wavelength. The power in a small band is the area under the curve in that band. So, the total power is the area under the whole curve.

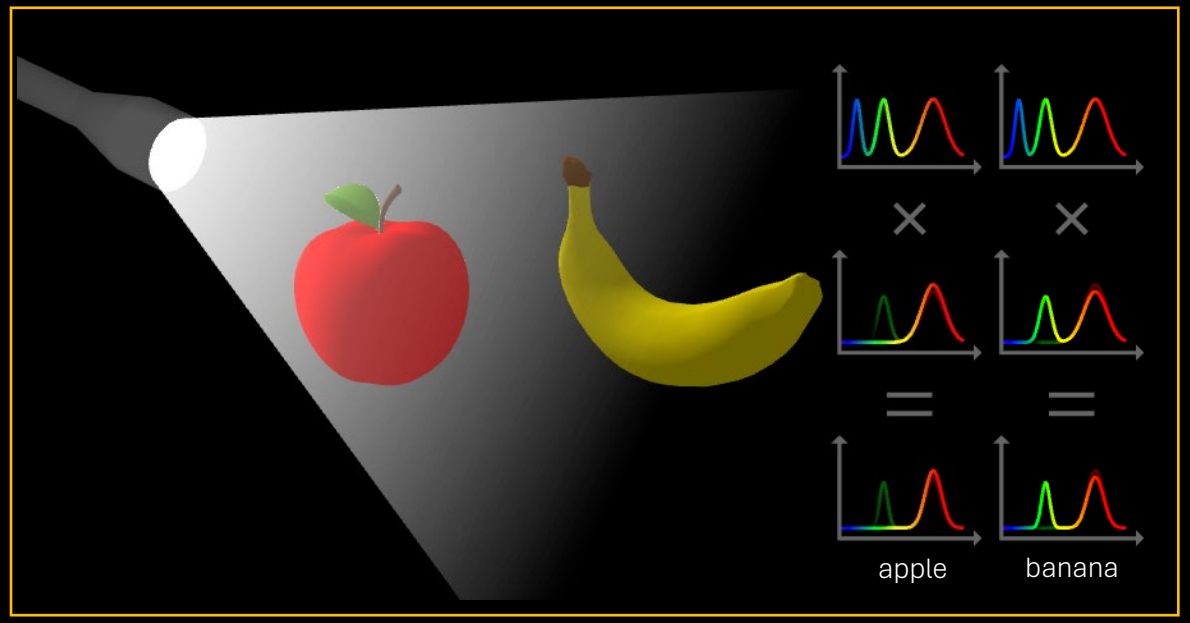




https://research.ng-london.org.uk/scientific/spd/

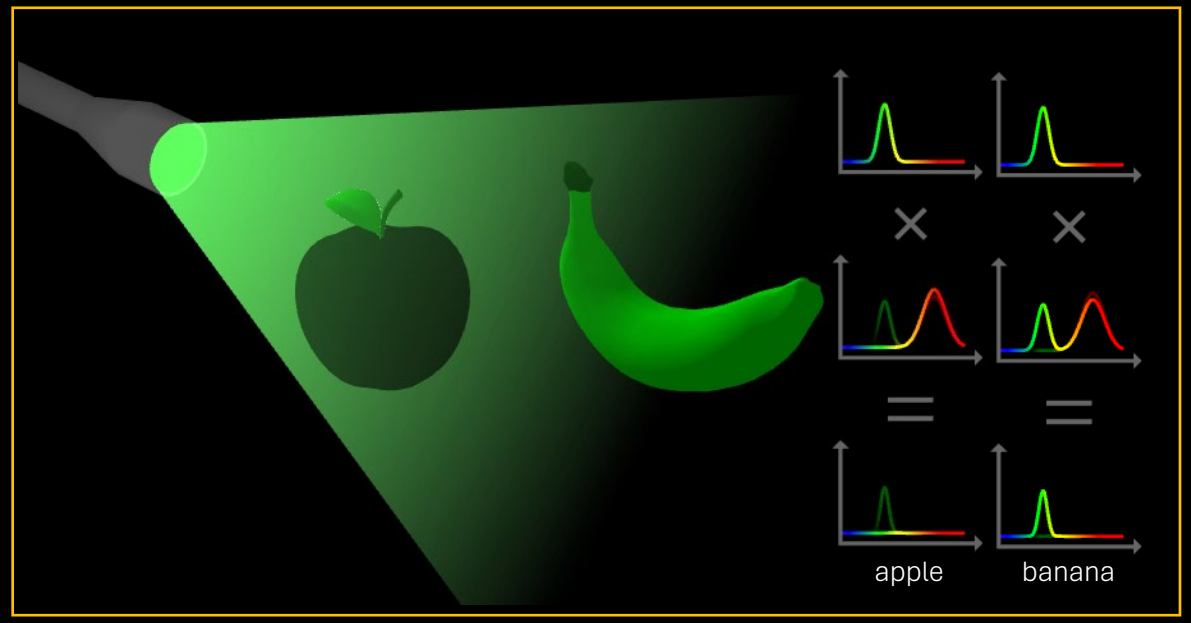
PHYSICS OF LIGHT | METAMERISM

While the spectral power distribution shows how the energy of light is spread across wavelengths, the spectral reflectance curve describes an object. For every wavelength it gives the fraction of the incident (incoming) light the object reflects. Two objects with different reflectance curves can still look identical under one light, because our eyes reduce the reflected light to just three cone responses. Under another light that match can break, an effect called metamerism. Below is an example of illuminant metamerism.



White Light

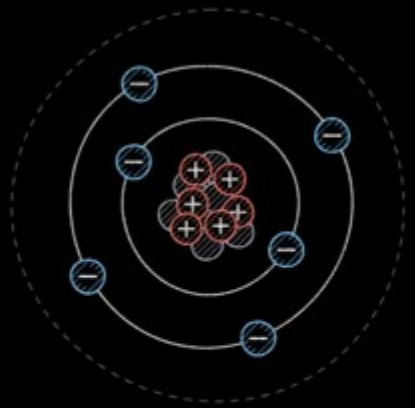
https://imadrahmoune.com/pbr/



Green Light

PHYSICS OF LIGHT | INTERACTION

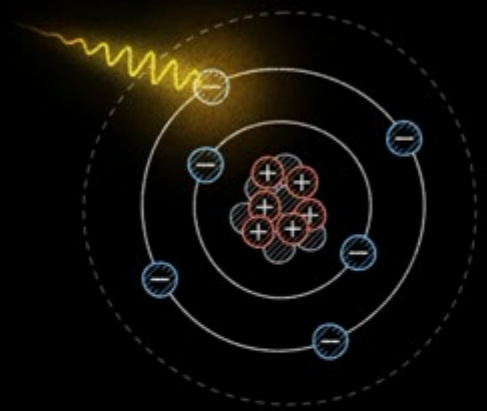
When we talk about the interaction of light with matter, what really happens when light hits an object?
It starts inside the atom. Electrons cannot have just any energy, only specific energy levels. Carbon e.g., has six electrons, two in the inner level and four in the outer one. In the ground state every electron sits in the lowest level available to it, the most stable situation.



GROUND STATE
The electron sits in its lowest energy level.
The rings are levels, not paths!

PHYSICS OF LIGHT | INTERACTION

So far, we described light as an electromagnetic wave, and that explains how it travels. When it exchanges energy with matter, however, it does so in small packets called photons. A photon hands over its energy as a whole, never partly. If that energy lifts an electron exactly to a higher level, the photon disappears and its energy is stored in the atom.

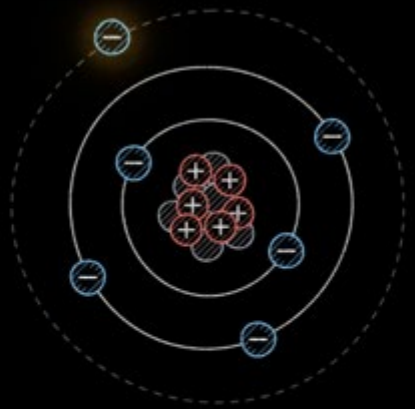


ABSORPTION

A photon with exactly the right energy is absorbed.

PHYSICS OF LIGHT | INTERACTION

The atom now holds more energy than it can keep. The excited state lasts only a tiny fraction of a second, so to us absorption and release seem instant. There is no fixed moment for the fall back. It is a matter of probability, and what happens next decides whether we see light or feel heat.

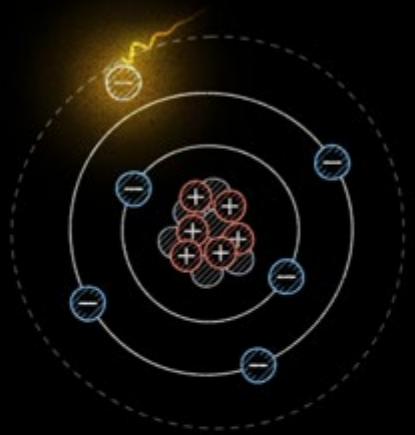


EXCITED STATE

Higher energy level, unstable and short lived.

PHYSICS OF LIGHT | INTERACTION

When the electron drops back, the atom releases the energy as a new photon. Its energy equals the gap, so its color is set by the atom itself, not by the incoming light. This is why every element has its own emission spectrum, which is used to identify elements in stars. The new photon leaves in a random direction.



EMISSION

The electron drops back and a new photon leaves.

PHYSICS OF LIGHT | INTERACTION

In most materials the energy does **not** leave as light. The atom passes it on to its neighbors through collisions and vibrations of the bonds. More vibration means higher temperature. This is why dark surfaces get hot in the sun, since they absorb most visible light and turn it into heat instead of sending it back.

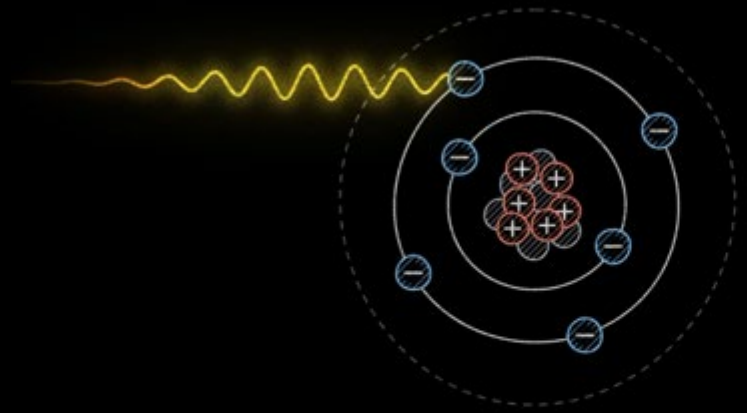


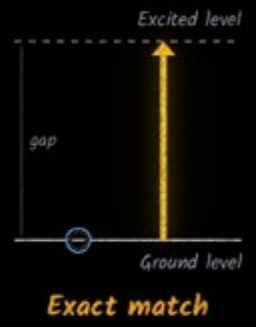
HEAT

The energy becomes vibration and spreads to the surroundings.

PHYSICS OF LIGHT | INTERACTION

A photon's energy depends only on its frequency, $E = h \times f$, where h is Planck's constant. Brighter light means more photons, not stronger ones, so intense red light cannot do what a single photon of the right frequency can. Absorption needs resonance, a match with the gap. In molecules and solids the levels widen into bands, so whole ranges of frequencies are absorbed. What is not absorbed gives objects their color.



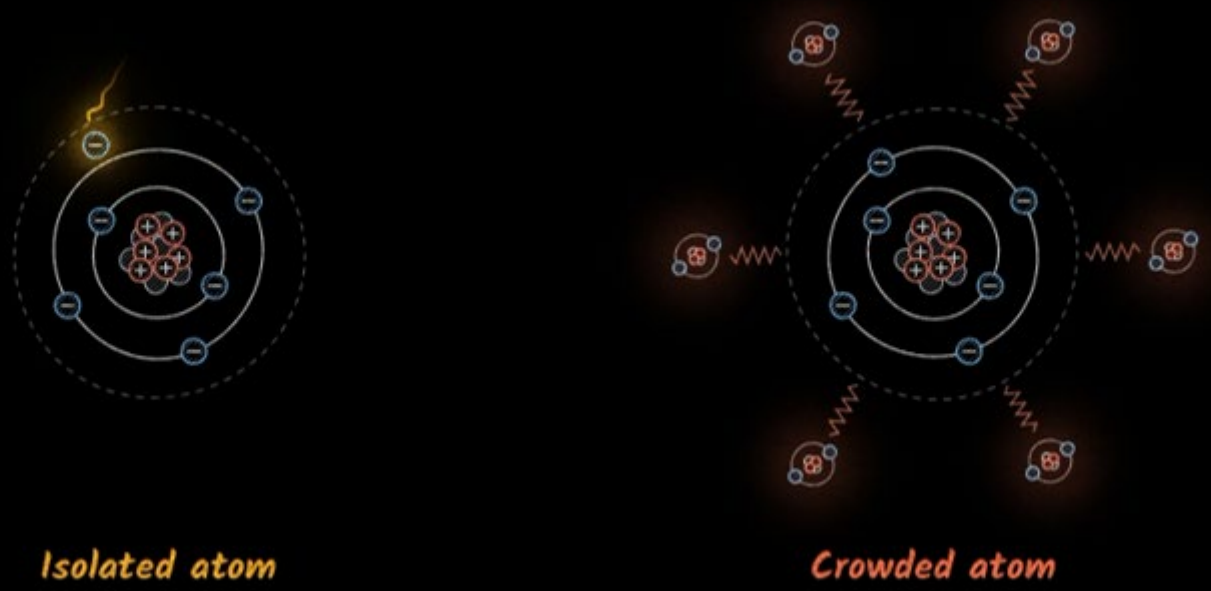


FREQUENCY MUST MATCH THE GAP

Frequency must match the gap. Below it electrons only shake, at it the photon is absorbed

PHYSICS OF LIGHT | INTERACTION

Which path wins depends on how quickly each one can happen. Atoms in a thin gas rarely collide, so they glow, as in a fluorescent tube. In dense materials heat almost always wins. Some rigid molecules, such as fluorescent dyes and phosphors, still emit but lose a little energy first, so their light shifts toward longer wavelengths. In rendering we therefore treat absorbed light as lost energy.

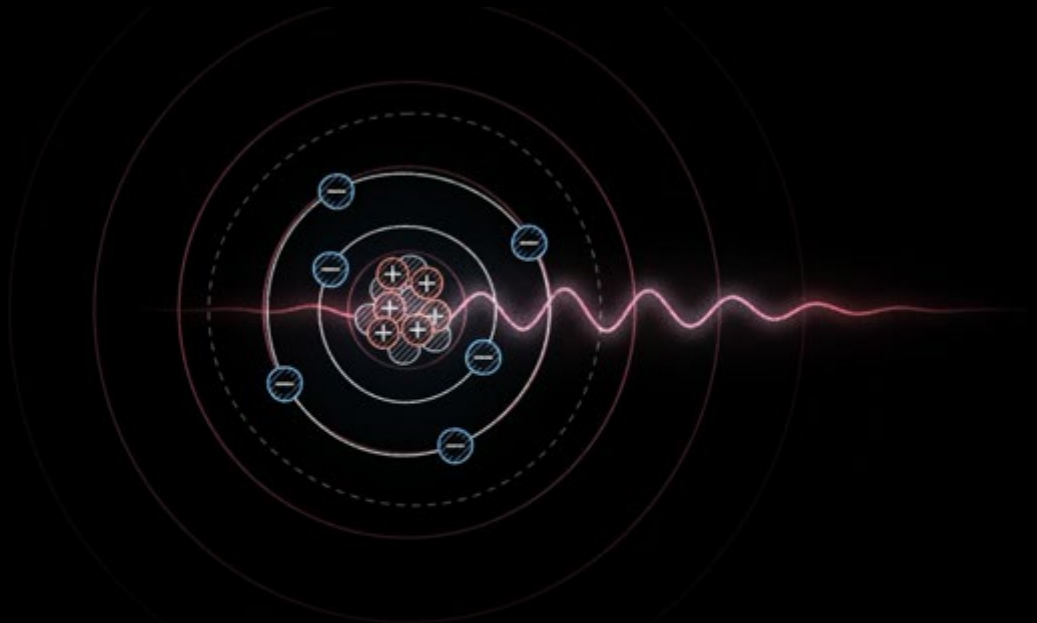


EMISSION || HEAT

Emission or heat, the faster path wins!
Isolated atoms emit light, crowded atoms lose it as heat

PHYSICS OF LIGHT | INTERACTION

Most visible light that hits glass, water or air matches no gap at all. The light's oscillating electric field pushes the negative electron cloud back and forth against the positive nucleus. This forced oscillation turns the atom into a tiny antenna. An accelerating charge radiates, so the atom sends out light right away. Because nothing is absorbed, glass and water are transparent to visible light.

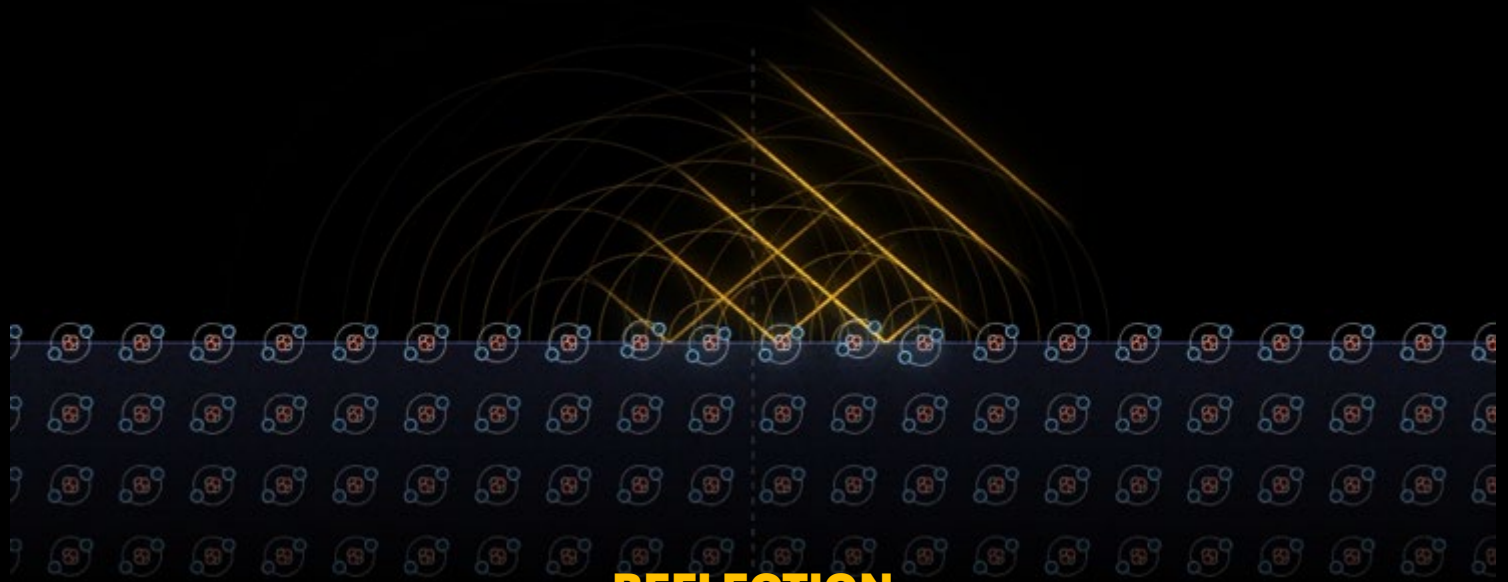


NO MATCHING ENERGY

Electrons shake in place and re-radiate the same light.

PHYSICS OF LIGHT | INTERACTION

Each surface atom sends out a wavelet. These wavelets interfere, similar to diffraction, adding up where crests meet crests and cancelling where crests meet troughs. They line up into a new wavefront in only one direction, which gives the law that the **angle of reflection equals the angle of incidence**. Metals reflect so strongly because their free electrons follow almost every visible frequency.

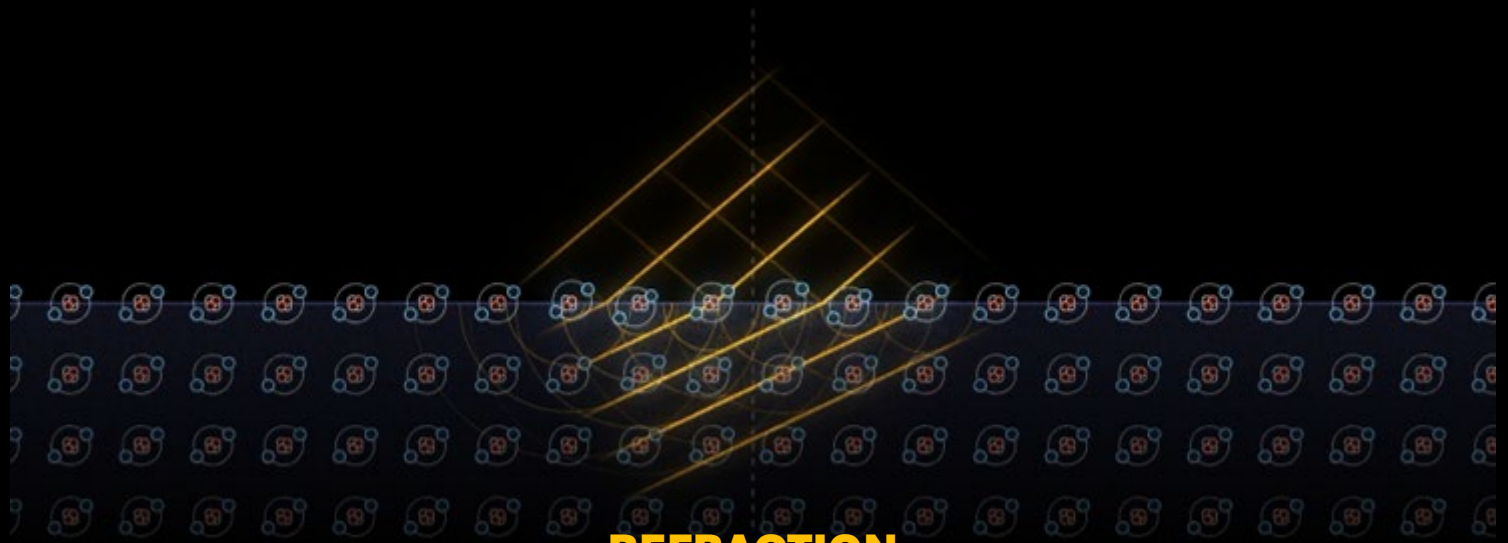


REFLECTION

Re-radiated waves only add up in the mirror direction.

PHYSICS OF LIGHT | INTERACTION

Inside the material the re-radiated waves lag slightly behind the incoming light. Added together, the wavefront moves forward more slowly, by a factor called the index of refraction. Entering at an angle, one side slows down first, so the light changes direction. The frequency stays the same, and only the wavelength gets shorter.



REFRACTION

Delayed re-radiation slows the wave and bends it.

PHYSICS OF LIGHT | INTERACTION

Reflection and refraction are not separate events. At every surface the light splits, and both happen at the same time. The angle decides the ratio. Looking straight at glass, only about 4% reflects and the rest passes through. Toward grazing angles, reflection quickly takes over. The direction of each part follows from the earlier slides, and how much goes each way is described by the Fresnel equations, which we will come back to later.



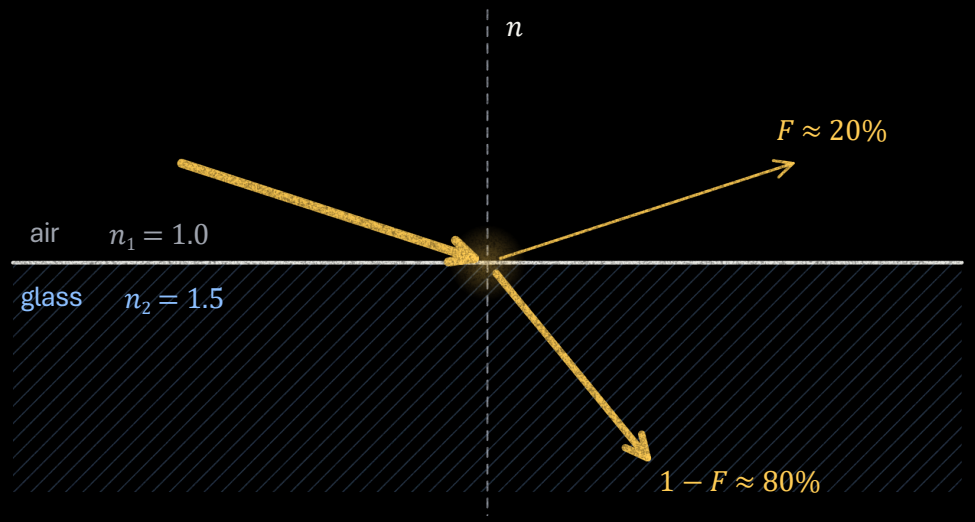
FRESNEL

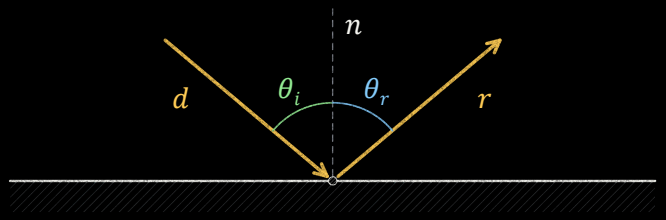
The angle decides how much light reflects and how much refracts/transmits.

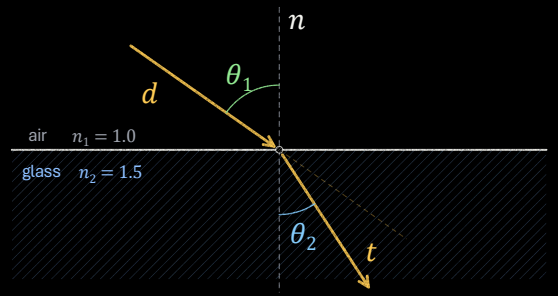
PHYSICS OF LIGHT | INTERACTION

Important to understand:

- **Fresnel** decides **how much light goes each way**. The reflected fraction is F, and $1 - F$ refracts.
- The **directions** follow from two separate laws:
  - The **law of reflection** → light leaves at the same angle it arrives, mirrored around the normal.
  - The **law of refraction** (Snell's Law) → refracted light bends toward the normal when it enters a denser medium, and away from it when it leaves.

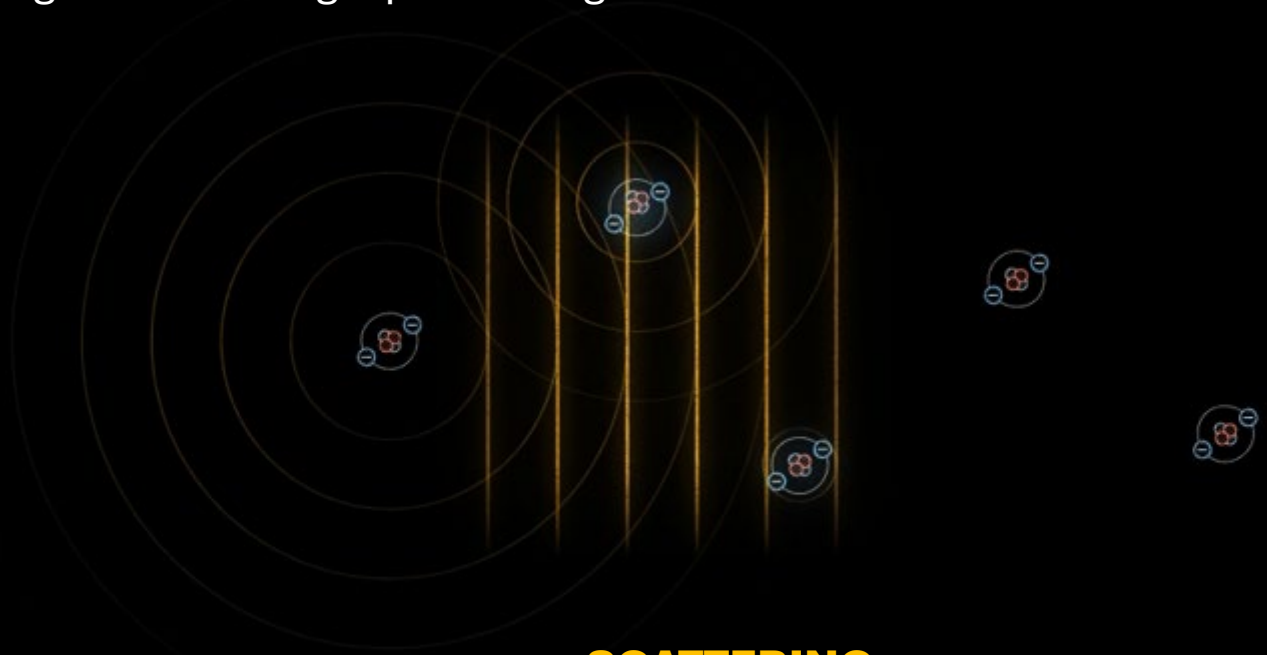






PHYSICS OF LIGHT | INTERACTION

In a gas the molecules are far apart and randomly placed, so their wavelets never line up into one clean wavefront. Some light is therefore sent sideways. Small molecules scatter short wavelengths far more strongly than long ones, called Rayleigh scattering. This is why the sky looks blue, and why the sun turns red at sunset, when blue is scattered away along the much longer path through air.



SCATTERING

Random molecules re-radiate in all directions.

PHYSICS OF LIGHT | ATOM VISUALIZATION



https://youtu.be/W2Xb2GFK2yc?si=su1FwqrbgsoVrM0J

ACT I

Light & Matter

ACT II

From Physics to Model

ACT III

Inside the Equation

ACT IV

BRDFs

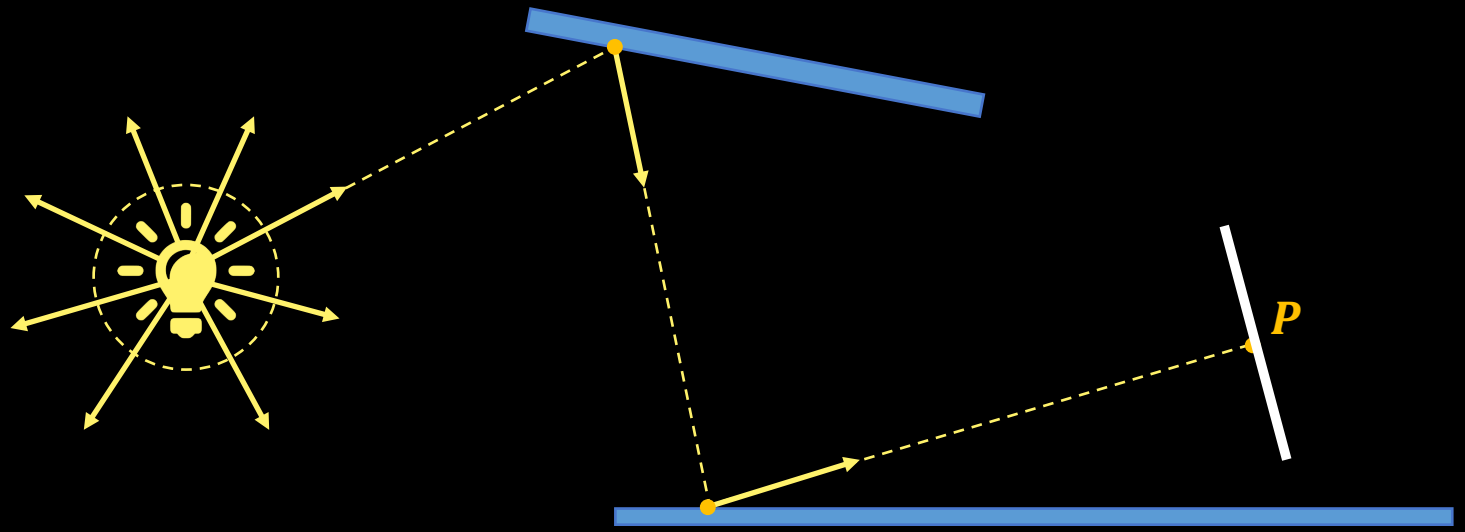
ACT V

Surface Variation

MODELING LIGHT | RAY OPTICS

We have seen that light behaves as a wave and as a stream of photons, all moving at the speed of light.

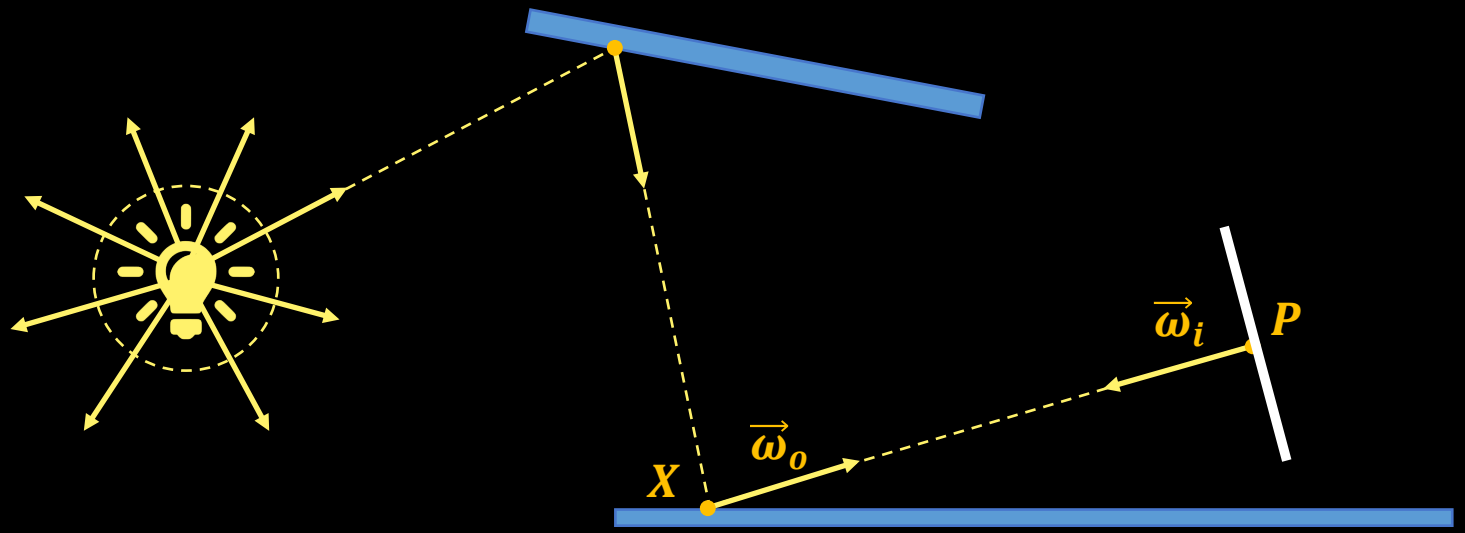
For rendering we go back to the simplest model from the start, ray optics. Most objects are huge compared to the wavelength of light, so light travels in straight lines and we can describe it with rays.



MODELING LIGHT | RAY OPTICS

We define a light field $L(p, \vec{\omega})$, where p is the position and $\vec{\omega}$ is the direction of propagation of light.
This does not need to be the actual light itself. This field can also represent a surface point which re-radiates light.

When light travels in a vacuum, it does not lose any energy. This is called the conservation of energy.
This can be written as follows: $L(P, \vec{\omega}_i) = L(X, -\vec{\omega}_o)$ where $\vec{\omega} = P - X$.

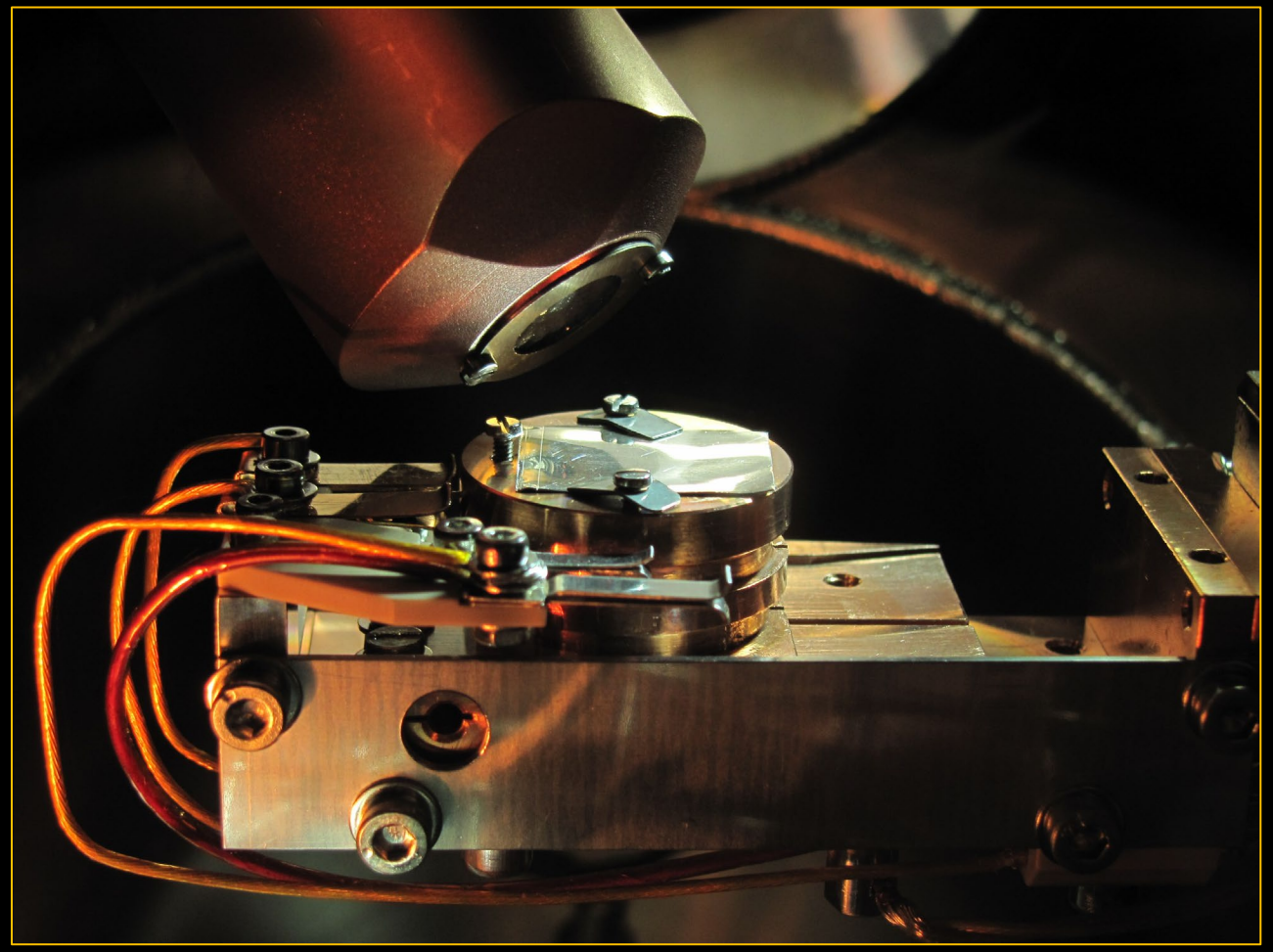


MODELING LIGHT | RADIOMETRY

How can we measure and/or quantify light?

That is the field of radiometry, which gives us units for how much energy light carries, where it arrives and in which direction it travels. A spectrometer measures exactly this, separately for every wavelength.

We are going to ignore per wavelength differences. In other words, our renderer will **not** be a spectral renderer!



https://en.wikipedia.org/wiki/Spectrometer

MODELING LIGHT | RADIOMETRY

Let's start with some units you know from high school physics:

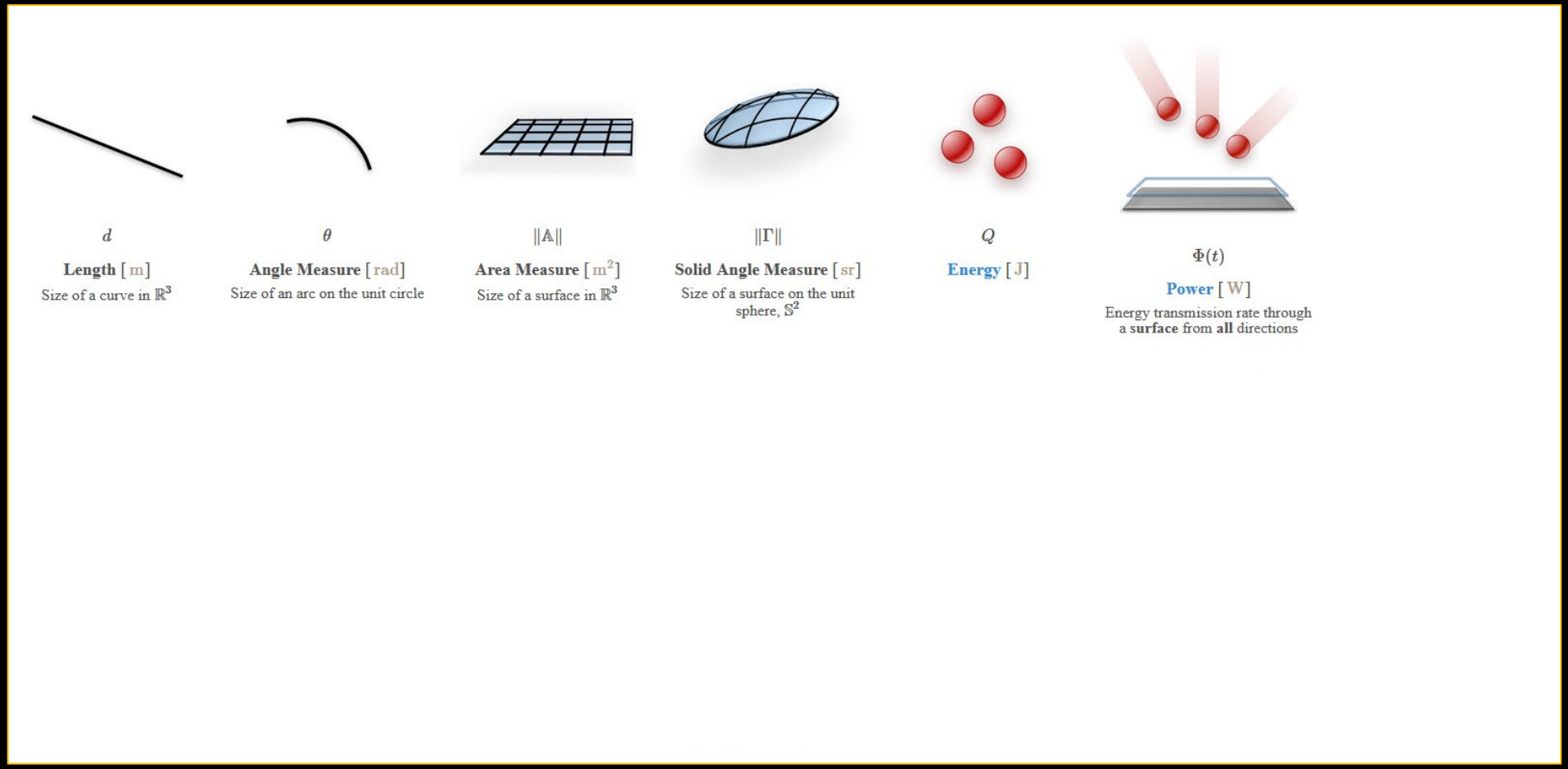
- Mass $\boldsymbol{m}$ is measured in kilograms (kg).
- Distance $\boldsymbol{x}$ is measured in meters (m).
- Time $\boldsymbol{t}$ is measured in seconds (s).
- Acceleration $\boldsymbol{a}$ is measures in meters per second squared (m/s^2).
- Force is expressed as $\boldsymbol{F = ma}$ and is measured in Newtons ($N = kg * m/s^2$).
- Work or Energy is expressed as $\boldsymbol{E = Fx}$ and is measured in Joules ($J = N * m$).

As discussed, our ray models light as a stream of photons, and these photons carry energy. In radiometry this **radiant energy** is written $\boldsymbol{Q}$, and like any other form of energy it is measured in Joules.

We are interested in how much light energy reaches a surface in our virtual world. Photons keep arriving over time, so the total energy keeps growing the longer we wait. Instead, we look at the rate at which energy arrives, the energy per second. This is called **radiant power** or **radiant flux**.

MODELING LIGHT | RADIOMETRY

Graphics Codex – Morgan McGuire



MODELING LIGHT | RADIOMETRY

Radiant flux is also denoted as Φ(phi, pron. “fie”). It is energy per unit time, $\Phi = \frac{dQ}{dt}$, measured in Watts.

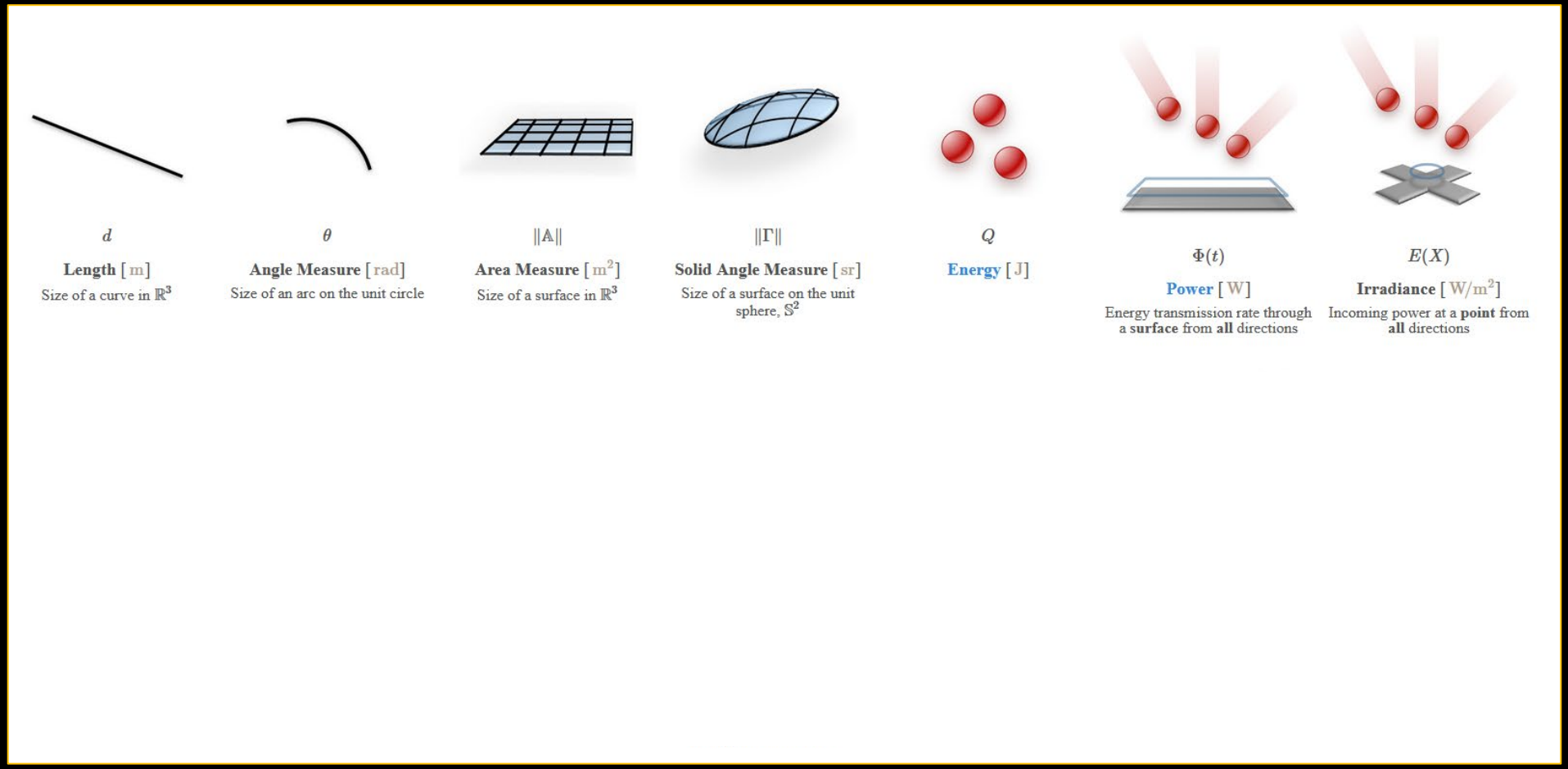
The flux arriving on an entire (oriented) surface $\mathbb{A}$ is written $\Phi(\mathbb{A})$. This is still a flux, measured in Watts.

To describe what happens at a single point, we divide the flux by the area it falls on. This is **radiant flux density**, the energy per unit time and per unit area, measured in W/m^2.

The flux density arriving at a point from all directions is called irradiance E. Irradiance is calculated as $E = \frac{\mathrm{d}\phi}{\mathrm{d}\mathbb{A}}$, a tiny amount of flux divided by the tiny area it lands on, thus also measured as W/m^2.

MODELING LIGHT | RADIOMETRY

Graphics Codex – Morgan McGuire



MODELING LIGHT | RADIOMETRY

Radiant flux is also denoted as Φ(phi, pron. "fie"). It is energy per unit time, $\Phi = \frac{dQ}{dt}$, measured in Watts.

The flux arriving on an entire (oriented) surface $\mathbb{A}$ is written $\Phi(\mathbb{A})$. This is still a flux, measured in Watts.

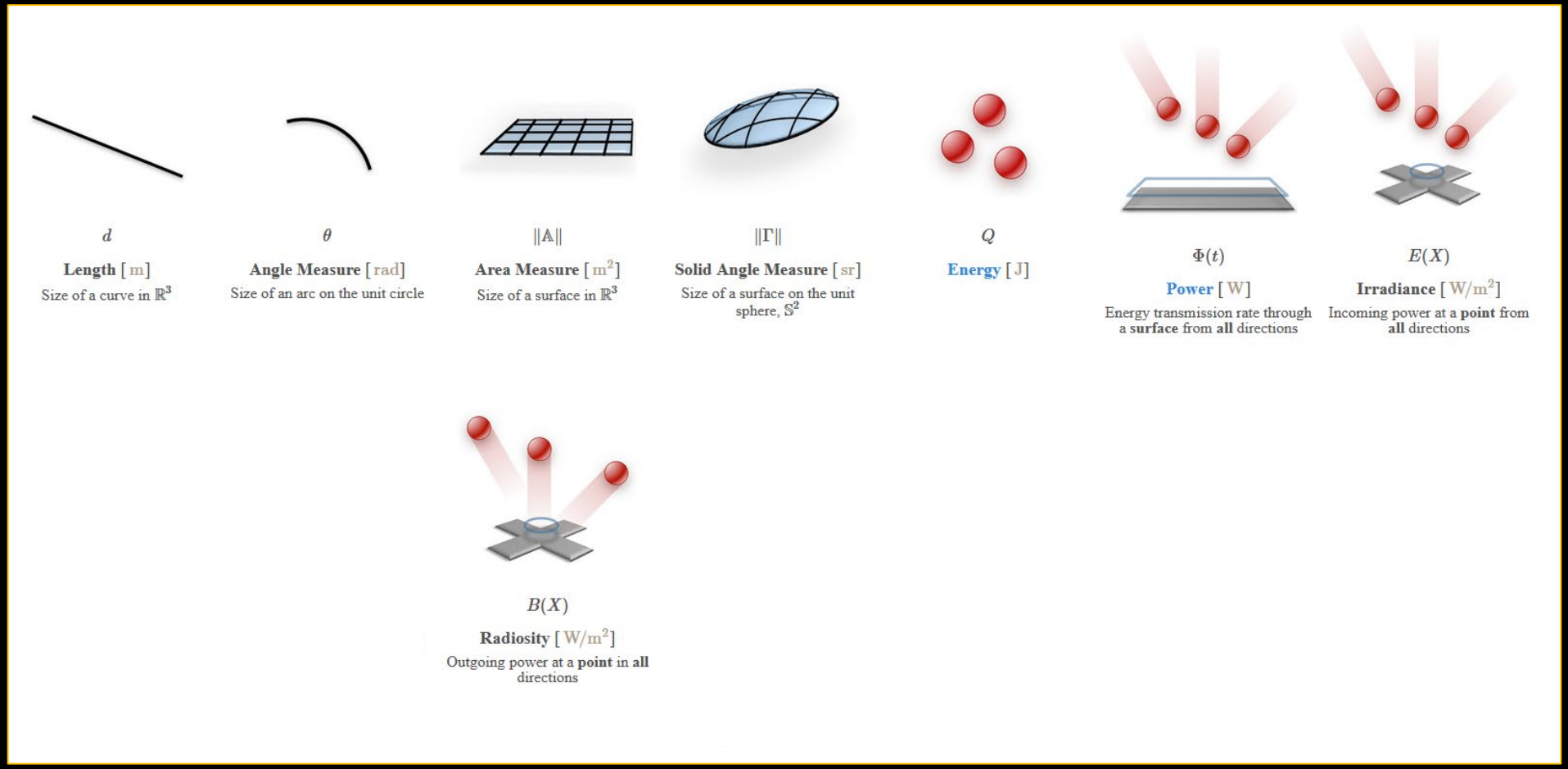
To describe what happens at a single point, we divide the flux by the area it falls on. This is **radiant flux density**, the energy per unit time and per unit area, measured in W/m^2.

The flux density arriving at a point from all directions is called irradiance E. Irradiance is calculated as $E = \frac{\mathrm{d}\phi}{\mathrm{d}\mathbb{A}}$, a tiny amount of flux divided by the tiny area it lands on, thus also measured as W/m^2.

Radiosity B is the same idea in the other direction. It measures the flux density leaving a point in all directions, also in W/m^2.

MODELING LIGHT | RADIOMETRY

Graphics Codex – Morgan McGuire



MODELING LIGHT | RADIOMETRY

Radiant flux is also denoted as Φ(phi, pron. "fie"). It is energy per unit time, $\Phi = \frac{dQ}{dt}$, measured in Watts.

The flux arriving on an entire (oriented) surface $\mathbb{A}$ is written $\Phi(\mathbb{A})$. This is still a flux, measured in Watts.

To describe what happens at a single point, we divide the flux by the area it falls on. This is **radiant flux density**, the energy per unit time and per unit area, measured in W/m^2.

The flux density arriving at a point from all directions is called irradiance E. Irradiance is calculated as $E = \frac{\mathrm{d}\phi}{\mathrm{d}\mathbb{A}}$, a tiny amount of flux divided by the tiny area it lands on, thus also measured as W/m^2.

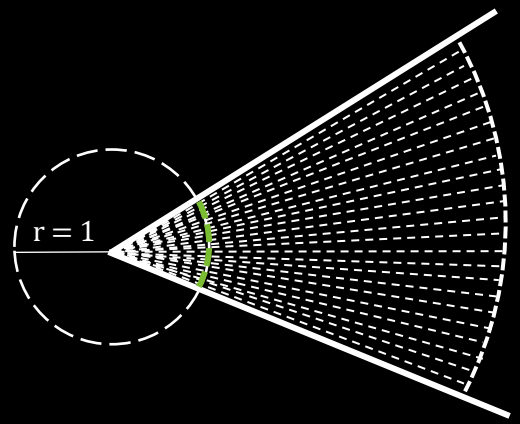
Radiosity B is the same idea in the other direction. It measures the flux density leaving a point in all directions, also in W/m^2.

To measure light along a ray, we need to restrict the directions it comes from or goes to. Instead of irradiance from all directions, we look at a tiny bundle of directions around a single direction.

MODELING LIGHT | RADIOMETRY

When we restricted light to time (radiant flux), we divided energy (J) by time (s), giving watts ($W = J/s$). To restrict it to a single point, we divided by area (m^2), giving irradiance in W/m^2. So, what should we divide by to restrict our measurement to a single direction?

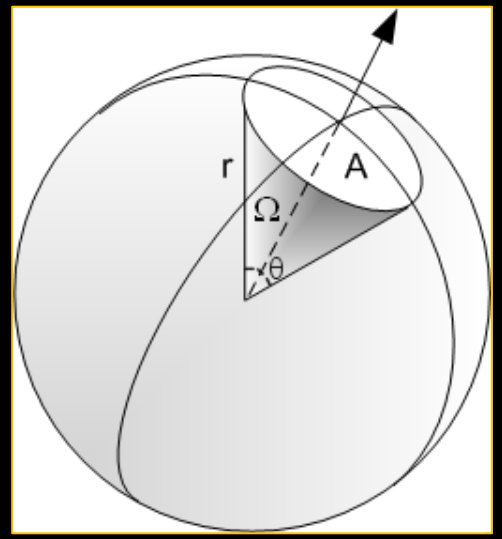
Let's take a look at what an angle is. Real-Time Rendering book describes it as "a measure of the size of a continuous set of directions in a plane, with a value in radians equal to the length of the arc this set of directions intersects on an enclosing circle with radius 1".



MODELING LIGHT | RADIOMETRY

Similarly, a solid angle measures the size of a continuous set of directions in 3D, measured in steradians (**sr**). Instead of an arc on a unit circle, it is the patch the direction cut out of a unit sphere.

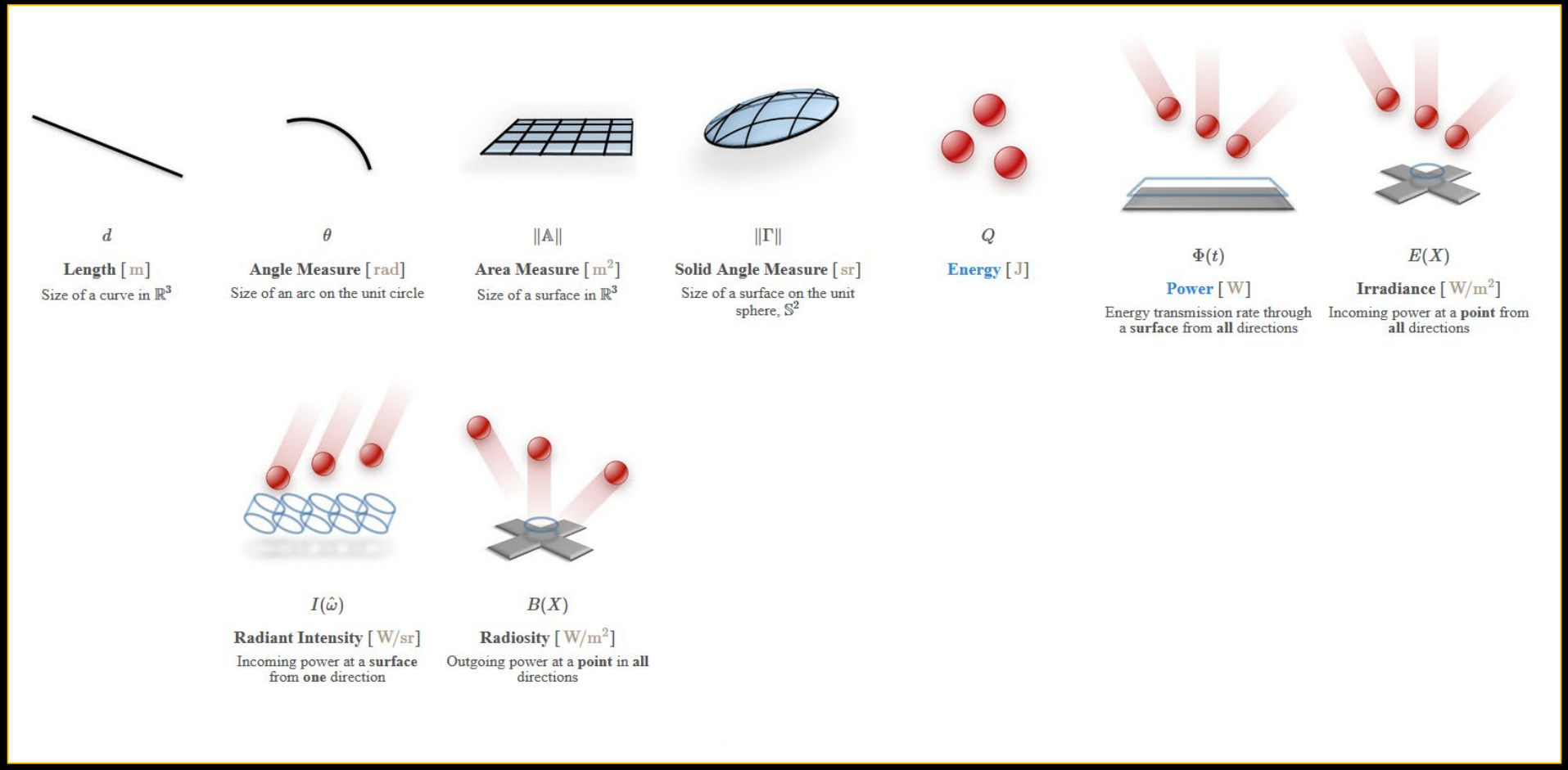
In 2D, an angle of 2π radians covers a whole unit circle. In 3D, a solid angle of 4π steradians covers a whole unit sphere, so 2π steradians covers half of it, which we call a **hemisphere**.



Radiant intensity I is the radiant flux per solid angle, the power a point source send into a direction. It is written as $I = \frac{d\phi}{d\omega}$ and is measured in W/sr.

MODELING LIGHT | RADIOMETRY

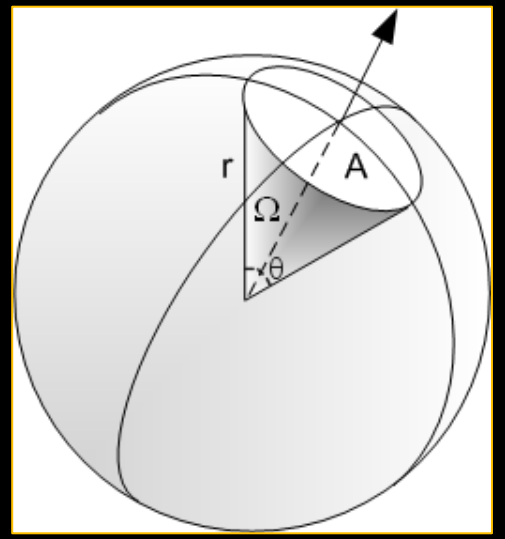
Graphics Codex – Morgan McGuire



MODELING LIGHT | RADIOMETRY

Similarly, a solid angle measures the size of a continuous set of directions in 3D, measured in steradians (**sr**). Instead of an arc on a unit circle, it is the patch the direction cut out of a unit sphere.

In 2D, an angle of 2π radians covers a whole unit circle. In 3D, a solid angle of 4π steradians covers a whole unit sphere, so 2π steradians covers half of it, which we call a **hemisphere**.

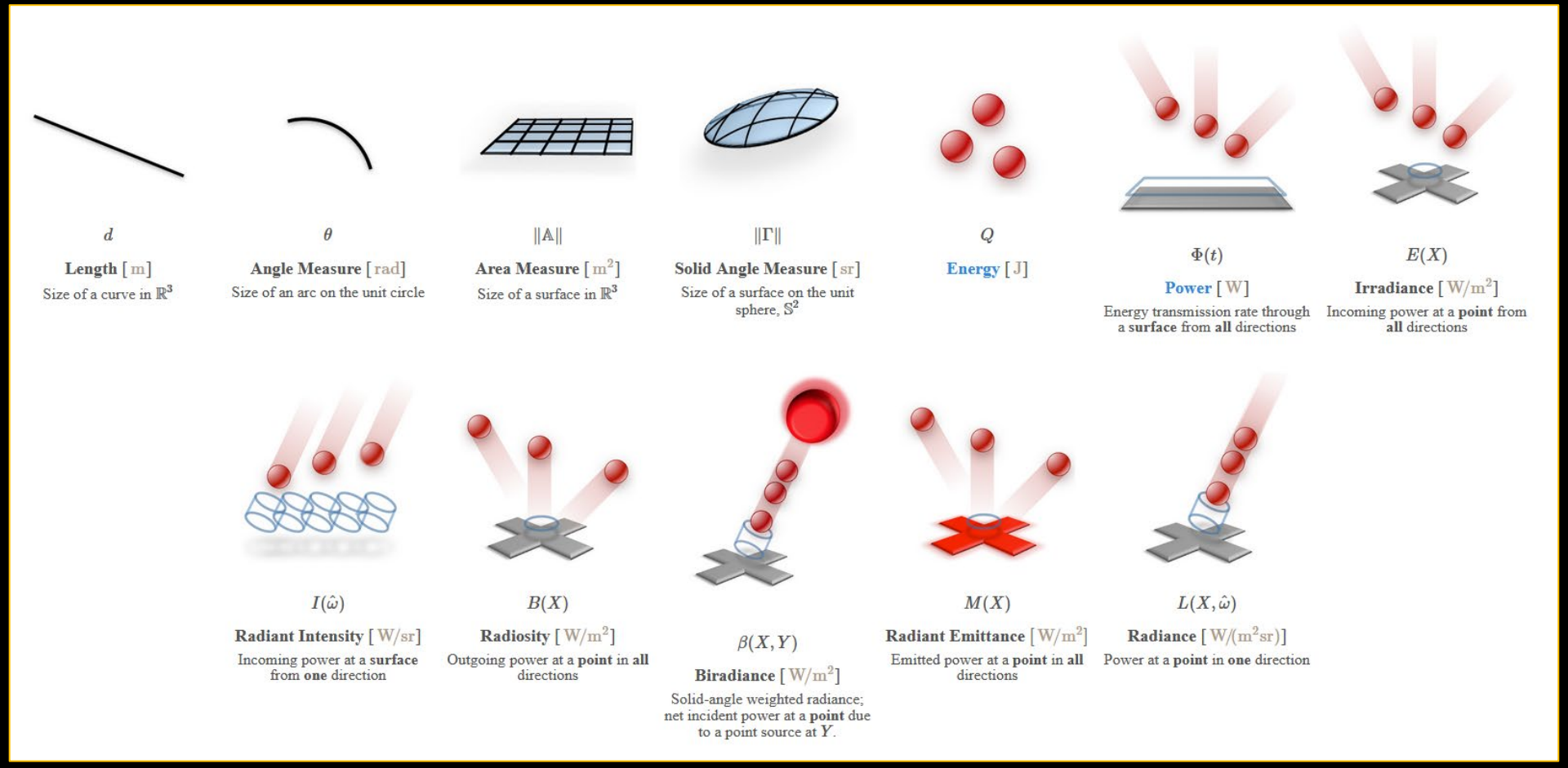


Radiant intensity I is the radiant flux per solid angle, the power a point source send into a direction. It is written as $I = \frac{d\phi}{d\omega}$ and is measured in W/sr.

Finally, **radiance** L is the flux per area and per solid angle, the light travelling along a single direction. It combines radiant intensity $I = \frac{d\phi}{d\omega}$ and irradiance $E = \frac{d\phi}{d\mathbb{A}}$, and is written as $\mathbf{L} = \frac{d^2\phi}{d\mathbb{A}\, d\omega}$, measure in W/m^2sr.

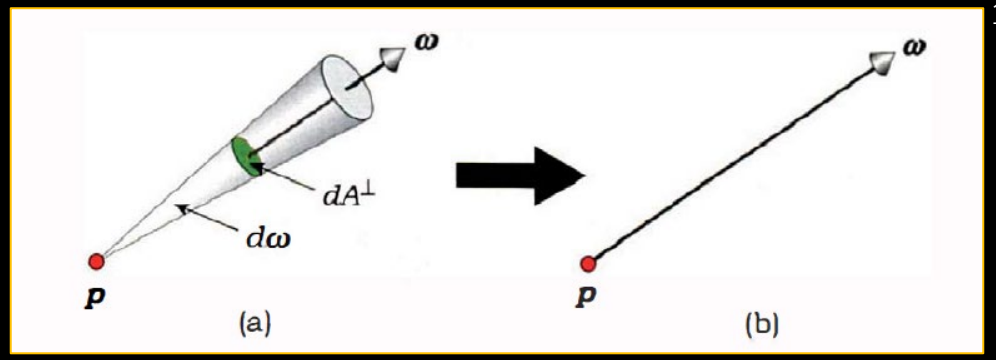
MODELING LIGHT | RADIOMETRY

Graphics Codex – Morgan McGuire



MODELING LIGHT | RADIOMETRY

It is this radiance that matters to us. For every pixel, we want the radiant energy per unit time, per unit area and per unit solid angle. But a solid angle is still a cone of direction. Didn't we want to measure along a ray?



1

Radiance is the limit where both the area and the solid angle approach zero, so the cone narrows to a single ray. In empty space, radiance also stays constant along the ray.

Also interesting to know, a small patch with area $\mathbb{A}$ at distance $\mathbf{R}$ covers a single angle of $\mathbf{d\omega} = \frac{\mathbb{A}\cos\theta}{R^2}$, where $\boldsymbol{\theta}$ is the angle at which the patch is tilted away from us. This is where the inverse square law comes from.

1. Ray Tracing From The Ground Up – Kevin Suffern

MODELING LIGHT | PHOTOMETRY

Radiometric & Photometric Terms & Units

Physics		Radiometry	Units	Photometry	Units
Energy	Q	Radiant Energy	Joules (W·sec)	Luminous Energy	Lumen·sec
Flux (Power)	Φ	Radiant Power	W	Luminous Power	Lumen (Candela sr)
Angular Flux Density	I	Radiant Intensity	W/sr	Luminous Intensity	Candela (Lumen/sr)
Spatial Flux Density	E	Irradiance (in) Radiosity (out)	W/m²	Illuminance (in) Luminosity (out)	Lux (Lumen/m²)
Spatio-Angular Flux Density	L	Radiance	W/m²/sr	Luminance	Nit (Candela/m²)

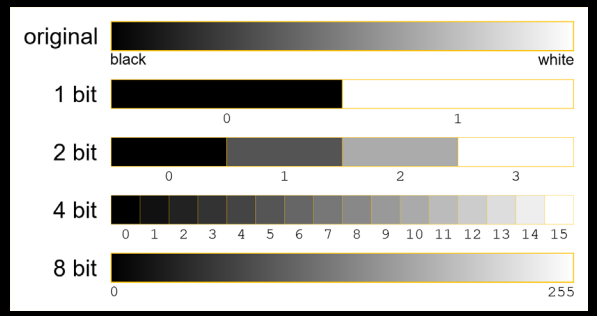
"Thus one nit is one lux per steradian is one candela per square meter is one lumen per square meter per steradian. Got it?" — *James Kajiya*

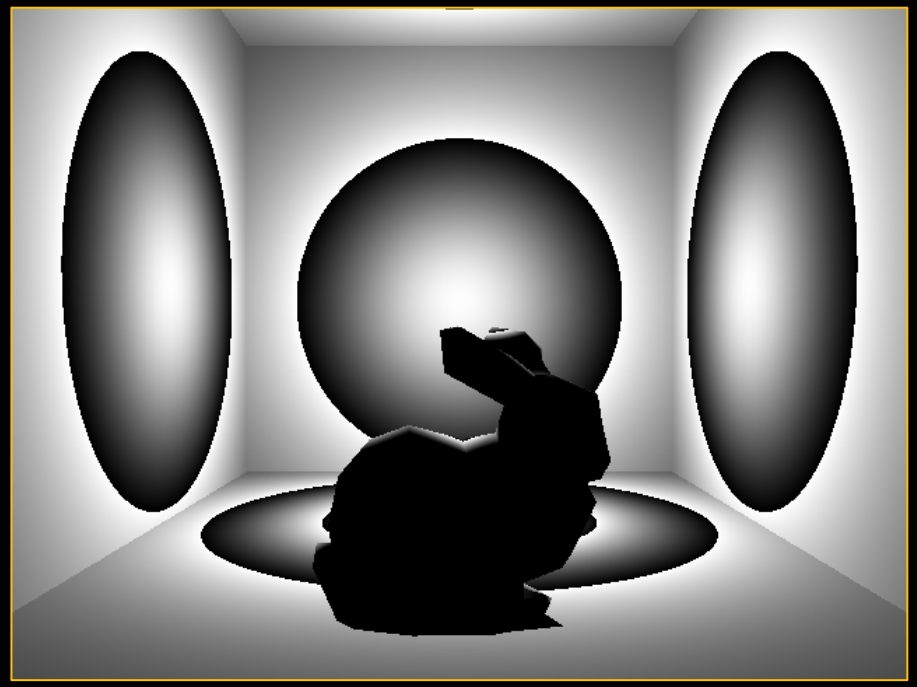
CS184/284A **Kanazawa & Ng**

MODELING LIGHT | TONE MAPPING

With radiometry we can now measure light, but there is a catch! Radiance has no upper limit. A sunlit wall can be thousands of times brighter than the same wall in the shade, and the sun itself is brighter still.

Meanwhile, a pixel does have a limit. We calculate light in floats, but to show it on screen each color channel is stored as a whole number between 0 and 255. Brighter values overflow and wrap around. We could clamp it but then we lose our relative range!





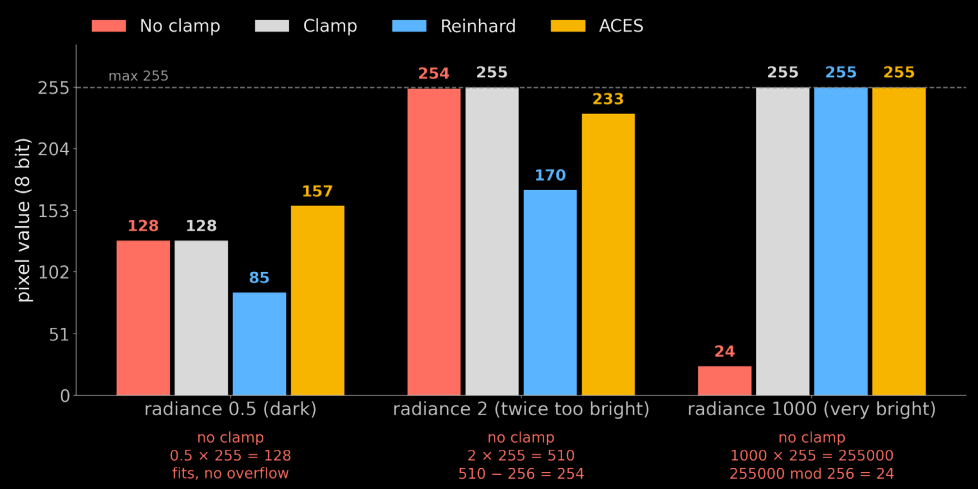
In other words, we need a way to fit an unlimited range into a limited one. So, how can we solve this issue without losing out relative range?

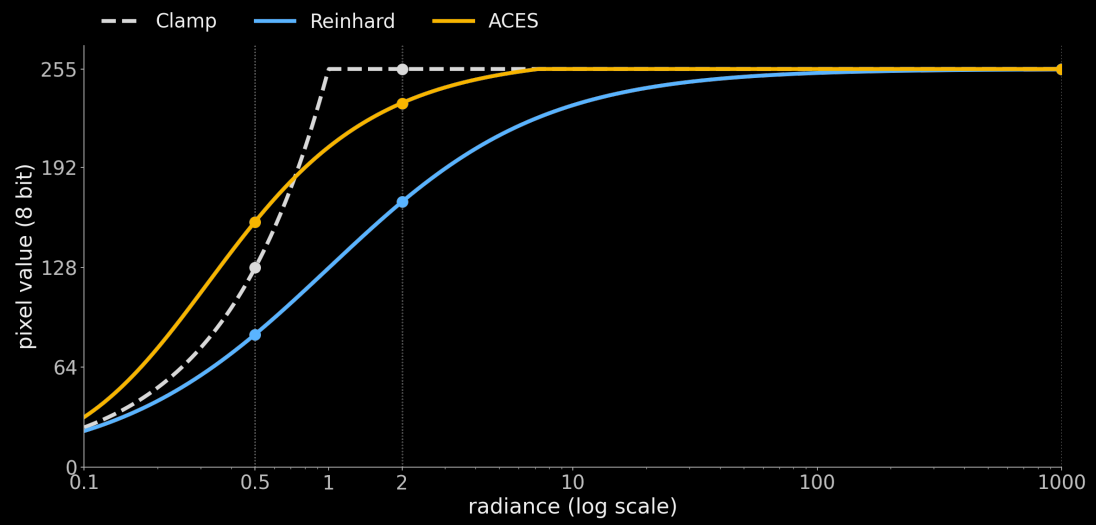
MODELING LIGHT | TONE MAPPING

Cameras and our eyes solve this every day. They compress the huge range of real light into an image we can see, keeping detail in both the dark and the bright parts.

Renderers do the same with tone mapping. A tone mapping curve maps any radiance value into the range 0 to 1, growing almost linearly for dark values and flatting for bright ones, so highlight roll off gently instead of clipping.

Popular curves are Reinhard (simple and smooth), and ACES (the filmic standard used in games and film). These implementations are already provided for you in the color code ☺





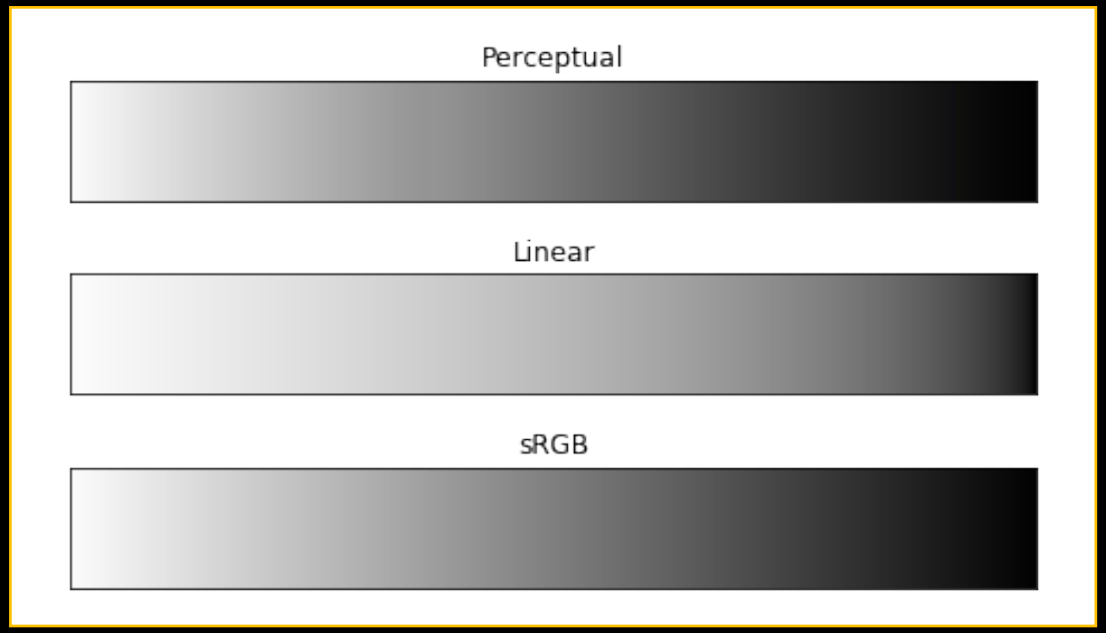
MODELING LIGHT | TONE MAPPING

Rendering in this full range, so before tone mapping, is called High Dynamic Range (HDR). After remapping we are in Low Dynamic Range (LDR).



MODELING LIGHT | TONE MAPPING

While this fixes the overflow and range, there is one last step. Our eyes notice small difference in dark tones much more than in bright ones, so storing light in equal steps (linearly) would waste most of our 256 levels on bright tones we can barely tell apart.

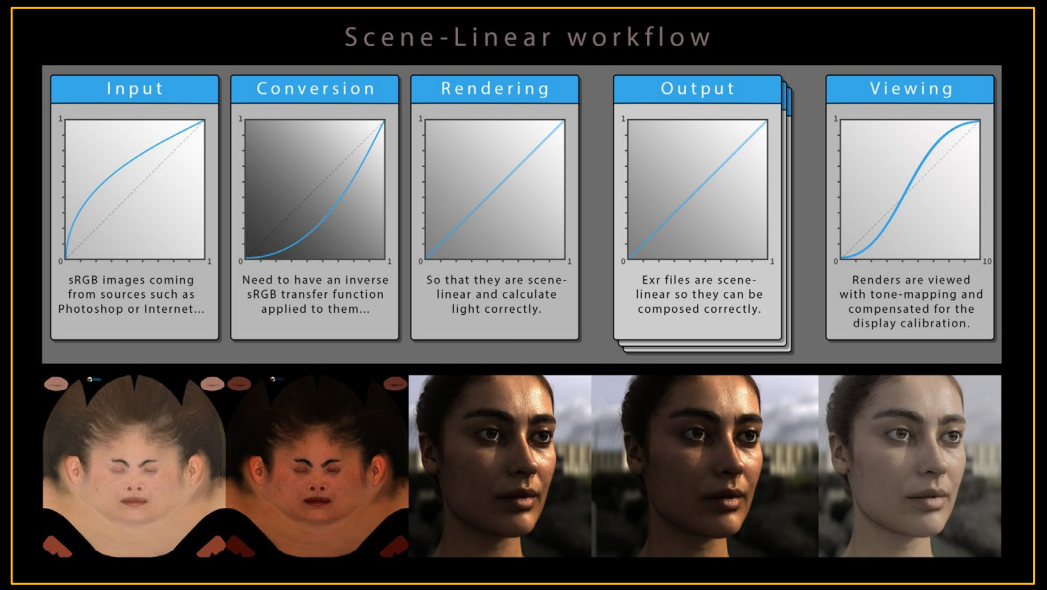


https://bottosson.github.io/posts/colorwrong/

MODELING LIGHT | TONE MAPPING

While this fixes the overflow and range, there is one last step. Our eyes notice small difference in dark tones much more than in bright ones, so storing light in equal steps (linearly) would waste most of our 256 levels on bright tones we can barely tell apart.

To fix this, screens therefor use sRGB, a curve that spends more levels on the darks. It is applied last, right before the value goes into the pixel. Thus important, **light itself adds up linearly, so all our calculations stay in linear values. Only the final result is tone mapped and encoded as sRGB**.

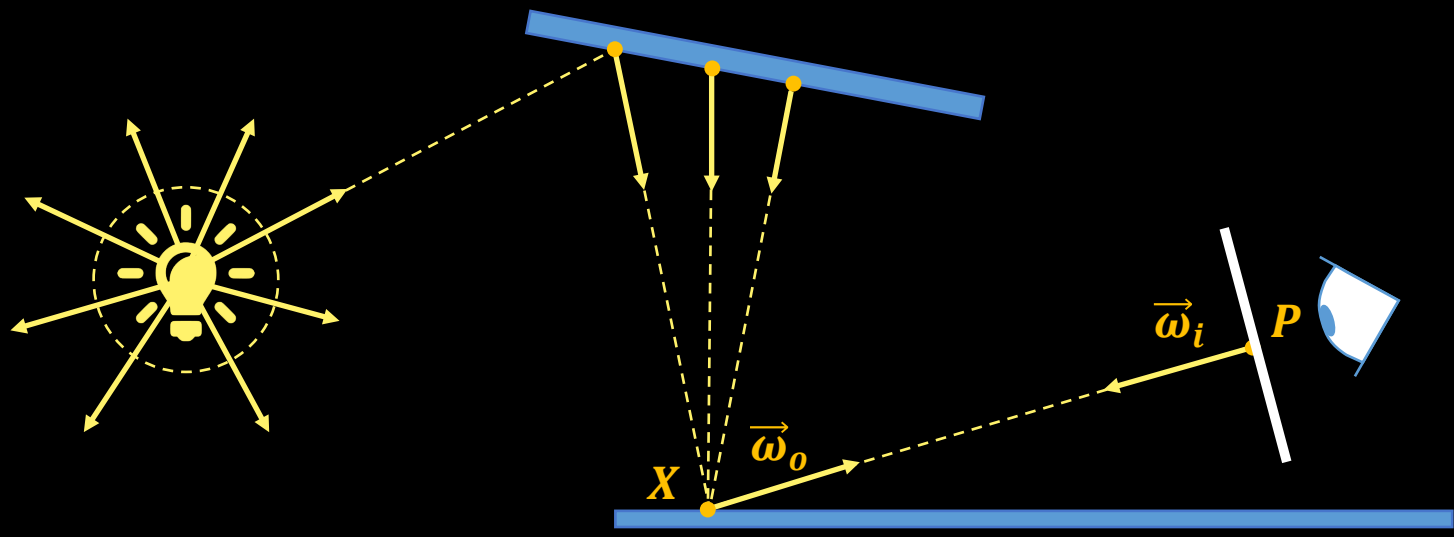


https://chrisbrejon.com/cg-cinematography/chapter-1-color-management/

MODELING LIGHT | RENDERING EQUATION

We now have the units to measure and display light, and we know from the physics how light interacts with matter. Next, we combine the two into a model we can compute.

Every point in the scene receives light and re-radiates part of it, so any point can act as a light source. The radiance leaving a point towards the camera consists of the light the point emits itself and the light it reflects.

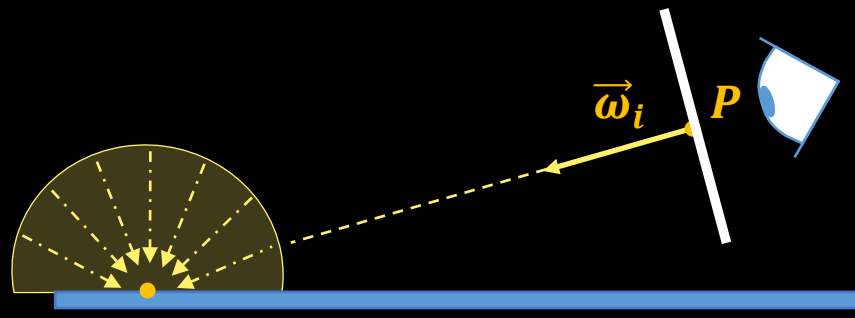


MODELING LIGHT | RENDERING EQUATION

For the reflected part, we add up the incoming radiance from all directions above the surface.

This model is the **rendering equation**, introduced by James Kajiya at SIGGRAPH in 1986. It describes the light leaving every point in the scene.

$$\underbrace{L_o(X, \vec{\omega}_{o,x}) = L_e(X, \vec{\omega}_{o,x}) + \int L_i(X, \vec{\omega}_{i,x})}_{Radiance = \frac{W}{sr * m^2}} \underbrace{f_{X,n}(\vec{\omega}_{i,x}, \vec{\omega}_{o,x})}_{sr^{-1}} \underbrace{cos\,\theta_i \, d\omega_{i,x}}_{sr}$$



ARE YOU OK? STILL WITH US?



ACT I

Light & Matter

ACT II

From Physics to Model

ACT III

Inside the Equation

ACT IV

BRDFs

ACT V

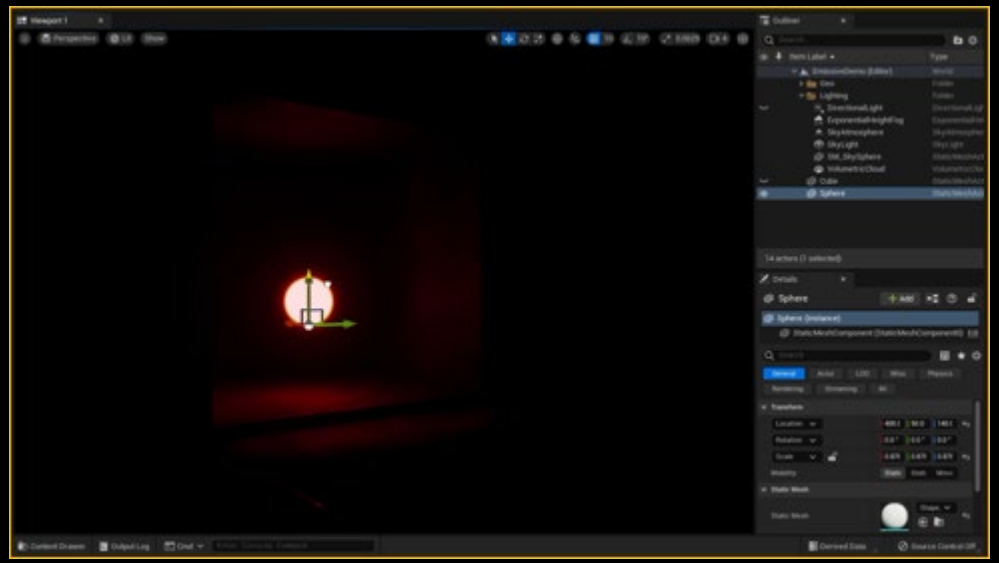
Surface Variation

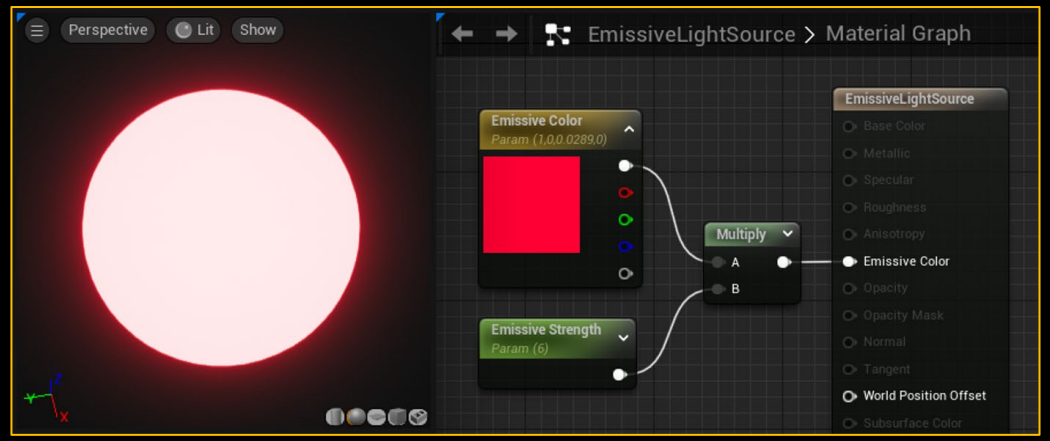
RENDERING EQUATION | EMISSION

While intimidating at first, this equation is conceptually not that hard to understand. Let's look at all parts, as most you already know!

$$L_o(X, \vec{\omega}_{o,x}) = L_e(X, \vec{\omega}_{o,x}) + \int L_i(X, \vec{\omega}_{i,x}) f_{X,n}(\vec{\omega}_{i,x}, \vec{\omega}_{o,x}) \cos\theta_i \, d\omega_{i,x}$$

The first term $L_e(X, \vec{\omega}_{o,x})$, is **emission** as discussed in the physics part of the lecture. In other words, it is light the object itself generates. We will not implement emission in our renderer this semester.





https://dev.epicgames.com/documentation/unreal-engine/using-the-emissive-material-input-in-unreal-engine

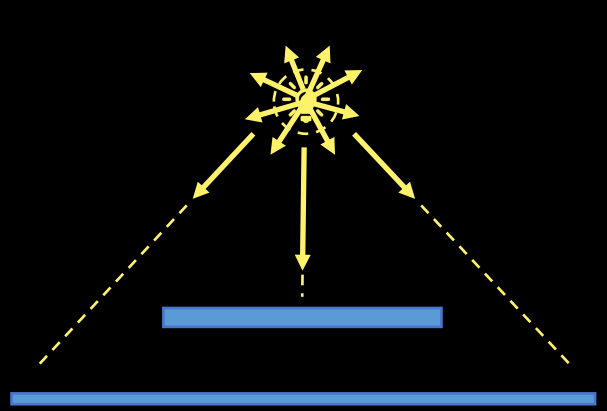
RENDERING EQUATION | INCIDENCE

While intimidating at first, this equation is conceptually not that hard to understand. Let's look at all parts, as most you already know!

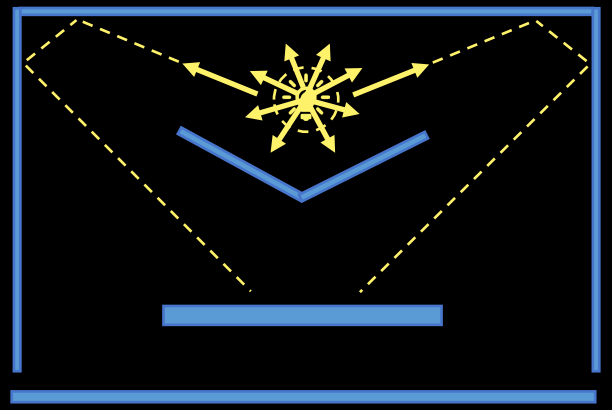
$$L_o(X, \vec{\omega}_{o,x}) = L_e(X, \vec{\omega}_{o,x}) + \int L_i(X, \vec{\omega}_{i,x}) f_{X,n}(\vec{\omega}_{i,x}, \vec{\omega}_{o,x}) \cos\theta_i \, d\omega_{i,x}$$

The second term $L_i(X, \vec{\omega}_{i,x})$, is the radiance arriving at point X in direction $\vec{\omega}_{i,x}$. We collect it from every direction on the hemisphere above the surface. Where the light comes from depends on what we see when we look out from the point along that direction:

- If we see a light source directly, it is **direct illumination**.
- If we see another surface, it is **indirect illumination**, light that already bounced at least once.



Direction Illumination



Indirection Illumination

RENDERING EQUATION | INCIDENCE

We can rewrite the function to split the direct and indirect part. For now, we will only focus on the direct lighting, which is similar as just solving the same equation as before but for direct lighting only.

$$L_o(X, \vec{\omega}_{o,x}) = L_e(X, \vec{\omega}_{o,x}) + \int L_i(X, \vec{\omega}_{i,x}) f_{X,n}(\vec{\omega}_{i,x}, \vec{\omega}_{o,x}) \cos\theta_i \, d\omega_{i,x}$$

$$L_o(X, \vec{\omega}_{o,x}) = L_e(X, \vec{\omega}_{o,x}) + \int_\Omega L_i^{direct}(X, \vec{\omega}_{i,x}) f_{X,n}(\vec{\omega}_{i,x}, \vec{\omega}_{o,x}) \cos\theta_i \, d\omega_{i,x} + \int_\Omega L_i^{indirect}(X, \vec{\omega}_{i,x}) f_{X,n}(\vec{\omega}_{i,x}, \vec{\omega}_{o,x}) \cos\theta_i \, d\omega_{i,x}$$

Solving the direct lighting is not that hard. Our supported lights types reach a point from a single direction, so the integral collapses into a sum over all light! Each contribution depends on two things:

- How much light arrives per area, its irradiance $E_k(X)$. A single direction has no solid angle, so no radiance.
- Whether the light is visible from the point at all, $V_k(X)$.

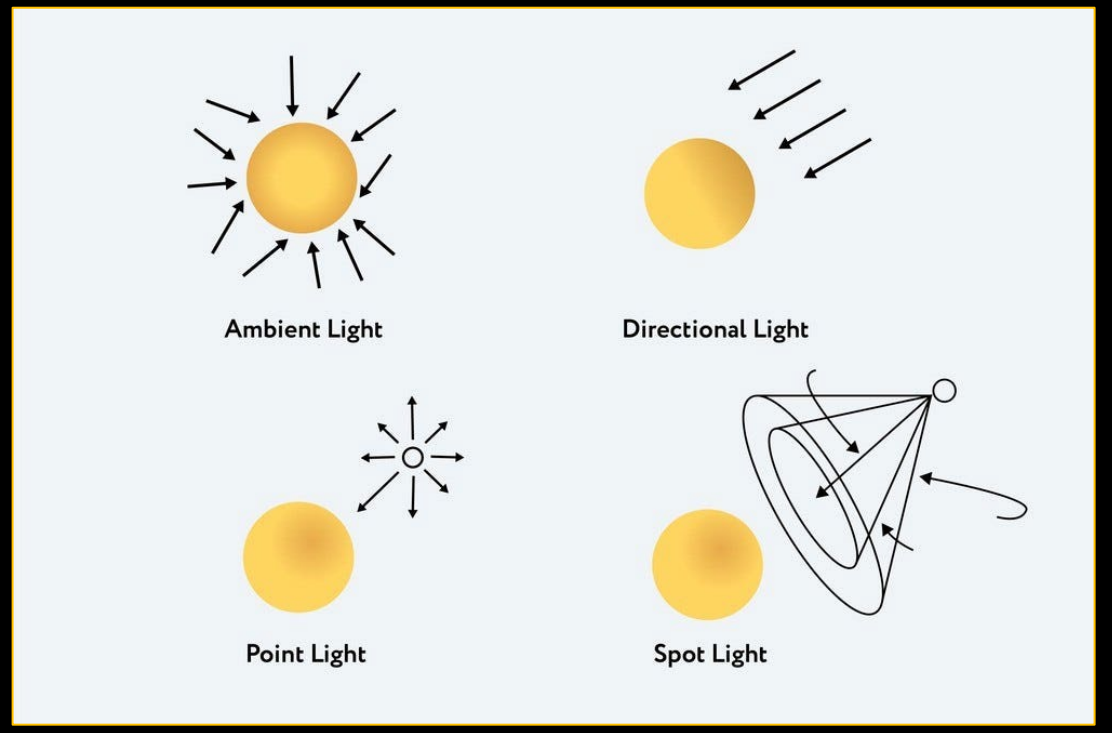
$$L_o(X, \vec{\omega}_{o,x}) = L_e(X, \vec{\omega}_{o,x}) + \sum_{k=1}^{N} E_k(X) V_k(X) f_{X,n}(\vec{\omega}_{k,x}, \vec{\omega}_{o,x}) + L_{indirect}$$

RENDERING EQUATION | INCIDENCE

Let's look how to calculate both $E_k(X)$ and $V_k(X)$. There are different types of lights (in game engines):

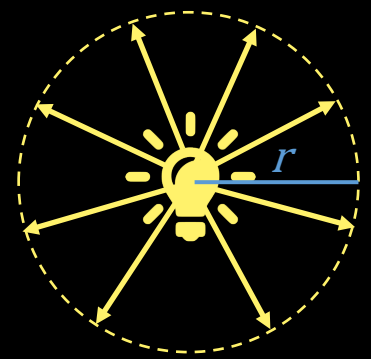
- Ambient: cheap, indirect illumination (objects receive a default radiance).
- Directional: light coming from afar (e.g. the sun), all incident light rays are considered parallel.
- Point: light radiating from a single point in space.
- Spot: like a point light but restricted in a cone shape.
- Area: light being emitted from a surface.

Almost all light types are not physically accurate. Real light sources have a surface; thus, they are area lights. While most offline renderers use area lights, for now we will only support the easy to implement light types (directional and point).



https://medium.com/@althafkhanbecse/title-enlightening-3d-worlds-mastering-lighting-techniques-in-three-js-c860caa8cdcf

RENDERING EQUATION | INCIDENCE



Let's describe a point light with parameters:

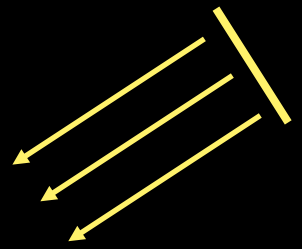
- Position [Vector3] – position of the light source.
- Color [ColorRgba] – color the light emits.
- Light Intensity [float] – how strongly the light shines into each direction (radiant intensity)

We want to know how much light arrives at a point, its irradiance $\boldsymbol{E}$ but we only have the radiant intensity $\boldsymbol{I}\left(\frac{\boldsymbol{W}}{\boldsymbol{sr}}\right)$.
A point light shines equally in all directions, so its total flux is $\boldsymbol{\phi} = \boldsymbol{4\pi} * \boldsymbol{I}$ (Watt), the full sphere of 4π steradians. At distance $\boldsymbol{r}$ this flux is spread over a sphere with area ($\mathbb{A}$) $\boldsymbol{4\pi r^2}$, so $\boldsymbol{E} = \frac{\boldsymbol{\phi}}{\mathbb{A}} = \frac{\boldsymbol{4\pi I}}{\boldsymbol{4\pi r^2}} = \frac{\boldsymbol{I}}{\boldsymbol{r^2}}$

$$\boldsymbol{E_{rgb}} = \boldsymbol{LightColor} * \frac{\boldsymbol{I}}{\|\boldsymbol{LightPosition} - \boldsymbol{PointToShade}\|^2}$$

This is the inverse square law again. The same light spreads over a sphere that grows with the distance squared, so twice as far away means only a quarter of the light. In graphics this is often called distance falloff or distance attenuation.

RENDERING EQUATION | INCIDENCE



Let's describe a directional light with parameters:

- Direction [Vector3] – directional light only has one direction (all considered parallel), no position.
- Color [ColorRgba] – color the light emits.
- Light Intensity [float] – the irradiance it delivers on a surface facing the light.

Calculating the irradiance of a directional light is even easier. All rays are parallel and there is no distance falloff. Given that the intensity is already defined in irradiance, it simple becomes:

$$E_{rgb} = LightColor * I$$

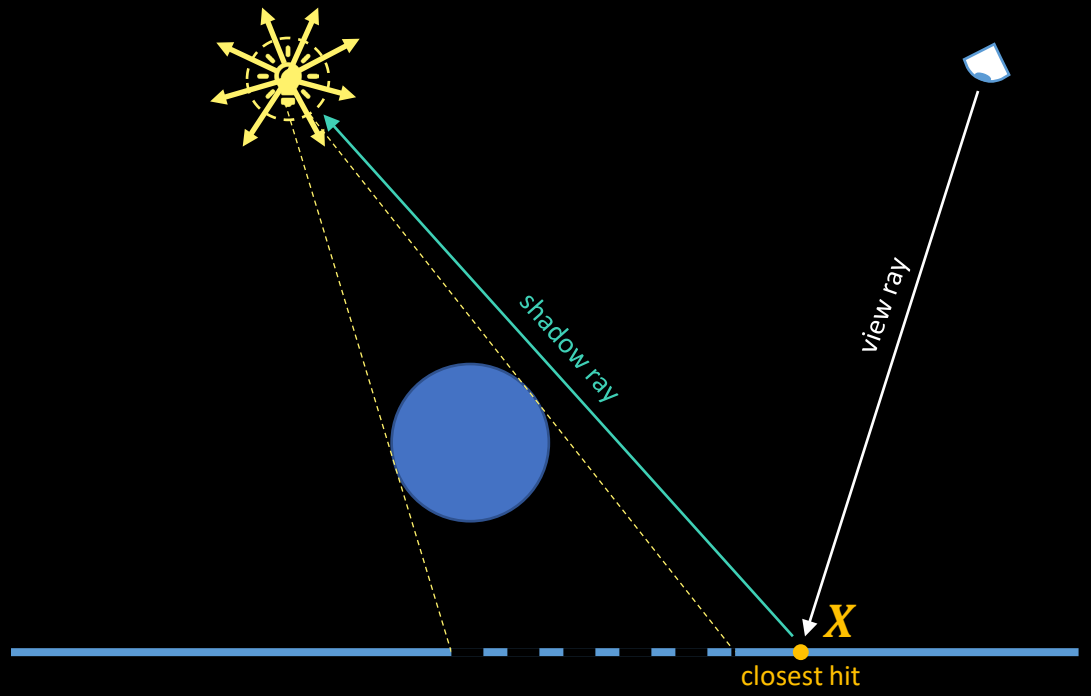
Fun fact, a directional light is like a point light infinitely far away. Its rays become parallel and the distance no longer matters. So, if you want a point light without falloff, keep its direction per point but use the directional light formula.

RENDERING EQUATION | SHADOW TERM

We know how to solve the irradiance term $E_k(X)$, but what about the visibility term $V_k(X)$?

It turns out we already have the solution… We are writing a ray/path tracer that solves the visibility problem for us. Thus, we simply shoot another ray from point $\boldsymbol{X}$ toward the light source to see if the light is directly visible or not. This is called the shadow ray.

If the shadow ray hits anything before it reaches the light, the light is blocked and $V_k = 0$. If it hits nothing, the light is visible and $V_k = 1$.



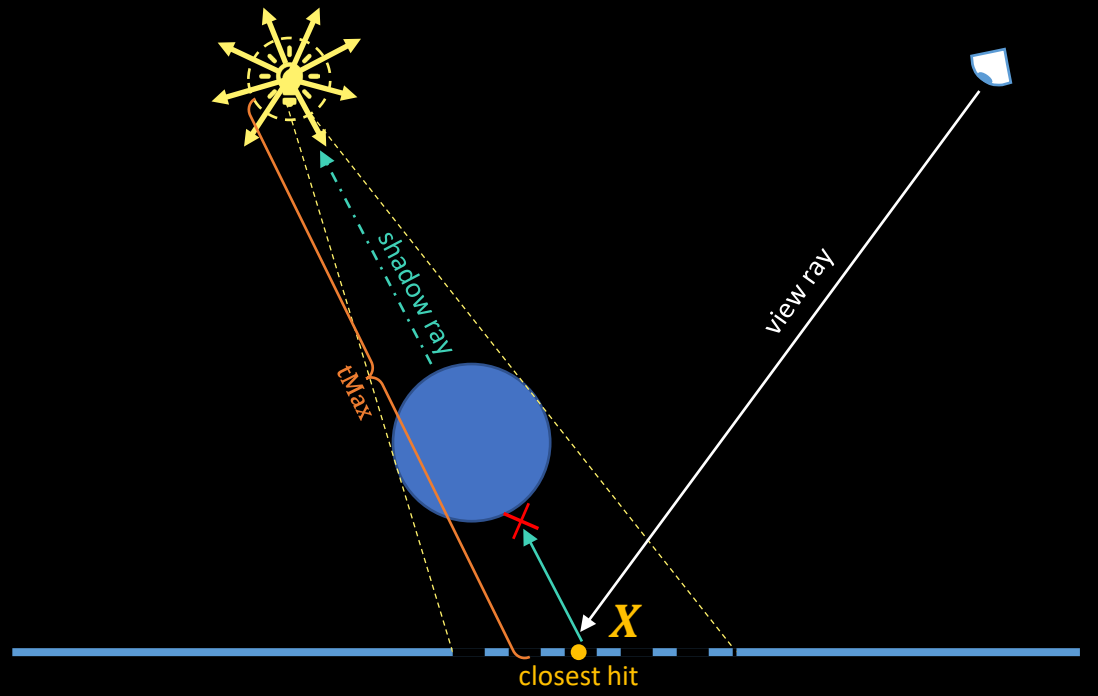
RENDERING EQUATION | SHADOW TERM

We know how to solve the irradiance term $E_k(X)$, but what about the visibility term $V_k(X)$?

It turns out we already have the solution… We are writing a ray/path tracer that solves the visibility problem for us. Thus, we simply shoot another ray from point $\boldsymbol{X}$ toward the light source to see if the light is directly visible or not. This is called the shadow ray.

If the shadow ray hits anything before it reaches the light, the light is blocked and $V_k = 0$. If it hits nothing, the light is visible and $V_k = 1$.

The light only needs to reach the light, so **tMax** is the distance to the light. For a directional light it is infinity.



RENDERING EQUATION | INCIDENCE

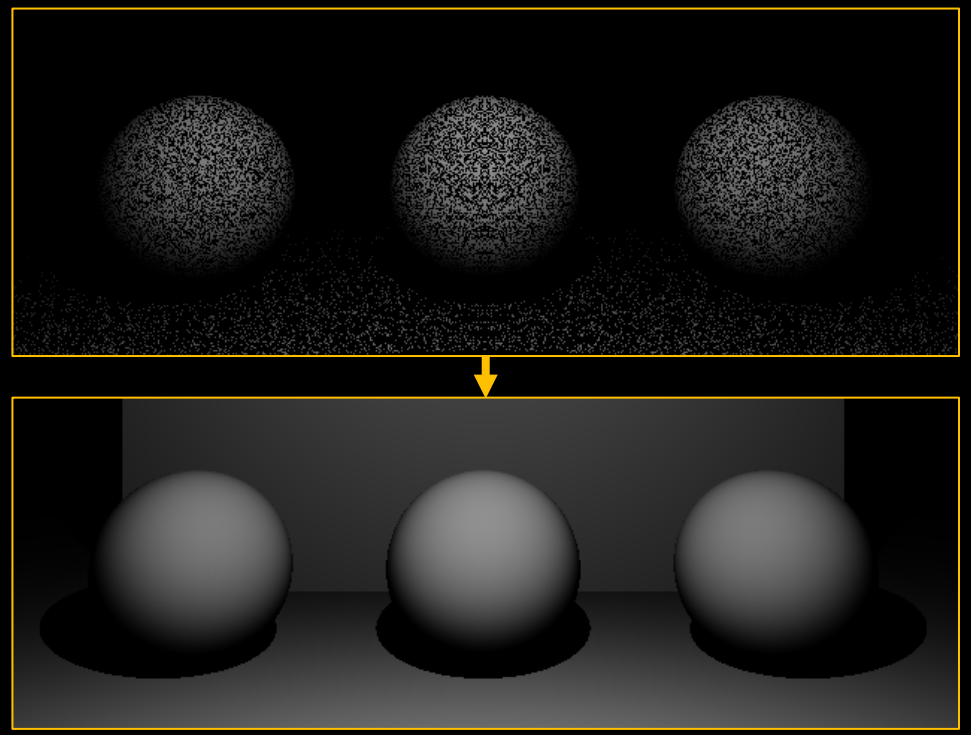
We know how to solve the irradiance term $E_k(X)$, but what about the visibility term $V_k(X)$?

It turns out we already have the solution… We are writing a ray/path tracer that solves the visibility problem for us. Thus, we simply shoot another ray from point $\boldsymbol{X}$ toward the light source to see if the light is directly visible or not. This is called the shadow ray.

If the shadow ray hits anything before it reaches the light, the light is blocked and $V_k = 0$. If it hits nothing, the light is visible and $V_k = 1$.

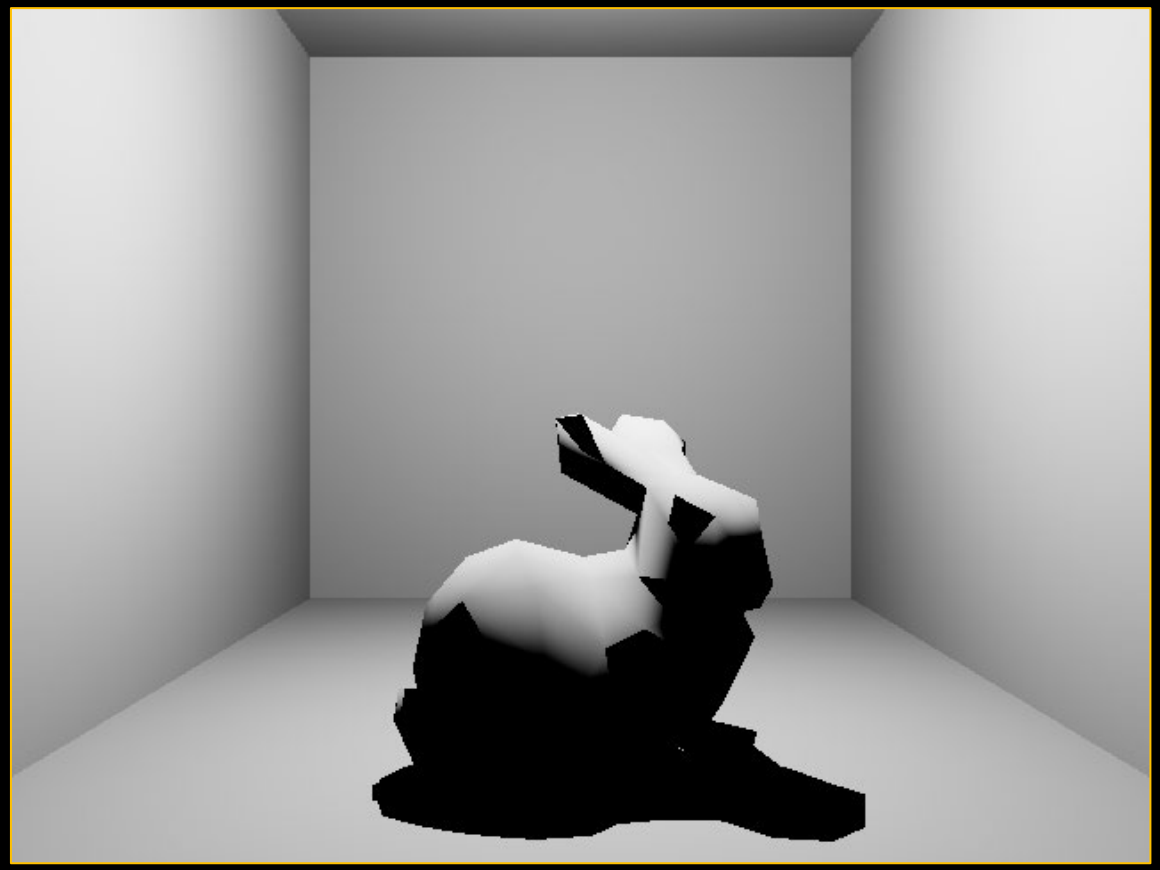
The light only needs to reach the light, so **tMax** is the distance to the light. For a directional light it is infinity.

The ray must not hit the surface it starts on. This self-intersection cases dark speckles called shadow acne. We prevent it with a small epsilon, either as the **tMin** of the ray, or by starting the ray slightly above the surface.



RENDERING EQUATION | SHADOW TERM

While this works great for our spheres and planes, it breaks when we render hard shadows for our bunny. Why?

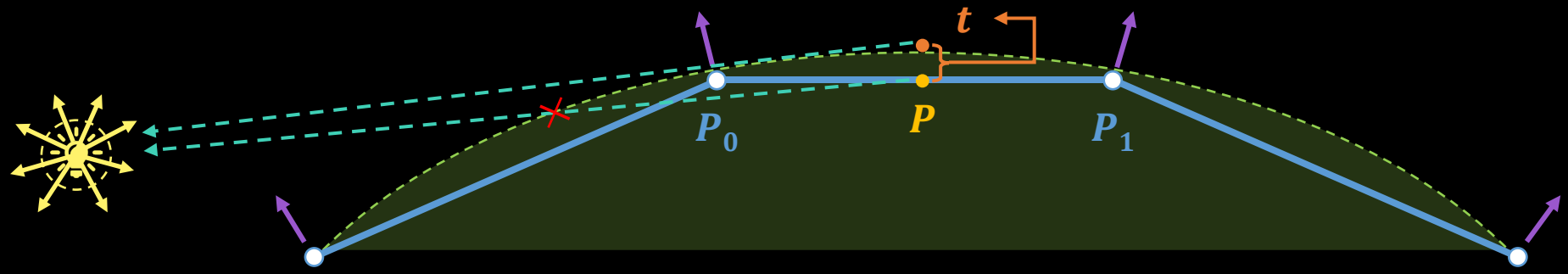


RENDERING EQUATION | SHADOW TERM

We interpolated our normals (using barycentric coordinates), which mimics a smooth surface. When we find a primary intersection point P, we shoot our shadow ray.

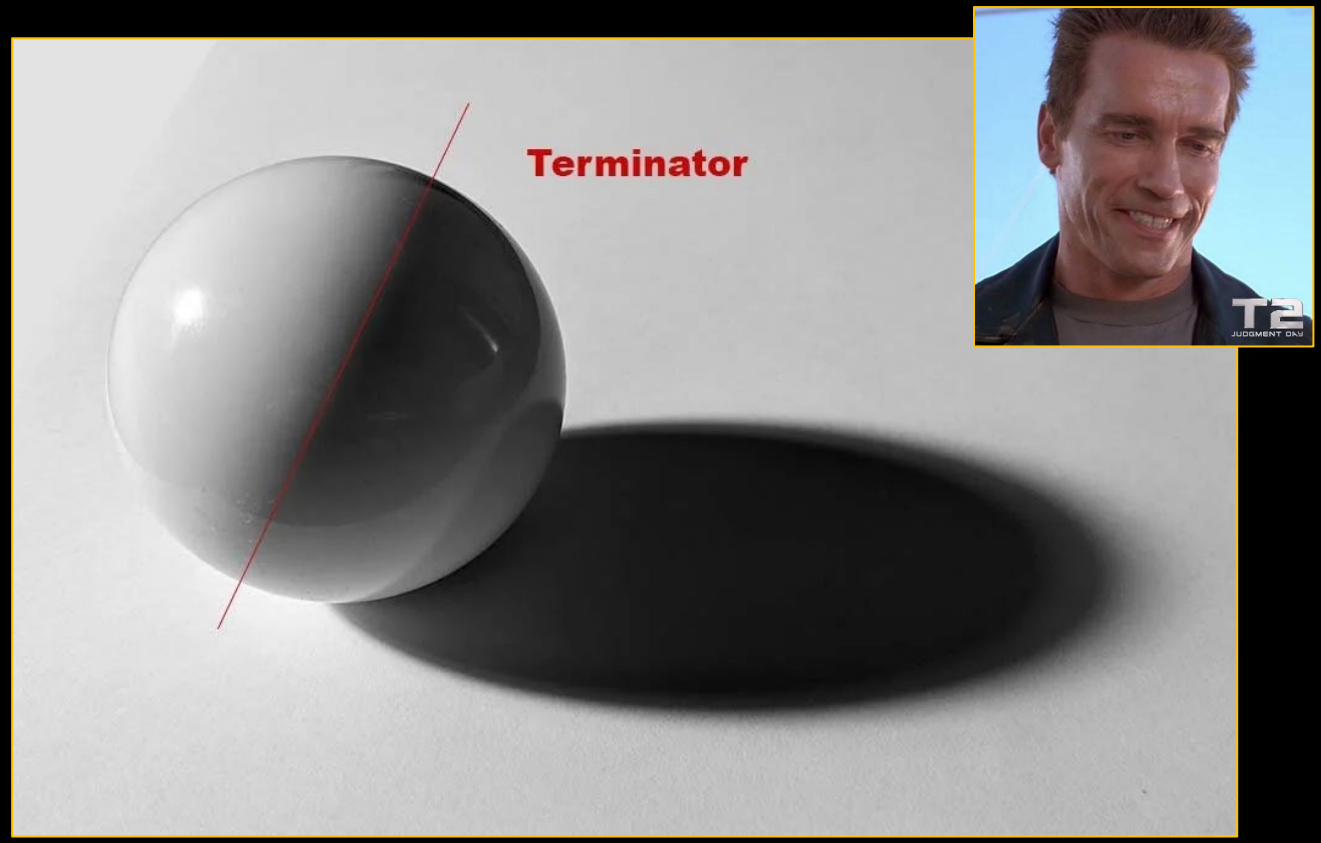
As you will notice, there is self intersection because the actual geometry does not match the smooth representation.

Just as with regular self intersection, we need to find an offset but this time this is not a small value but rather based on the smoothness based on the normals.



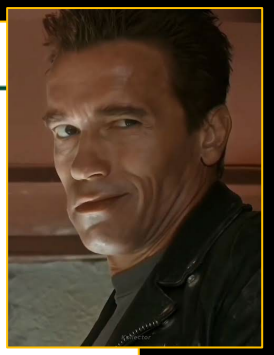
RENDERING EQUATION | SHADOW TERM

We will fix the problem based on the solution proposed in “Hacking the Shadow Terminator” by Johannes Hanika in Ray Tracing Gems II.



https://www.learntopaintpodcast.com/blog/understanding-light-amp-shadow-terminology

RENDERING EQUATION | SHADOW TERM



Listing 4-1. *Pseudocode of the simple shading point offset procedure outlined in [7].*

```
// get distance vectors from triangle vertices
vec3 tmpu = P - A, tmpv = P - B, tmpw = P - C
// project these onto the tangent planes
// defined by the shading normals
float dotu = min(0.0, dot(tmpu, nA))
float dotv = min(0.0, dot(tmpv, nB))
float dotw = min(0.0, dot(tmpw, nC))
tmpu -= dotu*nA
tmpv -= dotv*nB
tmpw -= dotw*nC
// finally P' is the barycentric mean of these three
vec3 Pp = P + u*tmpu + v*tmpv + w*tmpw
```

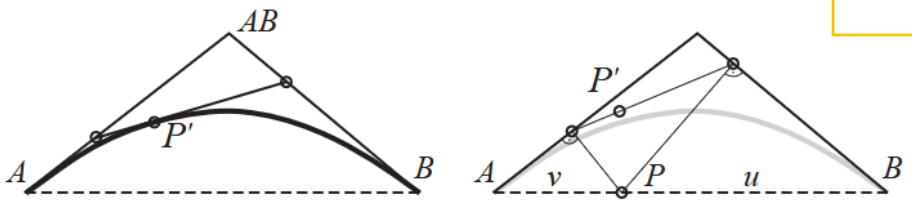


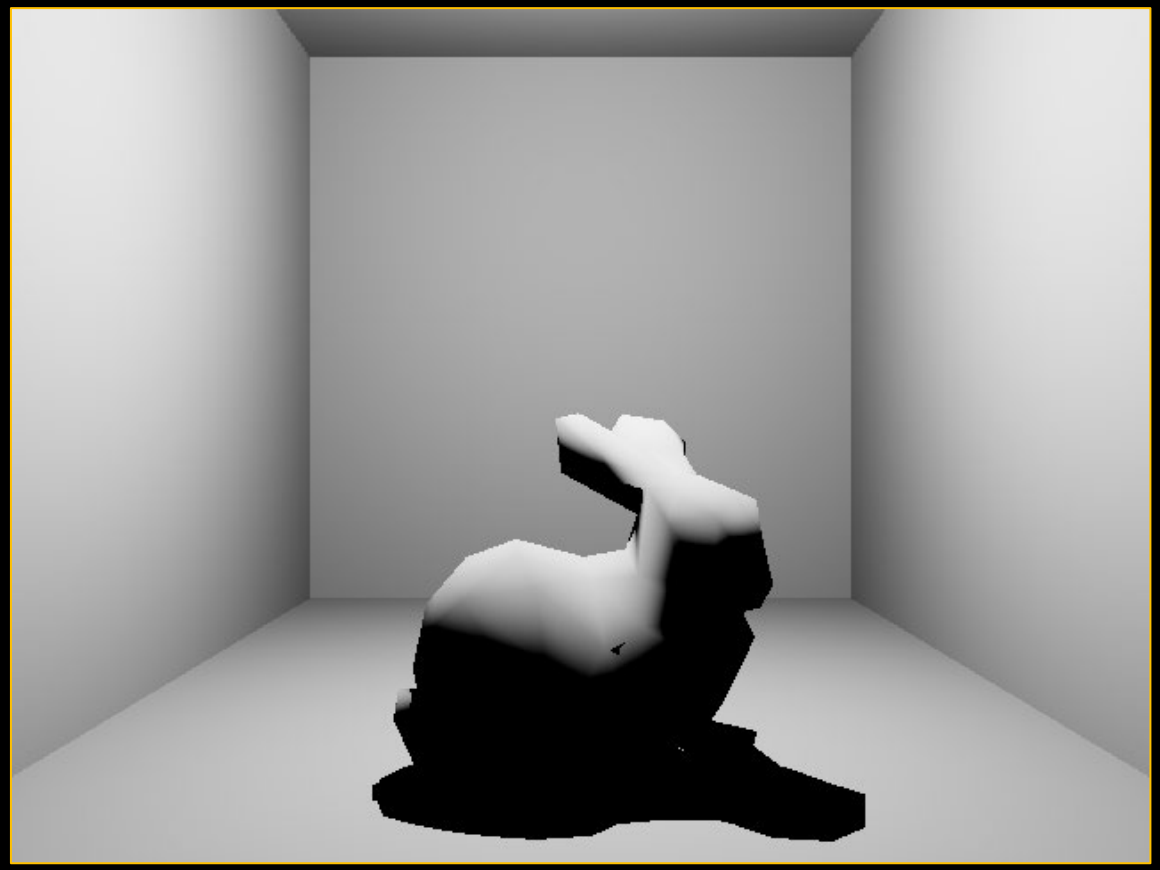
Figure 4-7. *Left: 2D side view visualization of a quadratic Bézier patch (for the w = 0 case), showing the line between A and B. The barycentric coordinates (u, v) are relevant for this 2D slice: de Casteljau proceeds by interpolating A and AB in the ratio v:u, as well as AB and B. The two results are then interpolated using v:u again to yield P′. Right: our simple version does not require the point AB for the computation: it projects the intersection point P on the flat triangle orthogonally to the tangent planes at A and B (note that AB lies on both of these planes, but we don't need to know where). These temporary points are then interpolated in the ratio v:u again, to yield the shading intersection point P′. The Bézier line from the left is replicated to emphasize the differences.*

Make sure to still add additional offset or to set tMin accordingly!

https://www.realtimerendering.com/raytracinggems/rtg2/index.html

RENDERING EQUATION | SHADOW TERM

While this technique works, there are some shortcomings and it does not work as well when the mismatch is too severe.



RENDERING EQUATION | BRDF

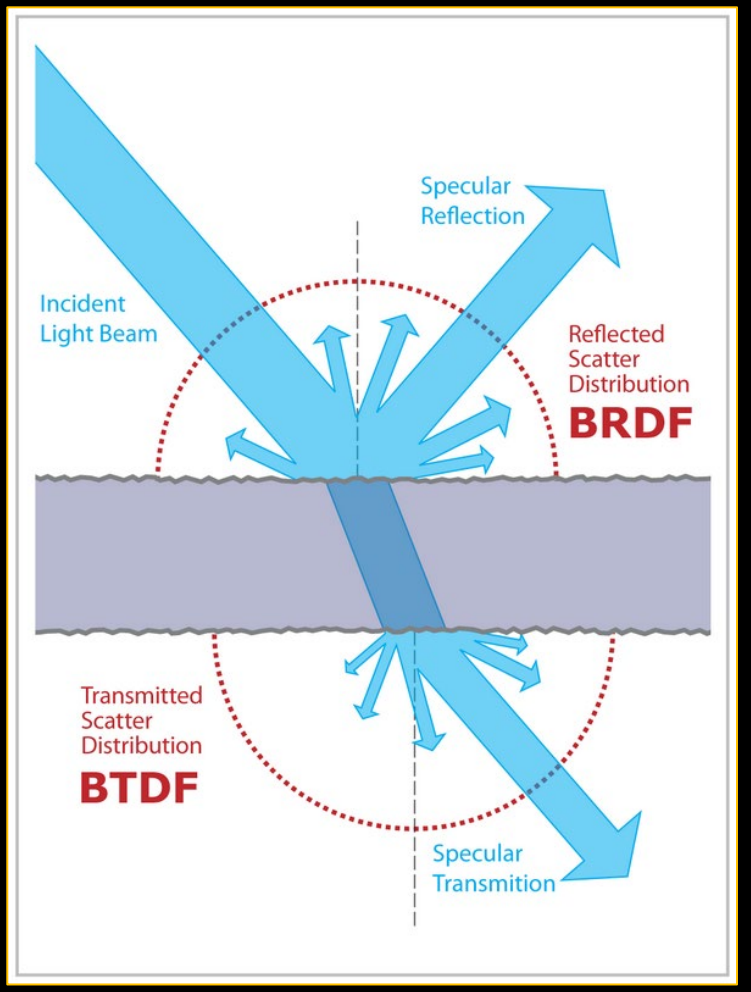
Even in our new summation formula for direct lighting, we have the unknown term $f_{X,n}(\vec{\omega}_{k,x}, \vec{\omega}_{o,x})$.

This is the **BSDF** or Bidirectional Scattering Distribution Function, which describes how light interacts with matter, as discussed in the physics section. The BSDF consists out of two parts:

- **BRDF** or Bidirectional Reflectance Distribution Function.
- **BTDF** or Bidirectional Transmittance Distribution Function.

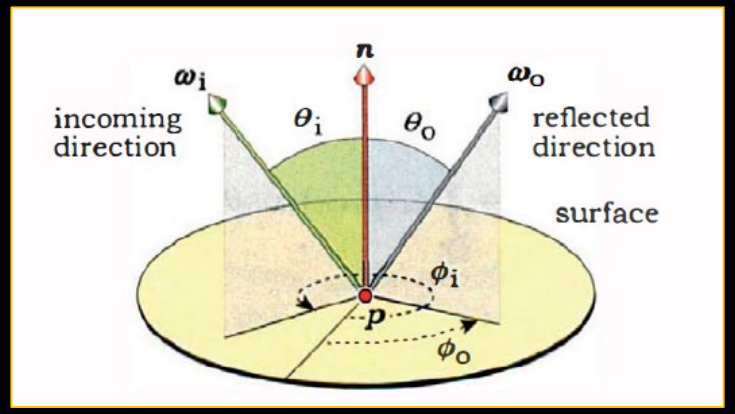
Related functions extend this further, like the BSSRDF (scattering surface) for light that enters at one point and leaves at another, as in skin or marble. But for now, since our equation only looks at the hemisphere above the surface, we will focus on the BRDF.





https://en.wikipedia.org/wiki/Bidirectional_scattering_distribution_function

RENDERING EQUATION | BRDF



The BRDF returns a value in inverse steradians (sr^{-1}). It divides the outgoing radiance, measured per steradian, by the incoming irradiance, measured without steradians. It tells us what fraction of the incoming light leaves per unit of solid angle in each direction.

The BRDF has a few useful properties:

- Positivity – a BRDF is never negative, since a surface cannot reflect a negative amount of light.
- Reciprocity – swapping the incoming and outgoing direction gives the same BRDF value.
- Linearity – materials can combine multiple BRDFs, for example a diffuse and specular part. The total reflected radiance is the sum of what each one reflects.
- Energy conservation – a surface can never reflect more energy than it receives, not counting light the surface emits itself.

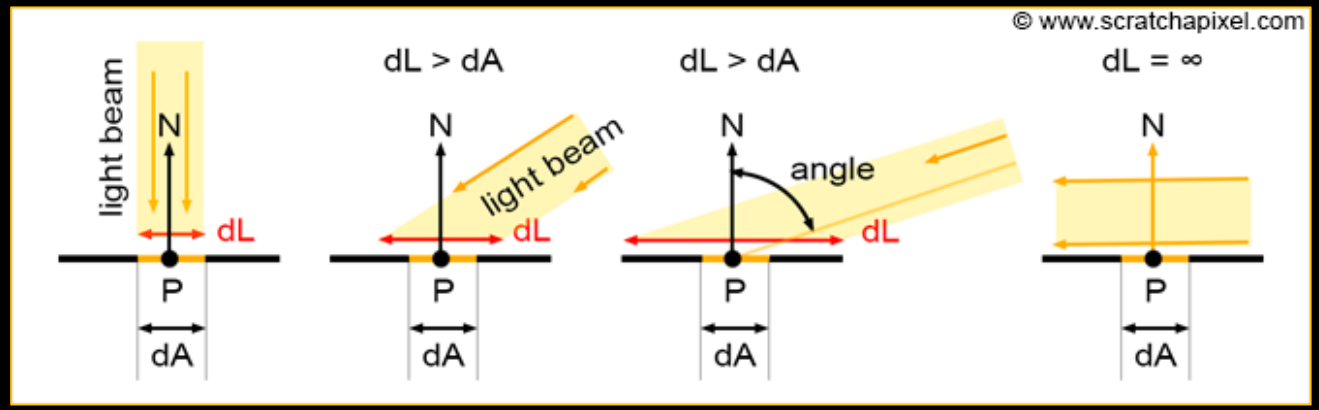
Together these properties make the BRDF largely responsible of how physically plausible a material looks, Since rendering only approximates real-world physics, our image are never perfectly accurate, but a good BRDF gets us very close!

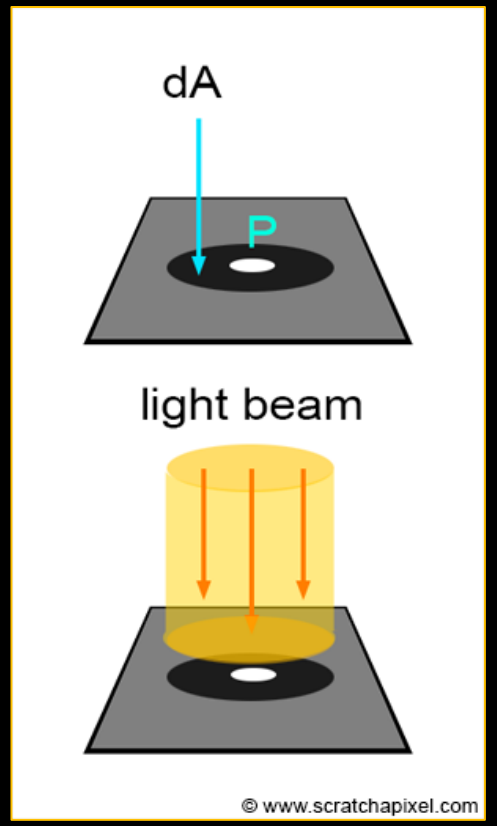
RENDERING EQUATION | COSINE LAW

Another missing term is $\cos\theta_i$. When light reaches a surface at an angle, the same beam spreads over a bigger area, so less light arrives per square meter. Lambert's cosine law states that the irradiance falls with the cosine of the angle between the light direction and the surface normal.

How do we calculate $\cos\theta_i$?

- Because $\vec{\omega}_{i,x}$ and $\vec{n}$ are both normalized, their dot product directly gives the cosine, $\cos\theta_i = \vec{\omega}_{i,x} \cdot \vec{n}$.
- Important, $\vec{\omega}_{i,x}$ must point from the surface toward the light, otherwise the sign flips.
- Light from below the surface gives us negative values, so we clamp it to zero, $max(0, \vec{\omega}_{i,x} \cdot \vec{n})$.





RENDERING EQUATION | SUMMARY

The last term missing is the integral $\int \dots \mathrm{d}\omega_{i,x}$ over all incoming directions. For direct illumination we do not need to solve it, because each light reaches the point from a single direction (for current light types). So, the integral collapses into a sum over all lights, as discussed before.

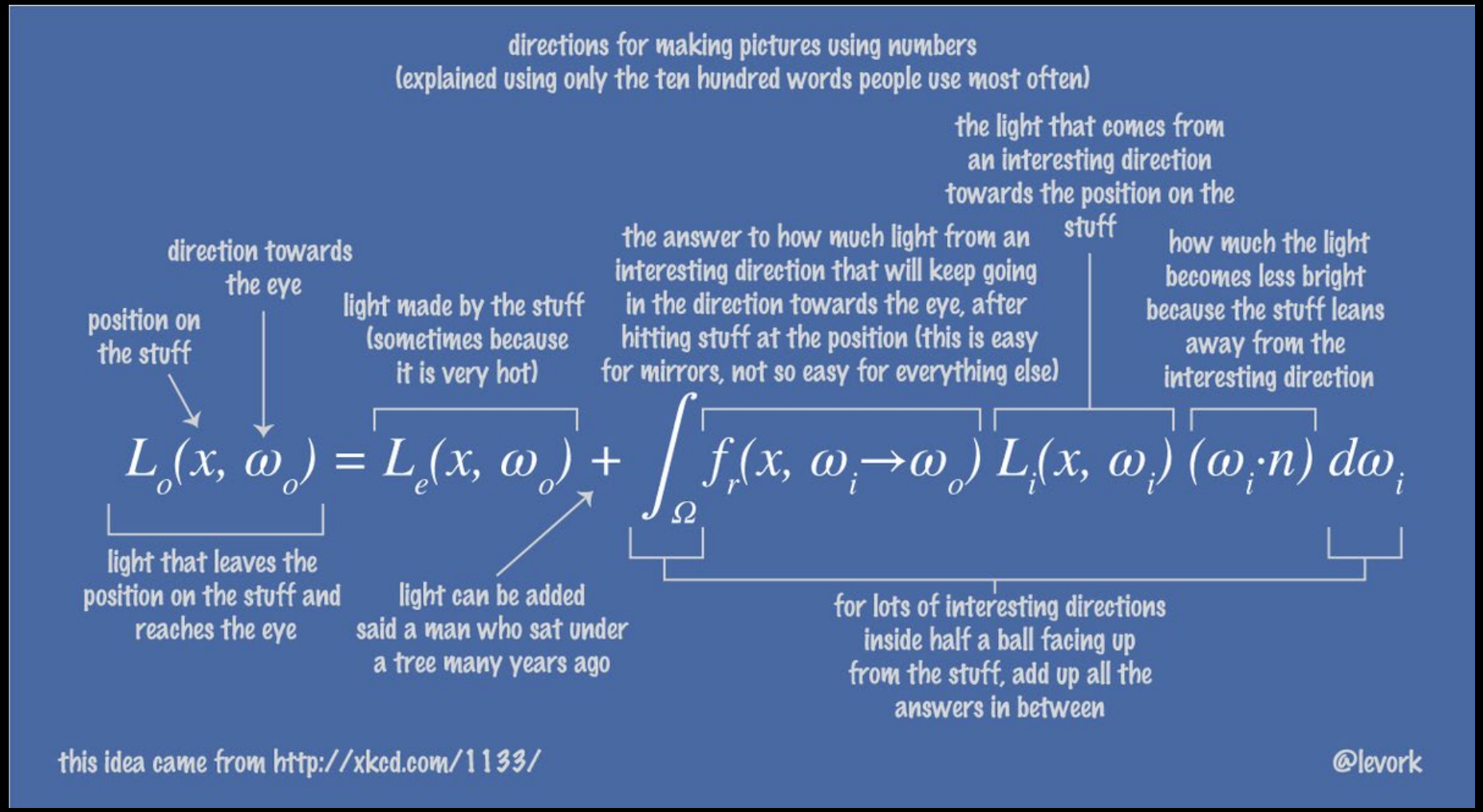
This gives us the entire rendering equation!

$$L_o(X, \vec{\omega}_{o,x}) = L_e(X, \vec{\omega}_{o,x}) + \int L_i(X, \vec{\omega}_{i,x}) f_{X,n}(\vec{\omega}_{i,x}, \vec{\omega}_{o,x}) \cos\theta_i \, \mathrm{d}\omega_{i,x}$$

Object Lights Materials View Angle

Outgoing radiance from point X with direction to the viewer $\vec{\omega}_{o,x}$ is equal to the sum of the incident radiance of all the lights from any direction and the light scattered from any direction by the material to the viewer, within the hemisphere, in the projected area based on the incident radiance and the normal, plus all the radiance emitted by the object.

RENDERING EQUATION | SUMMARY



ACT I

Light & Matter

ACT II

From Physics to Model

ACT III

Inside the Equation

ACT IV

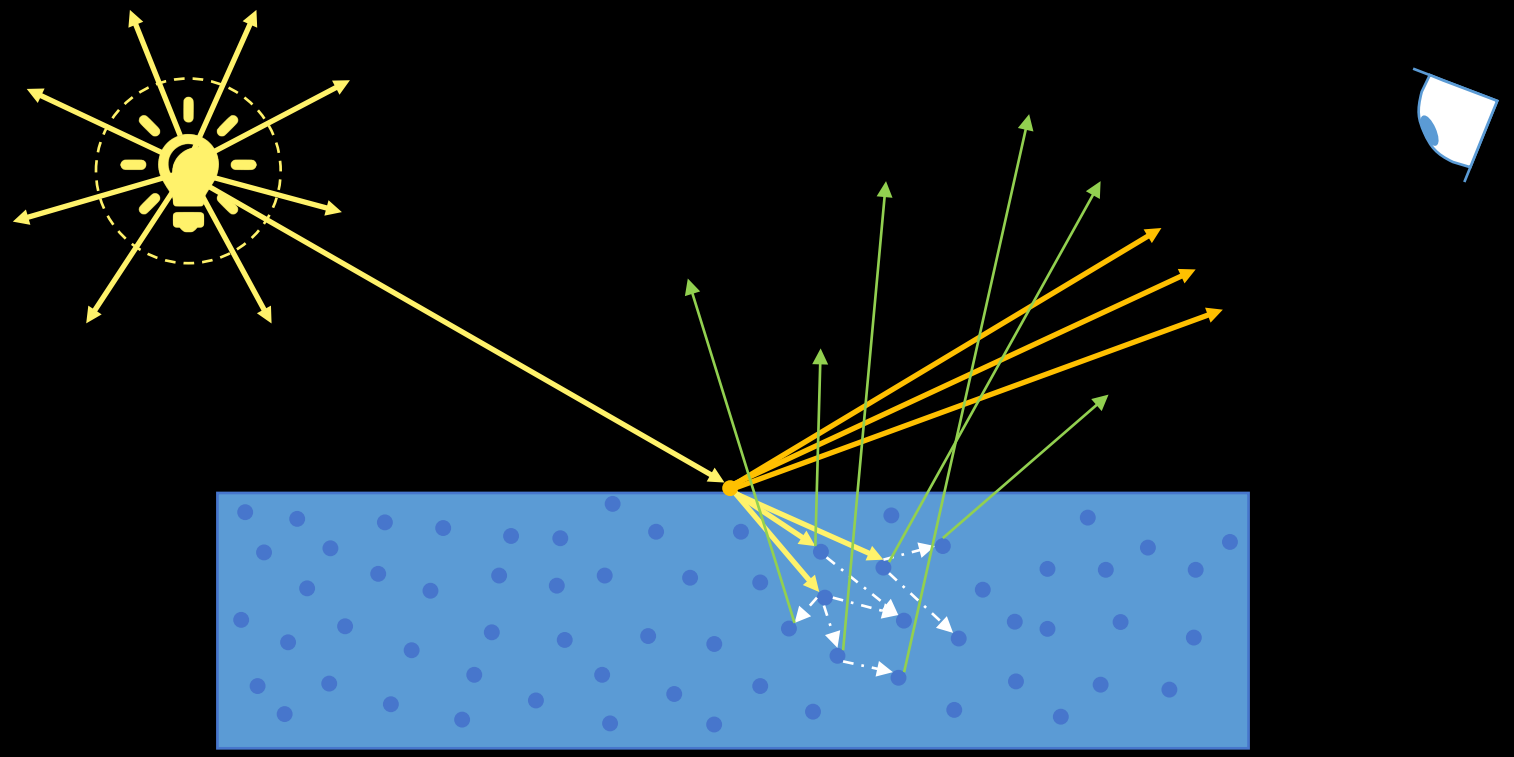
BRDFs

ACT V

Surface Variation

BRDF | LOBES

Let's take a closer look at the BRDF. One of its properties is that we can linearly combine BRDFs, and there is a good physical reason to do so. Watch what happens to the light when it hits the surface.

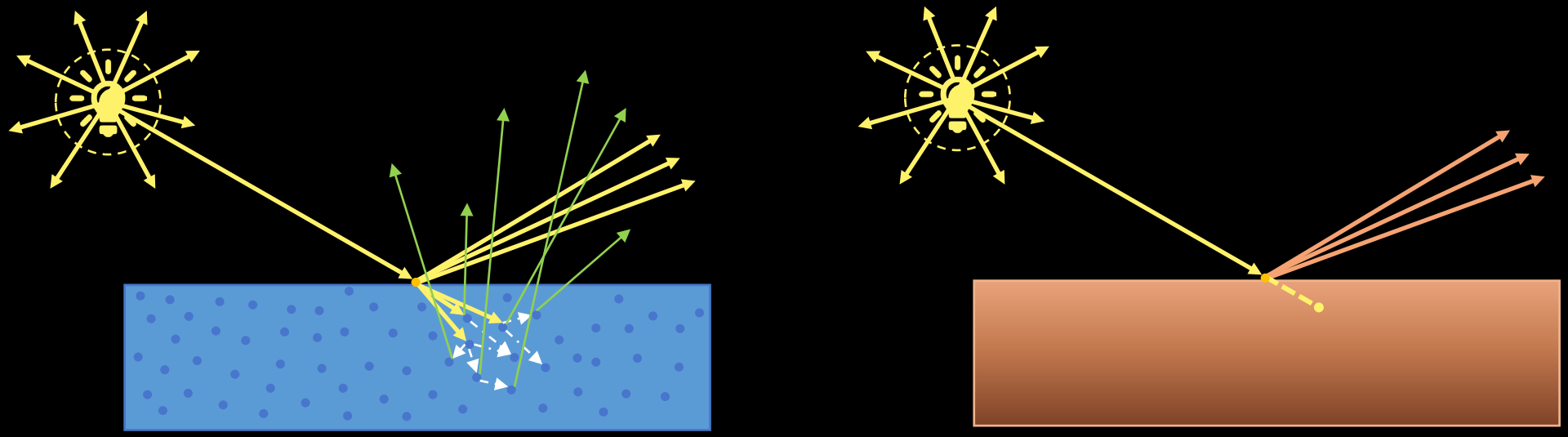


BRDF | LOBES

Part of the light reflects right at the surface, and part of it enters the material, scatters around and comes back out in all directions. So, we split the BRDF into two parts:

- Diffuse Reflection – the light that scattered inside the material.
- Specular Reflection – the light that reflected at the surface.

This is true for non-metals (**dielectrics**). Metals (**electrics**) absorb the light that enters almost immediately, nothing scatters back out. They only have a specular reflection, which takes on the metal's color.



Non-Metals

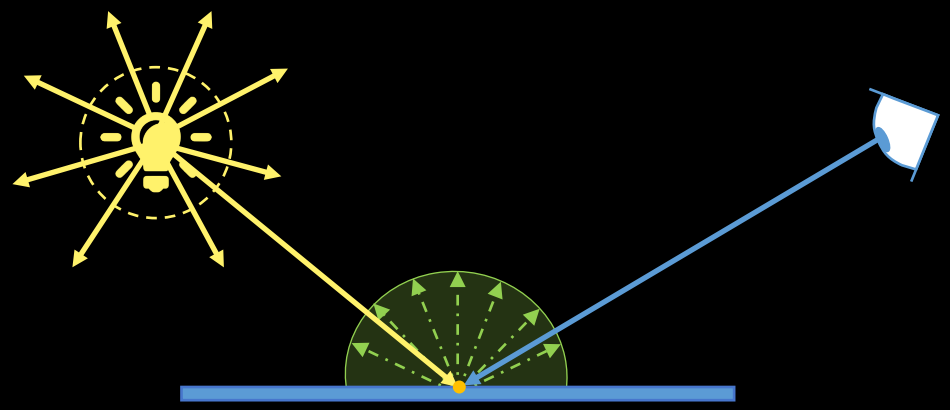
Metals

BRDF | DIFFUSE LOBE

Let's look at how we can implement both lobes. For diffuse, we use one of the simplest BRDFs, the Lambert BRDF. It is not physically accurate but works well for many materials.

Careful, despite the name, the Lambert BRDF is not the $cos\,\theta_i$ term in the rendering equation.
That cosine is a separate term, and the Lambert BRDF itself is just a constant.

The Lambert BRDF describes a perfect diffuse reflection. The reflected light is the same in every direction over the hemisphere, so the surface looks equally bright from every angle, independent of the view direction.
Because of this, the equation is very simple, a constant. This makes it ideal for basic matte objects.



BRDF | LAMBERT

As mentioned, the Lambert BRDF is a simple constant. The value ρ (rho, pron. "roe") is the diffuse reflectance of the surface, how much of the incoming light it reflects diffusely.

$$f_{X,n}(\vec{\omega}_{i,x}, \vec{\omega}_{o,x}) = \frac{\rho}{\pi}$$

The division by π makes sure a perfectly white surface, $\rho = 1$, reflects exactly all incoming light and never more, so the BRDF conserves energy.

Typical diffuse Lambert materials have the follow parameters:

- (kd) – Diffuse Coefficient [float] – how strongly the matte material reflects, between [0,1].
- (cd) – Diffuse Color [ColorRgba] – color of the material.

You can calculate ρ as a color by multiplying the diffuse color (cd) of the object with the diffuse reflection coefficient (kd). This is the albedo color of a non-metal.

BRDF | SPECULAR LOBE

Diffuse was the easy part, the same in every direction. The specular lobe is different, because it depends on both the light and the view direction. That is where the highlights come from.

No BRDF is physically accurate, as all of them are approximations. But some are more accurate than others, and the most accurate ones are called physically based. Hence the well-known term Physically Based Rendering or PBR, which relies on them.

For a BRDF to be physically based, it has to:

- have the BRDF properties previously discussed (positivity, reciprocity, linearity and energy conserving).
- be based on microfacet theory.
- use physically measured values, like the real reflectance of materials.

So, before we can implement the specular lobe, we need to know what the microfacet theory is.

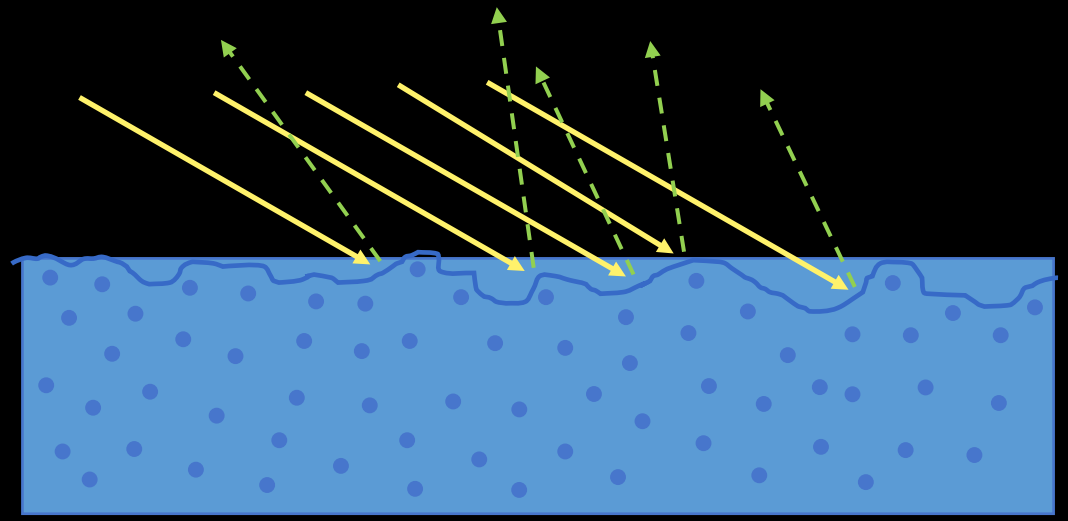
LET'S GET PHYSICAL

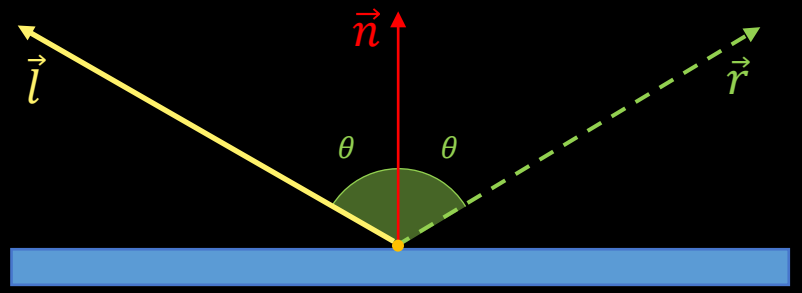


BRDF | SPECULAR LOBE

No surface is perfectly smooth, not even polished metal. Up close, every surface has tiny bumps and dents. Modeling each of them individually is impractical, so we describe them statistically, at the level of tiny facets that are smaller than a pixel but larger than the wavelength of light. This is called the microgeometry.

Physically based BRDFs are all about approximating how light behaves on this microgeometry, using parameters we can control, like roughness.

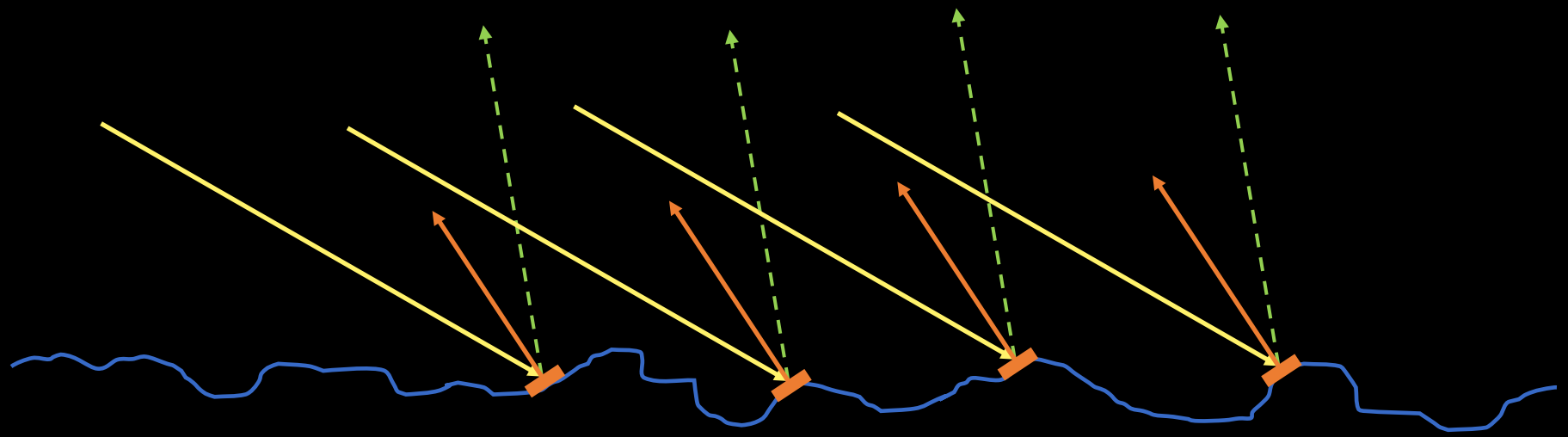




BRDF | SPECULAR LOBE

First, we model how rough the material is. In other words, how many of the microfacets on a surface area are aligned with the **half vector**.

Why do we use this half vector and not the normal of the surface?

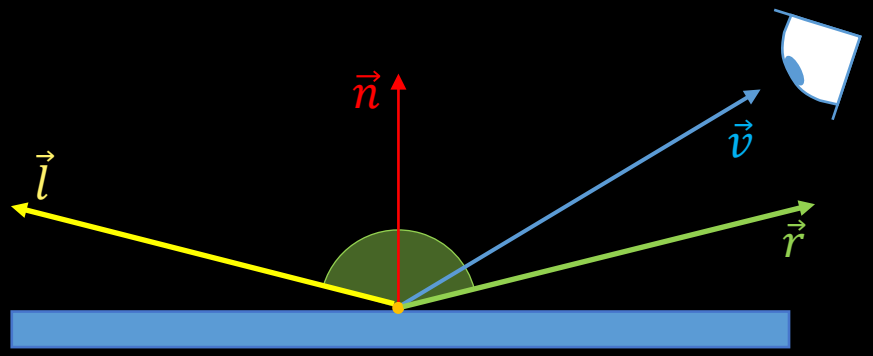


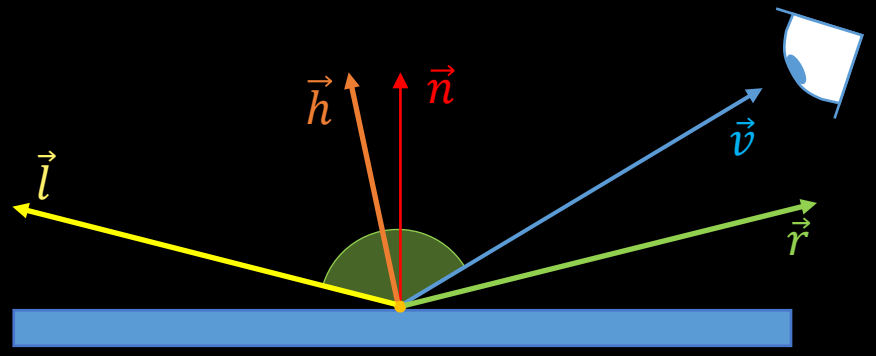
BRDF | SPECULAR LOBE

Each microfacet acts as a tiny perfect mirror. A mirror only reflects light from the light source toward the viewer when its normal lies exactly halfway between the light direction and the view direction.

That halfway direction is the half vector, so the microfacets aligned with it are exactly the ones that send light to the viewer.

The half vector is the normalized sum of the light and view direction: $\vec{h} = \frac{\vec{v} + \vec{l}}{\|\vec{v} + \vec{l}\|}$





BRDF | SPECULAR LOBE

To determine how rough the material is, we look at how the microfacet normals are spread around the surface normal $\vec{n}$. On a smooth surface almost all facets point along $\vec{n}$, on a rough surface they point in many directions.

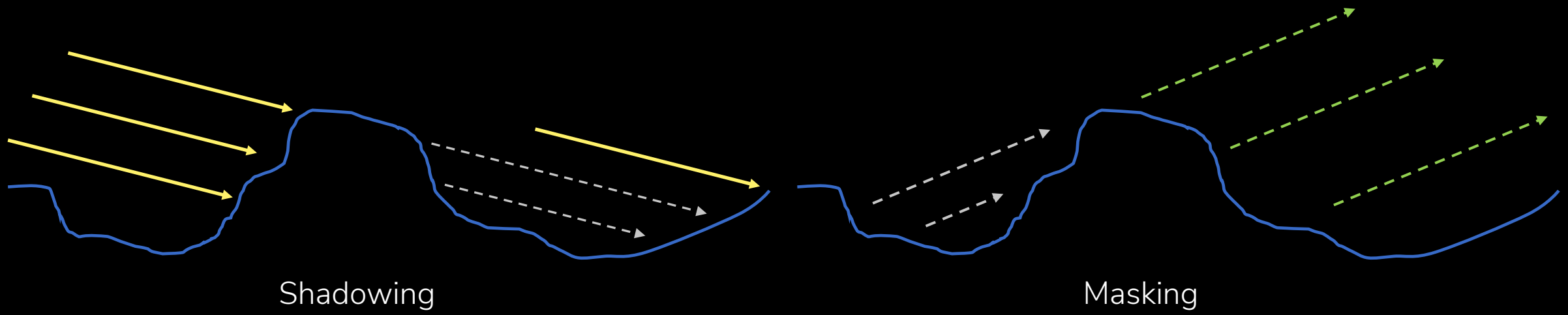
The function that tells us how many microfacets have their normal aligned with the half vector $\vec{h}$, and therefore, reflect light toward the viewer, is called the Normal Distribution Function (D).



BRDF | SPECULAR LOBE

Another property of rough surfaces is that microfacets block each other. Some incoming light never reaches a microfacet, because other microfacets are in the way. The same goes for reflected light, which can be blocked on its way to the viewer. Blocking the incoming light is called shadowing, and blocking the reflected light is called masking.

We again approximate this statistically, over a surface area, with a function called the Geometry Function ($\boldsymbol{G}$). It returns the fraction of microfacets that are both lit and visible, between 0 and 1. We don't trace rays for this, as that would be far too expensive.

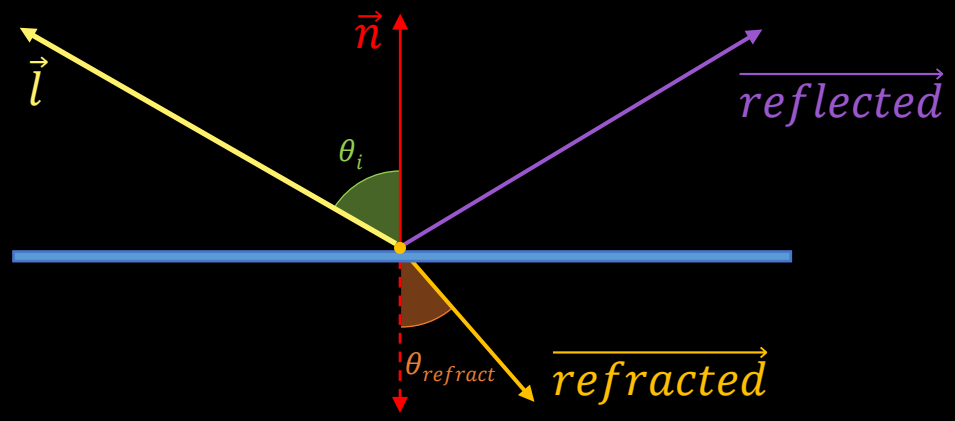


BRDF | SPECULAR LOBE

There is one last piece missing in our puzzle. So far, we looked at how many microfacets reflect towards us, based on the roughness or the material, and how much light the microfacets block.

What we have not used yet is how much of the light is reflected versus refracted. As we saw in the physics part, the Fresnel equations give us this ratio.

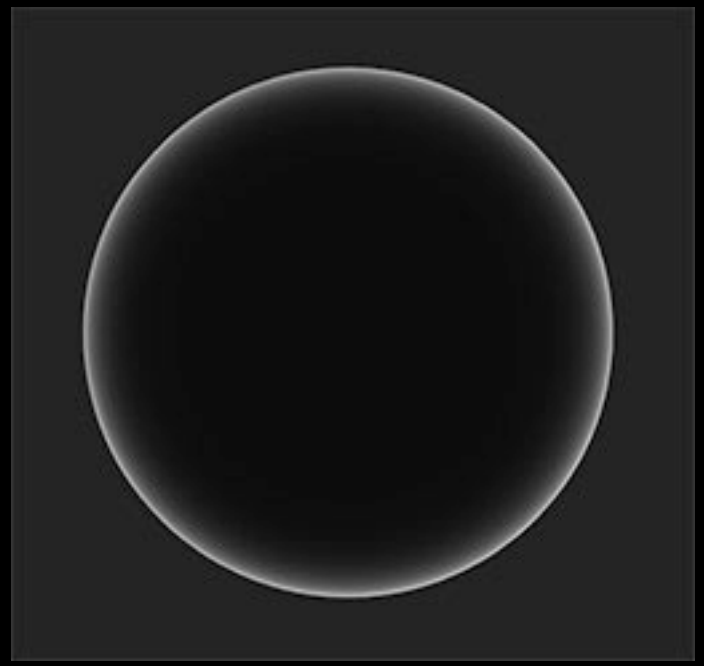
In our BRDF, the Fresnel term gives the fraction of light that is reflected, based on the view angle and the material. The rest is refracted into the material, where it can become diffuse light.

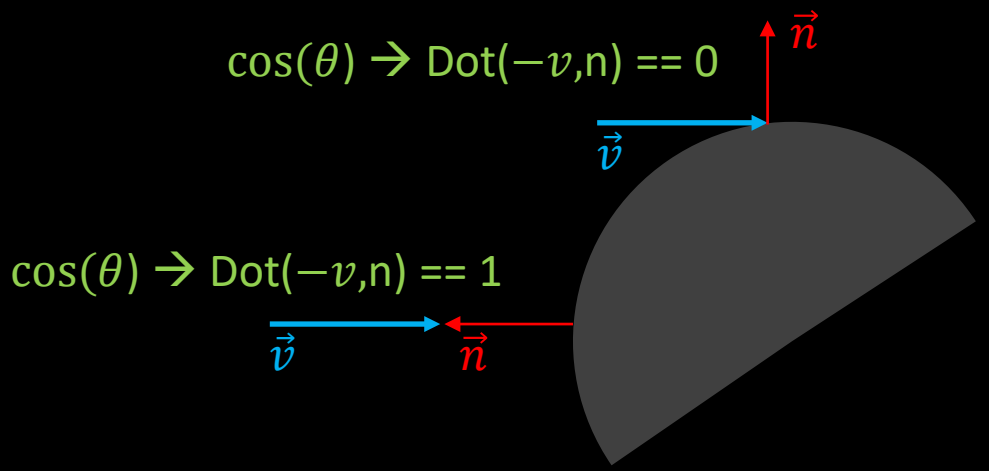


$$Snell's\ Law = \frac{\sin(\theta_i)}{\sin(\theta_{refract})}$$

BRDF | SPECULAR LOBE

To keep things simple, let's look at how Fresnel behaves when we only take into account the view direction and the normal of the surface. It has an interesting behavior.





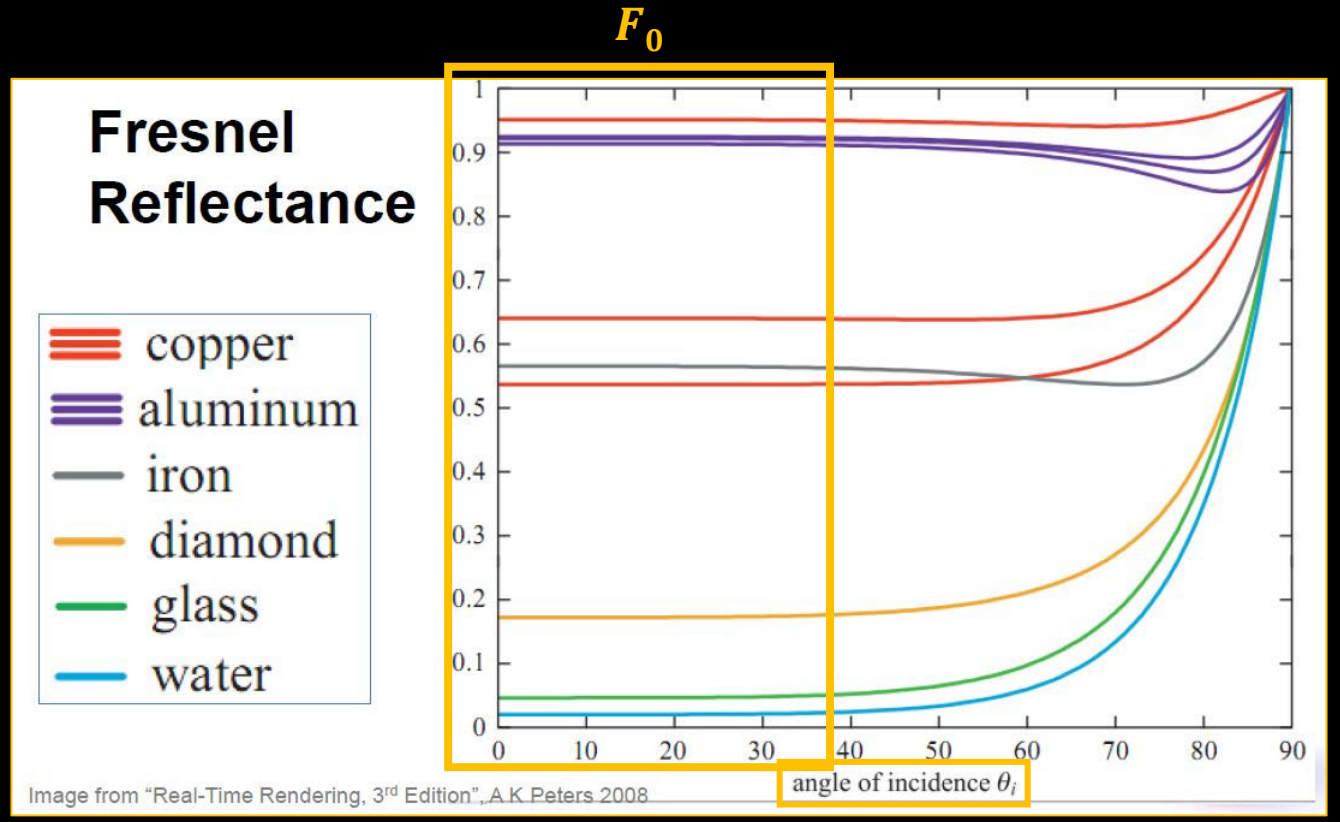
BRDF | SPECULAR LOBE

As we saw in the physics part, when the view direction and the normal are (almost) perpendicular, so their dot product is close to 0, the reflectance rises sharply toward 100%. The surface becomes mirror-like, the tint of its specular color fades to white and we see far more reflections of the environment.



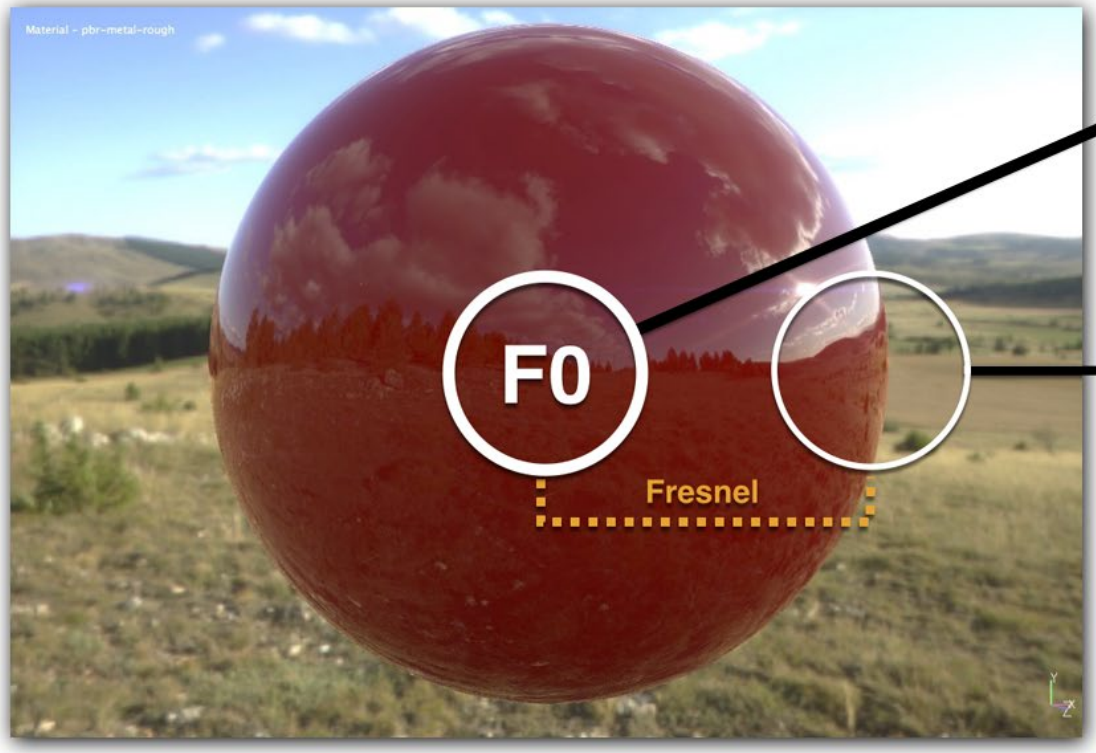
BRDF | SPECULAR LOBE

Some smart people measured this effect for different materials and put it in a graph.



Every material follows the same curve shape, only the starting value at 0 degrees differs.
This value F_0 is the surface's characteristic specular color.

BRDF | SPECULAR LOBE



F0 = 2-5%
Common dielectric materials

100%
Grazing angle

Substance Painter Viewport

https://www.adobe.com/learn/substance-3d-designer/web/the-pbr-guide-part-1

BRDF | SPECULAR LOBE

The indices of refraction in the table describe how strongly a material slows down light. They decide both where the refracted light goes, Snell's law, and how much light is reflected, the Fresnel equations.

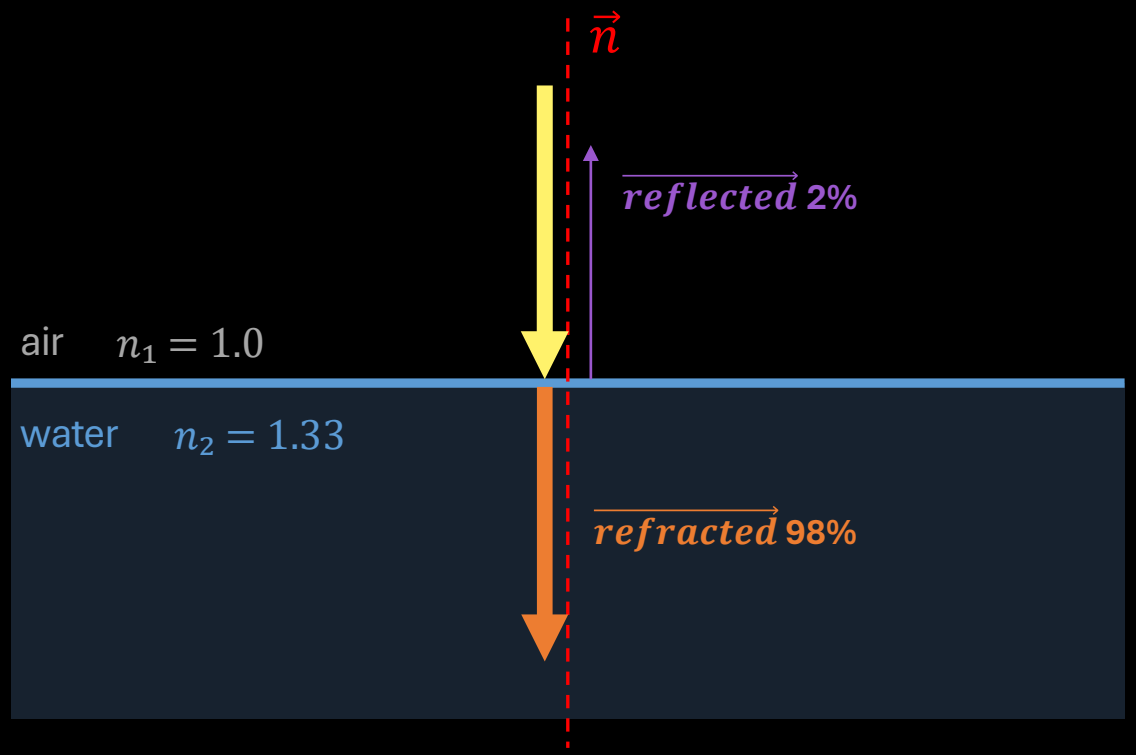
When we look straight at the surface, the Fresnel equations simplify to a formula that only needs the two indices, $F0 = \left(\frac{n_1 - n_2}{n_1 + n_2}\right)^2$. For air into water this gives $F0 \approx 0.02$.

In our tracer, light always travels between air or vacuum and a material, so we can use these values directly. Light between two other materials, like water and glass in an underwater scene, needs both of their indices.

Indices of refraction	
medium	index of refraction n
vacuum	1.00 exactly
air	1.0003
water	1.33
oil	1.46
glass	1.50
diamond	2.41

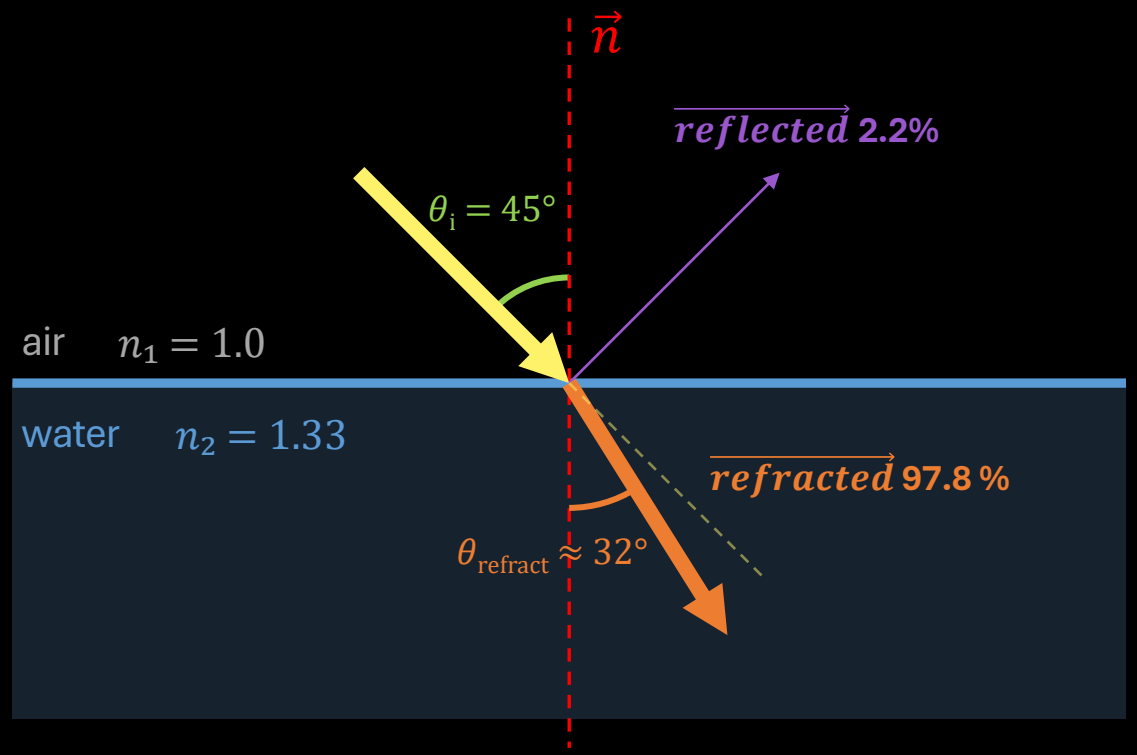
BRDF | SPECULAR LOBE

looking straight at the surface, 0°



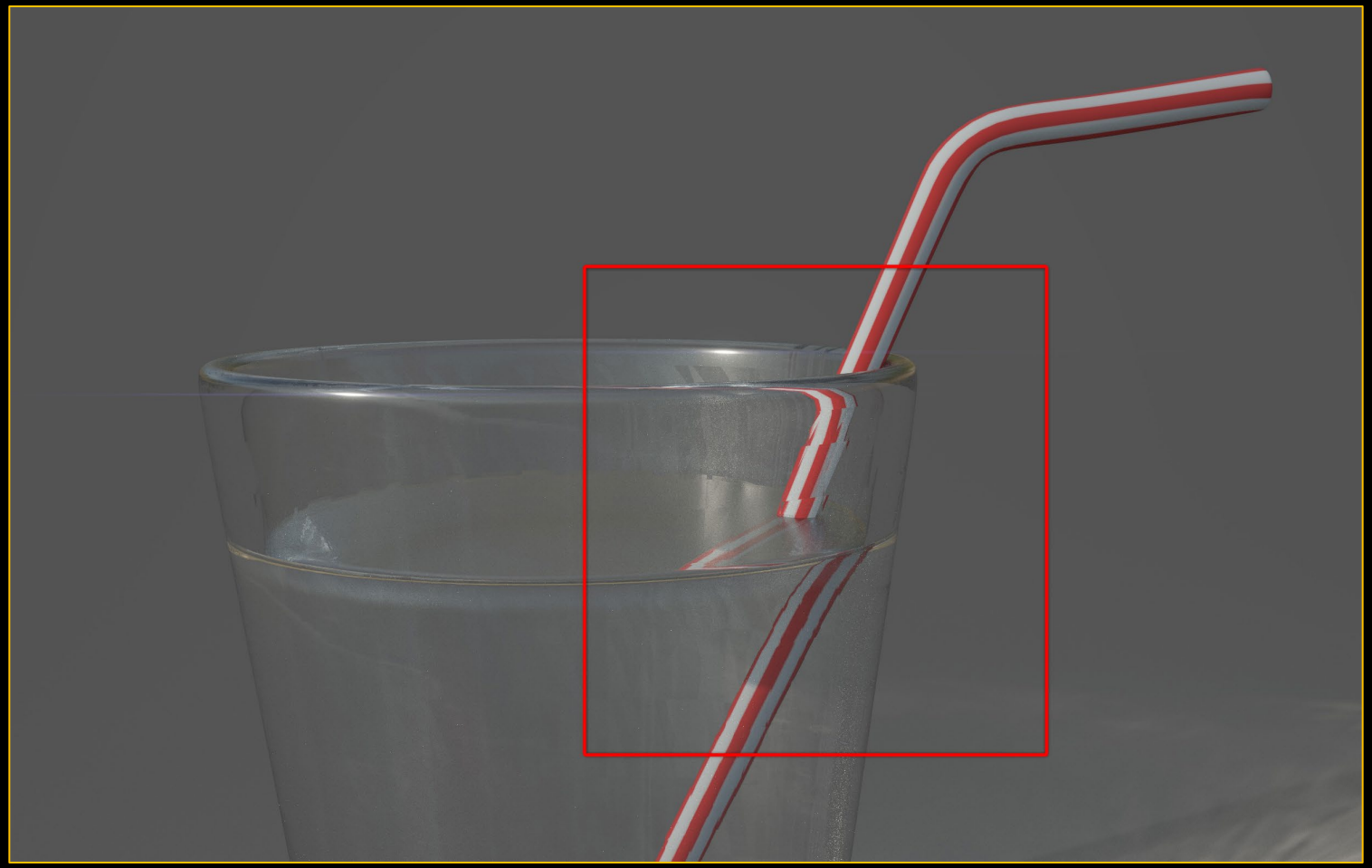
$$F_0 = \left(\frac{n_1 - n_2}{n_1 + n_2}\right)^2 = \left(\frac{1 - 1.33}{1 + 1.33}\right)^2 \approx 0.02$$

light arriving at 45°



$$\frac{\sin(\theta_i)}{\sin(\theta_{refract})} = \frac{n_2}{n_1} \rightarrow \frac{\sin(45°)}{\sin(\theta_{refract})} = \frac{1.33}{1.0} \rightarrow \theta_{refract} \approx 32°$$

BRDF | SPECULAR LOBE



https://www.adobe.com/learn/substance-3d-designer/web/the-pbr-guide-part-1

BRDF | SPECULAR LOBE

Dielectric	Linear	Texture	Color	Notes
Water	0.02	39		
Living tissue	0.02–0.04	39–56		Watery tissues are toward the lower bound, dry ones are higher
Skin	0.028	47		
Eyes	0.025	44		Dry cornea (tears have a similar value to water)
Hair	0.046	61		
Teeth	0.058	68		
Fabric	0.04–0.056	56–67		Polyester highest, most others under 0.05
Stone	0.035–0.056	53–67		Values for the minerals most often found in stone
Plastics, glass	0.04–0.05	56–63		Not including crystal glass
Crystal glass	0.05–0.07	63–75		
Gems	0.05–0.08	63–80		Not including diamonds and diamond simulants
Diamond-like	0.13–0.2	101–124		Diamonds and diamond simulants (e.g., cubic zirconia, moissanite)

Metal	Linear	Texture	Color
Titanium	0.542,0.497,0.449	194,187,179	
Chromium	0.549,0.556,0.554	196,197,196	
Iron	0.562,0.565,0.578	198,198,200	
Nickel	0.660,0.609,0.526	212,205,192	
Platinum	0.673,0.637,0.585	214,209,201	
Copper	0.955,0.638,0.538	250,209,194	
Palladium	0.733,0.697,0.652	222,217,211	
Mercury	0.781,0.780,0.778	229,228,228	
Brass (C260)	0.910,0.778,0.423	245,228,174	
Zinc	0.664,0.824,0.850	213,234,237	
Gold	1.000,0.782,0.344	255,229,158	
Aluminum	0.913,0.922,0.924	245,246,246	
Silver	0.972,0.960,0.915	252,250,245	

BRDF | SPECULAR LOBE

We can also see that the range of non-metals (dielectrics) is much smaller, roughly 0.02 to 0.05, compared to metals, which range from about 0.5 to 1.0 and are often colored.

So, besides the fact that metals do not scatter diffuse light, they also have a very different range for their specular color.

Most game engines simplify this. They give all dielectrics a default value of 0.04, like plastic, while metals take their specular color from a texture. This is called the metalness workflow. Unreal Engine uses it, and other engines like Unity let you choose between this metalness workflow and the specular workflow.

We will use the metalness workflow! → Requirement!

BRDF | SPECULAR LOBE

There are different methods to implement every function. Have a look at the following page to find more methods, and feel free to implement and compare them. But luckily for us there are already smart people who did this for us and picked the “right” methods to get a good result. Do keep in mind that there are still a lot of advancements in this area!

More methods at http://graphicrants.blogspot.com/2013/08/specular-brdf-reference.html

We are going to use the following functions.

- $\boldsymbol{D}$, Trowbridge-Reitz GGX
- $\boldsymbol{F}$, Schlick
- $\boldsymbol{G}$, Schlick-GGX (Schlick-Beckmann with remapped roughness)

Let’s dive into some of the implementations!



https://link.springer.com/article/10.1007/s00371-026-04701-w

BRDF | COOK-TORRANCE

This brings us to the PBR BRDF, that takes the three functions we've discussed into account. Just as Unreal Engine 4, we are going to use the Cook-Torrance BRDF, which looks like this.

$$f_{cook-torrance} = \frac{DFG}{4(\vec{v} \cdot \vec{n})(\vec{l} \cdot \vec{n})}$$

D, Normal Distribution Function → How many microfacets point in right direction?
F, Fresnel Function → How reflective are the microfacets?
G, Geometry Function → How much shadowing and masking is happening because of the microfacets?

The denominator converts between the microfacets and the surface as a whole. We will not go deeper into it, but if you want to, read paper "Microfacet models for refraction through rough surfaces" by Walter et. al.: https://dl.acm.org/doi/10.5555/2383847.2383874

BRDF | COOK-TORRANCE

For the Normal Distribution function, we are going to use Trowbridge-Reitz GGX, which looks as follows.

$$N_{float}(\vec{n}, \vec{h}, \alpha) = \frac{\alpha^2}{\pi\left((\vec{n} \cdot \vec{h})^2(\alpha^2 - 1) + 1\right)^2}$$

So, you are going to need to use the half vector, which we compare with the surface normal, with α being the roughness parameter.

These are the input parameters.

- $\vec{n}$, normal of the surface / hit point
- $\vec{h}$, half vector between view direction and light direction
- α , roughness value squared → squared for perceptual linearity for artists!

For the α parameter, we use the same definition as Disney and Unreal Engine 4, which is → α = roughness^2 = r^2
Important note: α^2 = (roughness^2)^2

BRDF | COOK-TORRANCE

For the Fresnel function we are going to use Schlick's approximation, which looks as follows.

$$F_{rgb}(\vec{h}, \vec{v}, F_0) = F_0 + (1 - F_0)\left(1 - (\vec{h} \cdot \vec{v})\right)^5$$

As mentioned before, it describes the reflectivity of the microfacets.

These are the input parameters.

- $\vec{h}$, half vector between view direction and light direction
- $\vec{v}$, view direction
- F_0 , the base reflectivity of the surface → which is (0.04, 0.04, 0.04) for dielectrics or the albedo value for metals.

Determining which value to use for F_0 is crucial! You can easily achieve this by using a metalness value (0 or 1) → F_0 = (metalness == 0) ? (0.04, 0.04, 0.04) : Albedo_{rgb}

BRDF | COOK-TORRANCE

Finally, for the Geometry function we are going to use Schlick-GGX (Schlick-Beckmann with remapped roughness), which looks as follows.

$$G_{float}(\vec{n}, \vec{v}, k) = \frac{\vec{n} \cdot \vec{v}}{(\vec{n} \cdot \vec{v})(1 - k) + k}$$

It describes how microfacets shadow and mask each other.

These are the input parameters.

- $\vec{n}$, normal of the surface / hitpoint
- $\vec{v}$, view direction
- k , the roughness r remapped, depending on whether we use the function for direct or indirect lighting. We use direct lighting.

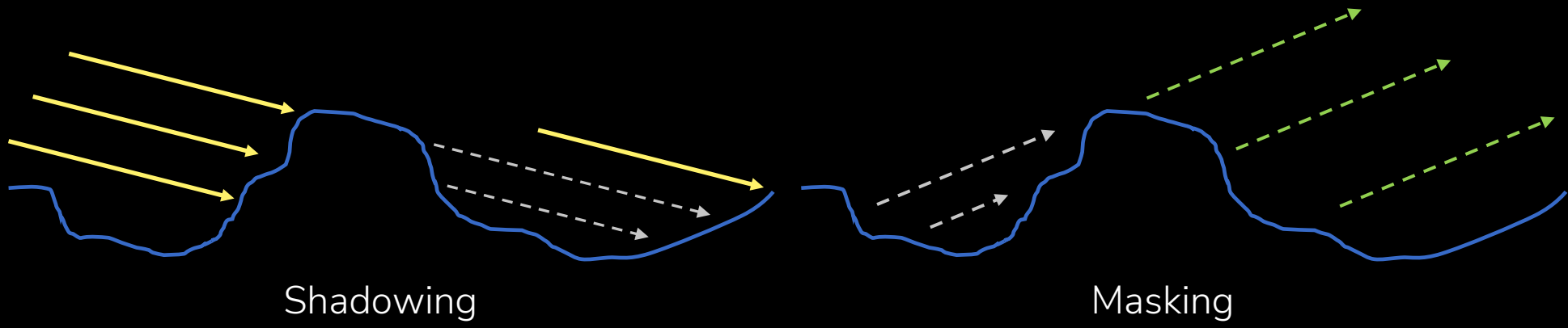
$$k_{direct} = \frac{(r + 1)^2}{8} \qquad k_{indirect} = \frac{r^2}{2}$$

BRDF | COOK-TORRANCE

To get a correct approximation we will have to use this function for both the shadowing and the masking! Both can be combined using the Smith's method.

$$G_{Smith}(\vec{n}, \vec{v}, \vec{l}, k) = G(\vec{n}, \vec{v}, k)\, G(\vec{n}, \vec{l}, k)$$

So, you just use the previous method, calculate it for both the masking (using view direction) and the shadowing (light direction), and you multiply both values.



BRDF | COOK-TORRANCE

Typical Cook Torrance materials have the follow parameters:

- Albedo Color [ColorRgba] –diffuse color is non-metal, else F0 when metal.
- Metalness [bool] – 0 when non-metal, 1 when metal.
- Roughness [float] – roughness of material, 0.001 = smooth while 1.0 = rough.
  Make sure never to use 0, else you cancel everything out.
- … (IOR – Index of Refraction, Transmission, Thickness, etc.) → We will discuss later…

When implementing the entire Cook-Torrance, thus using all discussed terms, this is the pseudocode that can be used in your material.

```
Determine F0 value → (0.04, 0.04, 0.04) or Albedo based on Metalness
Calculate half vector between view direction and light direction
Calculate Fresnel (F)
Calculate Normal Distribution (D)
Calculate Geometry (G)
Calculate specular => Cook-Torrance → (DFG)/4(dot(v,n)dot(l,n))
Determine kd -> 1 - Fresnel, cancel out if it's a metal (kd = 0)
Calculate Diffuse => BRDF::Lambert using the kd
Return final color -> diffuse + specular
```

ACT I

Light & Matter

ACT II

From Physics to Model

ACT III

Inside the Equation

ACT IV

BRDFs

ACT V

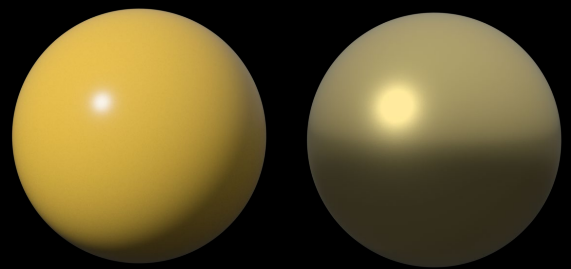
Surface Variation

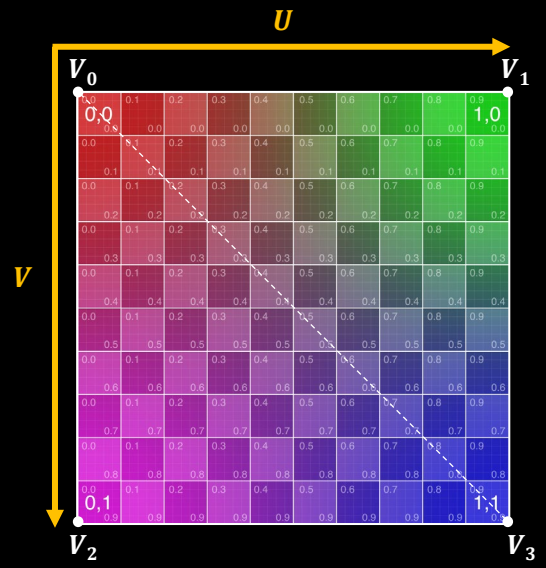
SURFACE VARIATION | TEXTURES

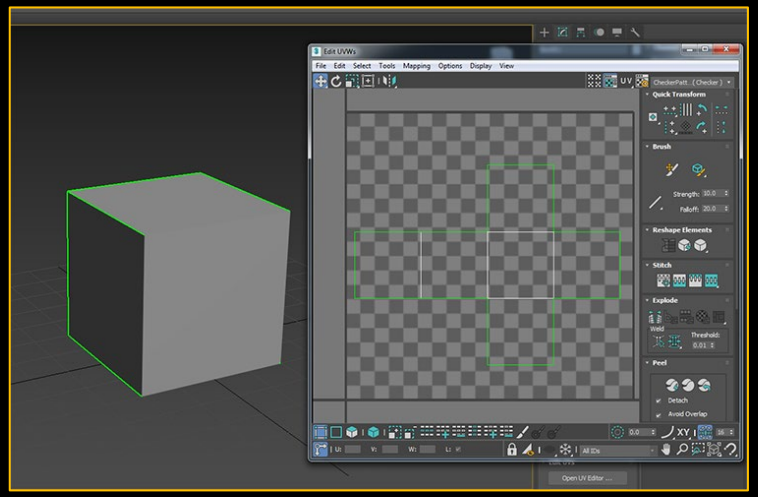
We could use hardcoded values (albedo, F_0, roughness, etc.) that are the same for the entire surface, or we can let these values vary across the surface of our 3D meshes using textures.

Vertices can have attributes, as mentioned before, and one of them are the texture or UV coordinates. These are defined by the artists in their DCC.

Between the vertices, we interpolate the UV coordinates with barycentric coordinates and use them to look up a texel, a single pixel of the texture. That gives every point on the surface its own value.

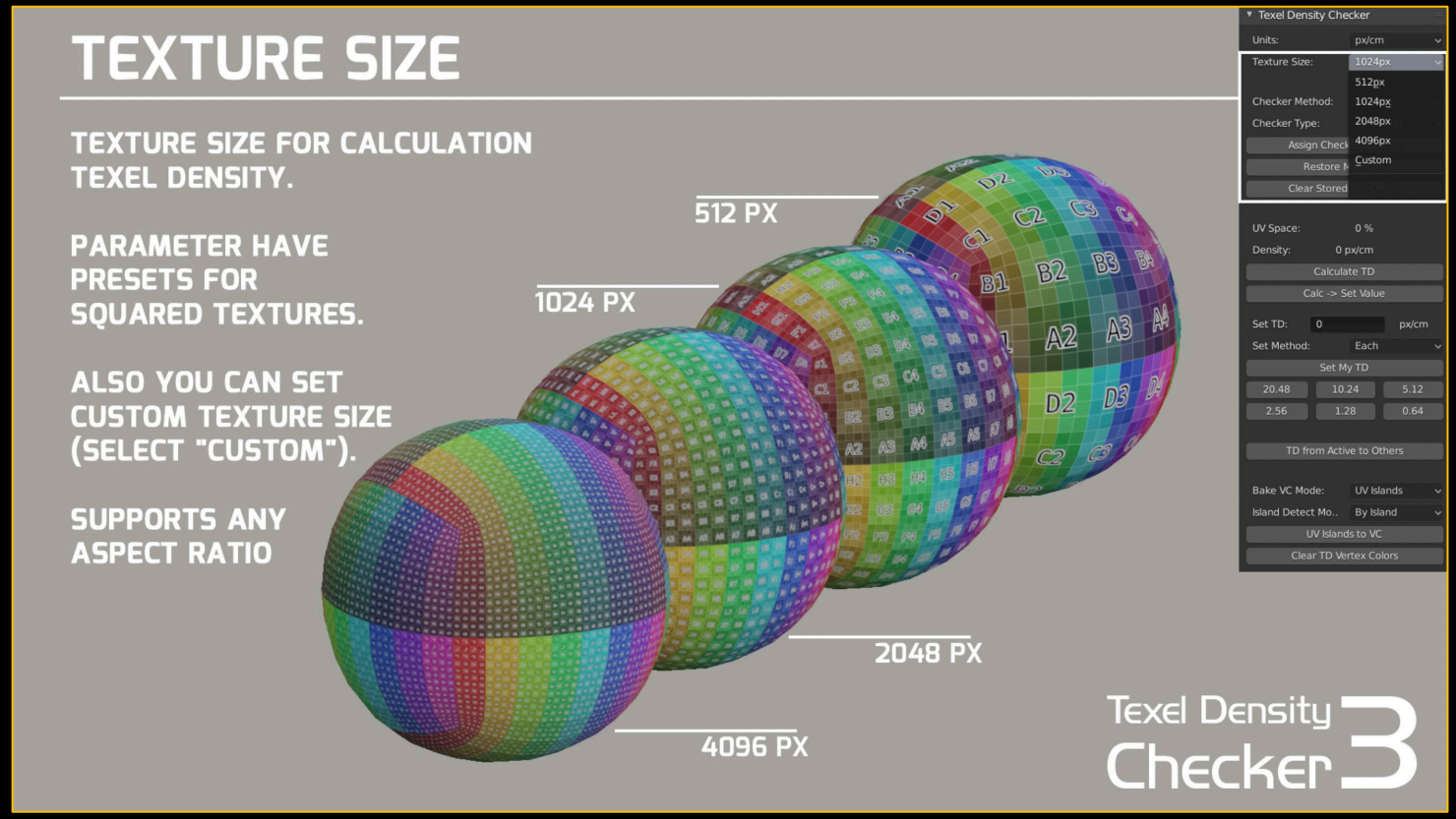






SURFACE VARIATION | TEXTURES

The texel density, the number of texels per unit of surface, determines how much variation a texture can store. Finer detail needs more texels on the same area.

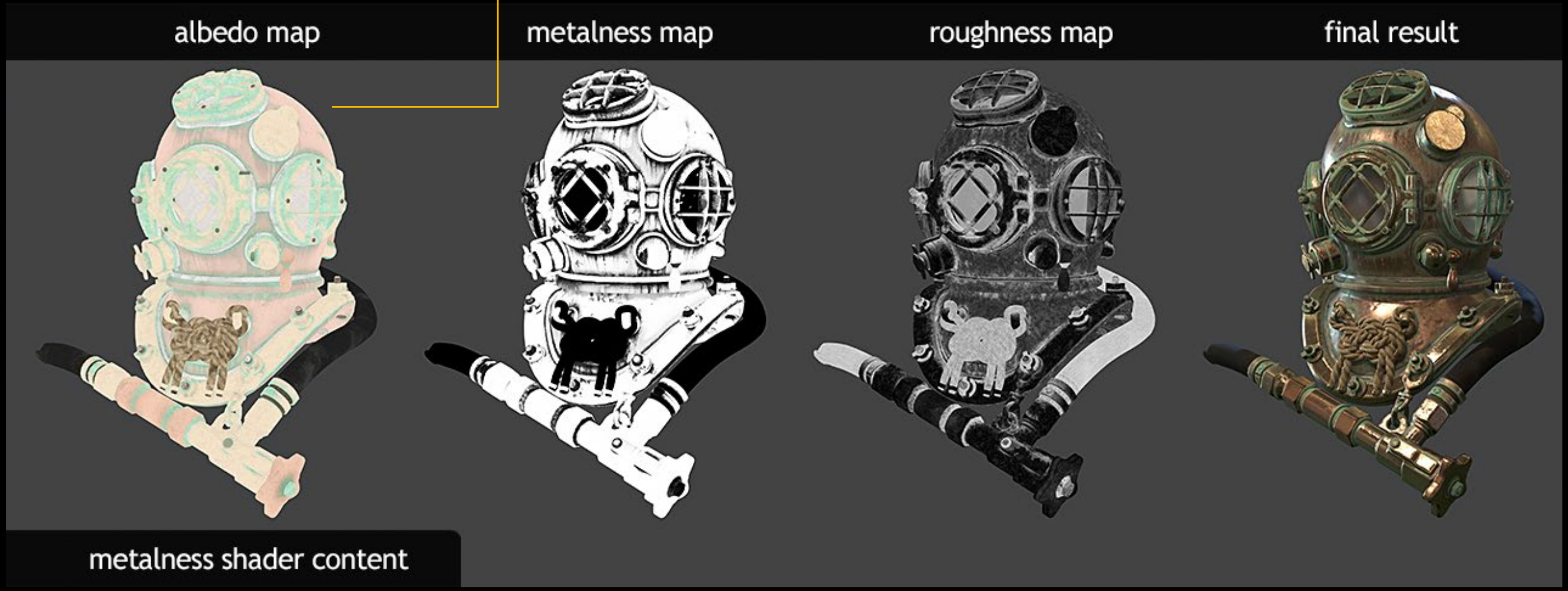


https://80.lv/articles/a-cool-add-on-for-working-with-texel-density-in-blender

SURFACE VARIATION | TEXTURES

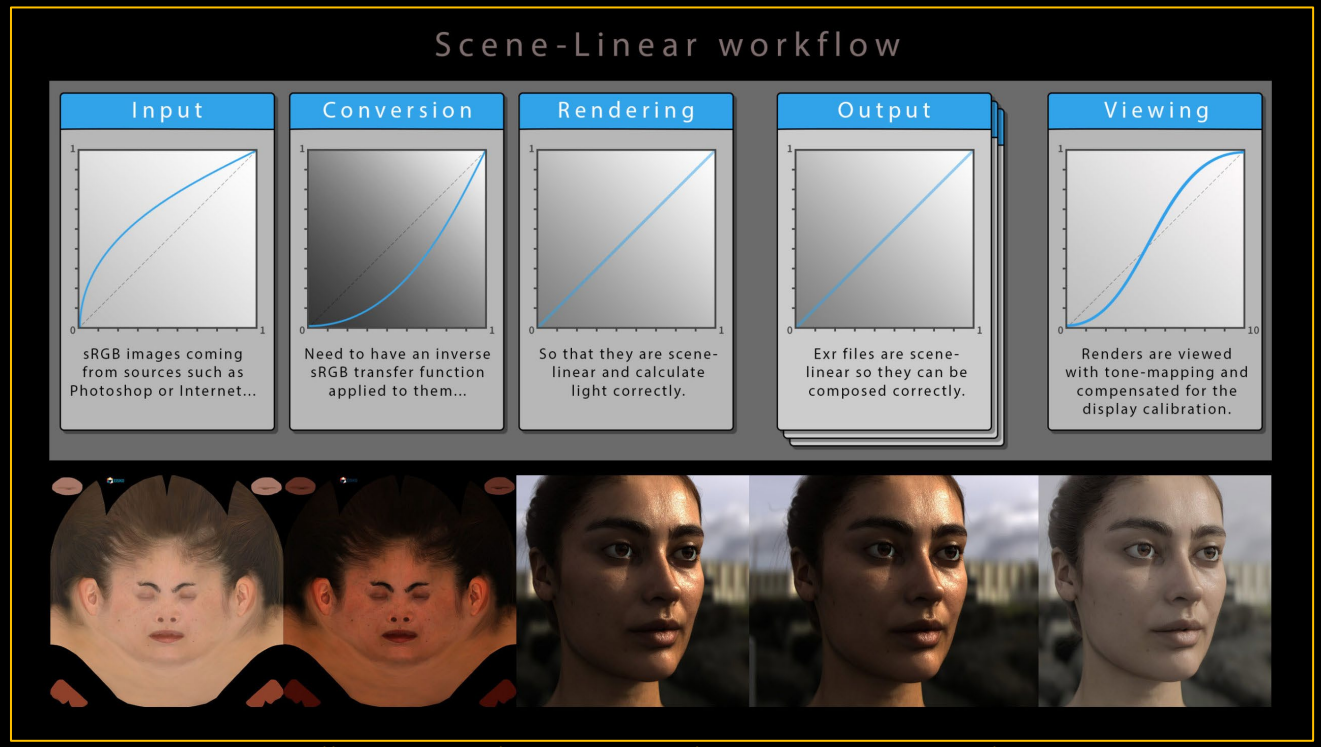
Platinum	0.673,0.637,0.585	214,209,201
Copper	0.955,0.638,0.538	250,209,194
Palladium	0.733,0.697,0.652	222,217,211

The Cook-Torrance BRDF values are often stored in different textures!



SURFACE VARIATION | TEXTURES

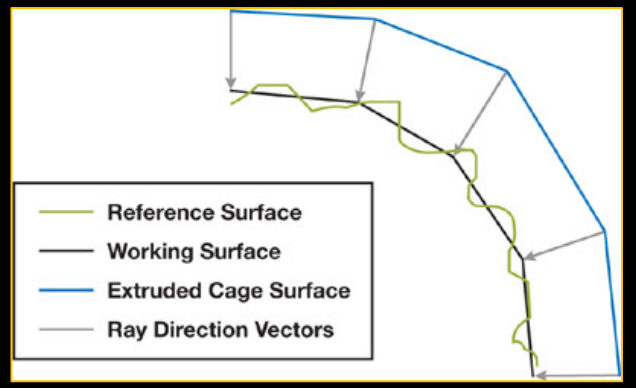
Retrieving the texel value of a texture, using the UV coordinates, is called sampling as well. Textures, depending on what they store (only albedo for us), can also be in sRGB space. So, make sure to always convert to linear space when needed! We call this remapping the range.

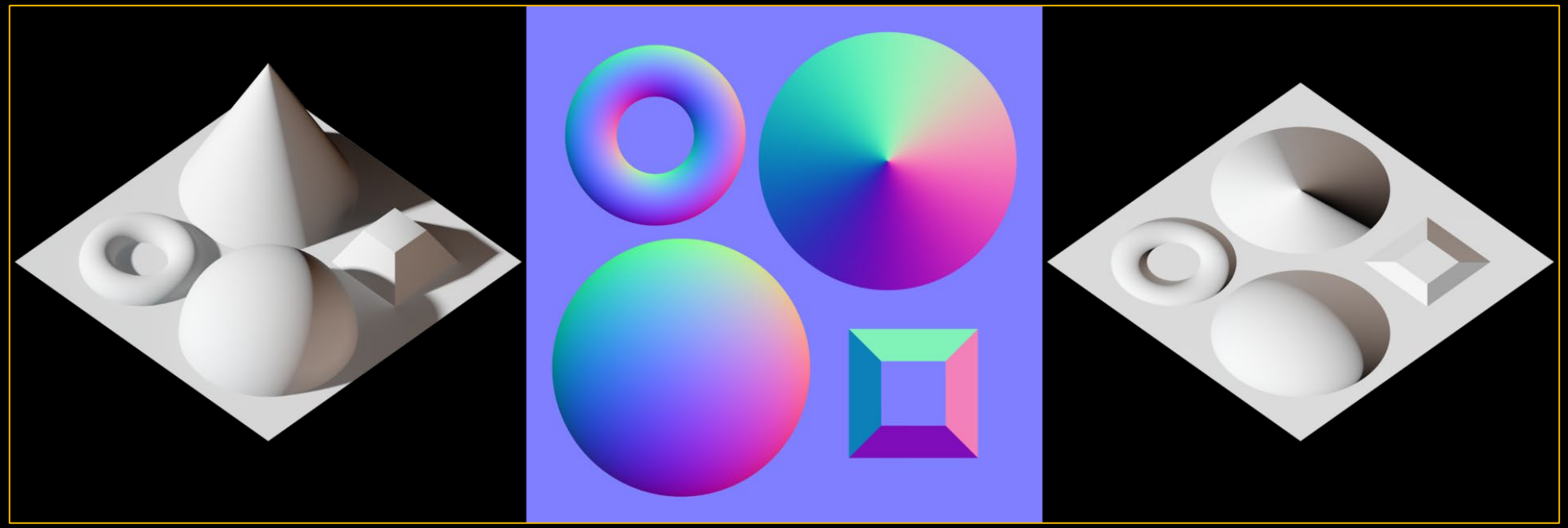


https://chrisbrejon.com/cg-cinematography/chapter-1-color-management/

SURFACE VARIATION | TEXTURES

Textures do not only store colors, but they can also store data like roughness values or even vectors like normals. So far, we have used the interpolated vertex normals. Artists often create a high-detail version of a 3D mesh and bake its details into a normal map texture. This way, a simple mesh gets a normal per texel instead of per vertex, so it looks detailed without the extra geometry.





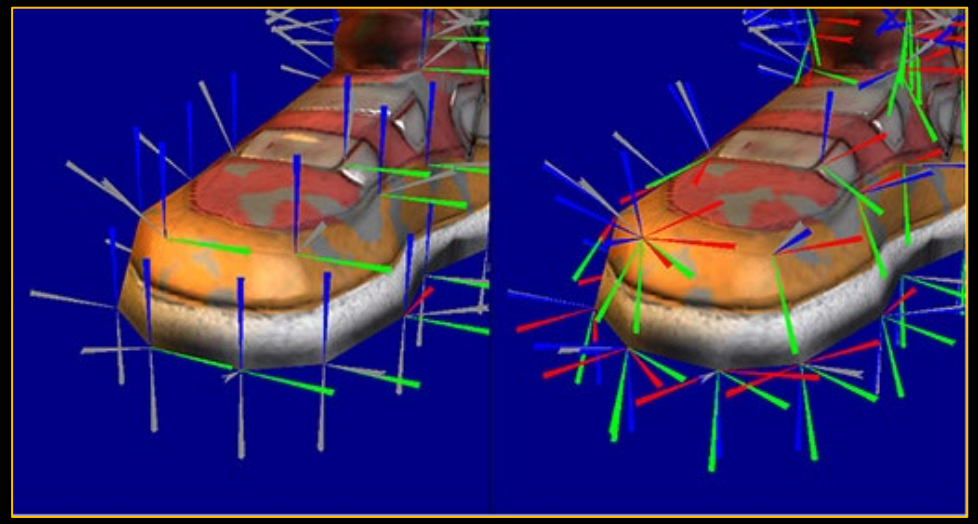
https://developer.nvidia.com/gpugems/gpugems3/part-iv-image-effects/chapter-22-baking-normal-maps-gpu

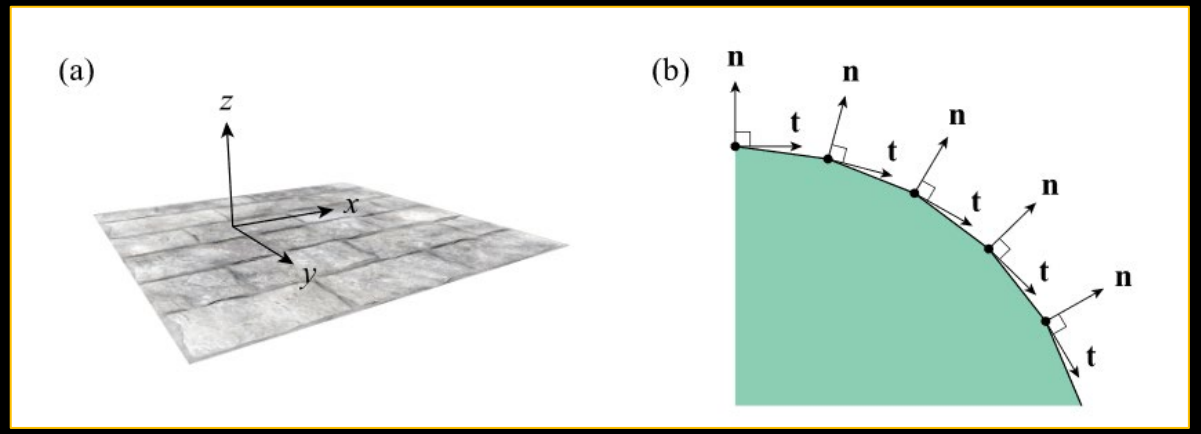
SURFACE VARIATION | TEXTURES

Why do normal maps look purplish?

They store the normal in tangent space, where the z-axis points away from the surface, along the normal. A normal has values between [−1, 1], but a texture stores values between [0, 1]. So, we remap each component with $\boldsymbol{n} \cdot 0.5 + 0.5$.

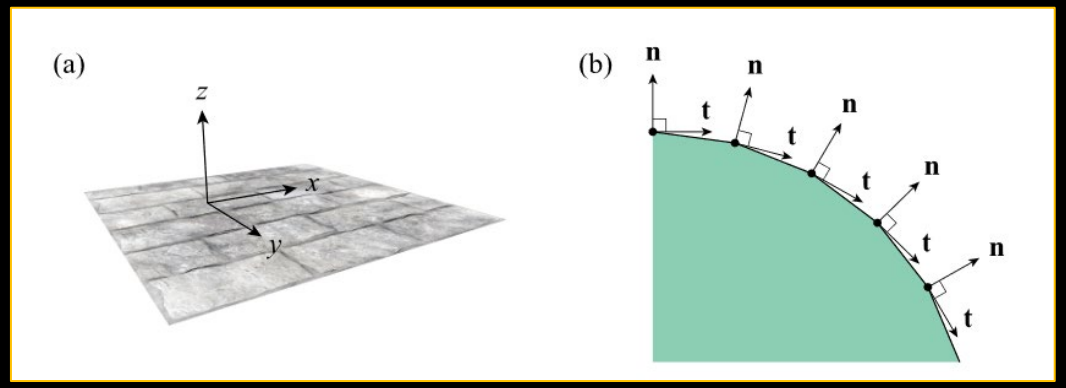
A flat surface has normal (0, 0, 1), which becomes (0.5, 0.5, 1.0), so RGB (128, 128, 255), that light blue-purple. Since most normals point roughly straight out of the surface, most texels are close to this color.

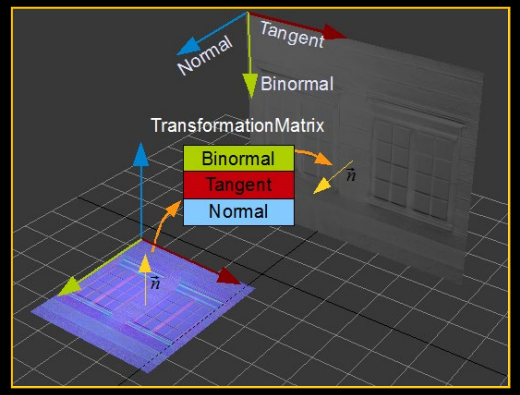




SURFACE VARIATION | TEXTURES

Normal maps store normals in tangent space, not in model space, and that tangent space differs per point on the surface. So, we must transform each sampled normal from tangent space to world space.



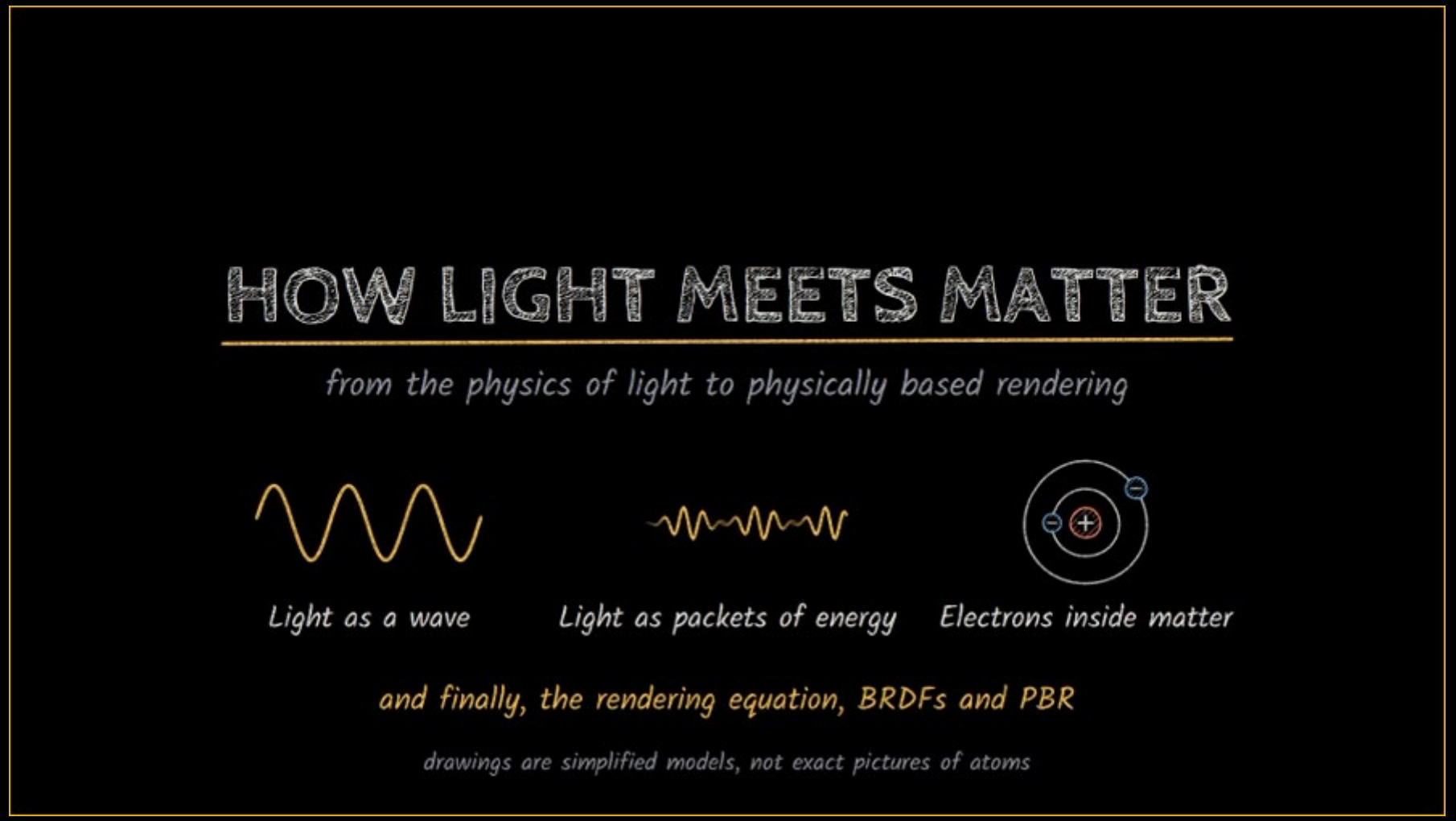


How can we build this transformation while shading a pixel?

The vertex normal and tangent are provided for you (see Vertex struct) in local space. ConstructShadingInput interpolates them across the triangle, and you should bring them into world space using the instance transformation matrix. This gives us the three needed vectors of the ONB, here called TBN, basis:

- The interpolated vertex normal $\boldsymbol{N}$.
- The interpolated vertex tangent $\boldsymbol{T}$.
- The interpolated vertex bitangent $\boldsymbol{B}$.

PHYSICS OF LIGHT | THE FULL STORY

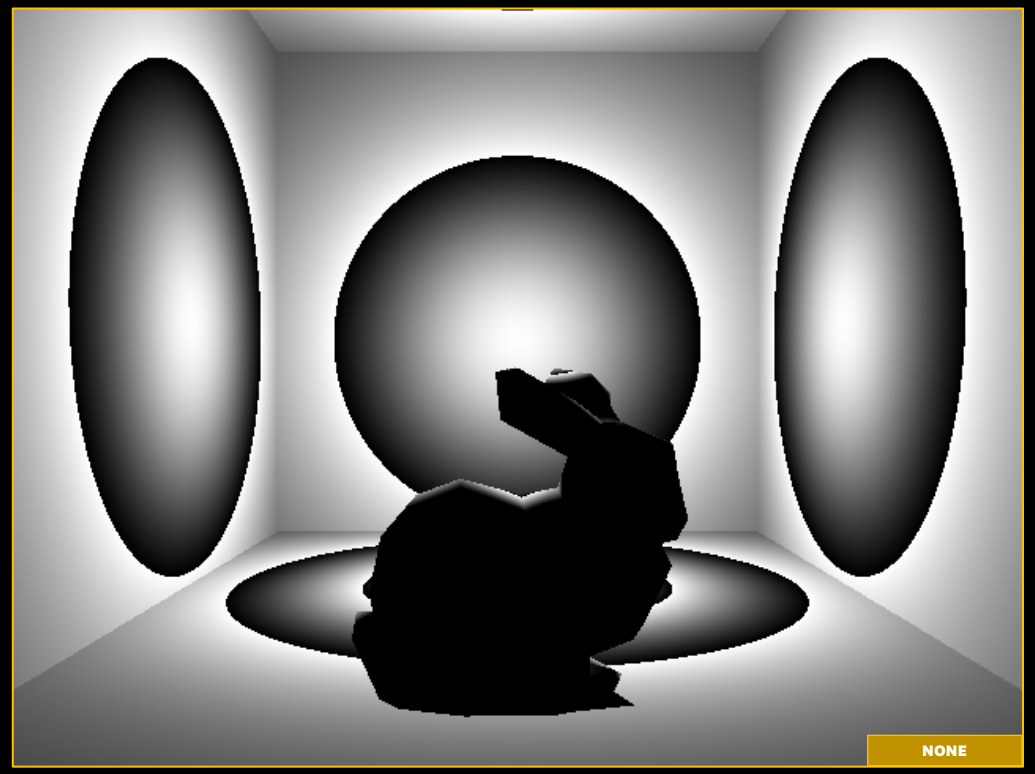


https://youtu.be/y9hMvkgy5pg?si=X3q-N8neowvsaPzD

PRAXIS

Step 00 | Radiance

Add radiance calculation and see the bad result below!




```cpp
//============================================
// Step 00 - Observed Radiance
// In the pixel loop, iterate the scene's lights via lights_factory.GetAll().
// Build the light direction (directional lights point the other way),
// and note Normalize() hands you the distance as a return value (which you
// might want to use later, for now you can ignore this).
Vector3 light_dir = light->type == LightType::kDirectional
    ? -light->direction
    : (light->origin - shading_input.world_position);
const float light_distance = light_dir.Normalize();

// use the observed area (Lambert's cosine law). Lights below the horizon of
// the surface contribute nothing.
const float cos_theta = Vector3::Dot(shading_input.world_normal, light_dir);
if (cos_theta <= 0.f) continue;

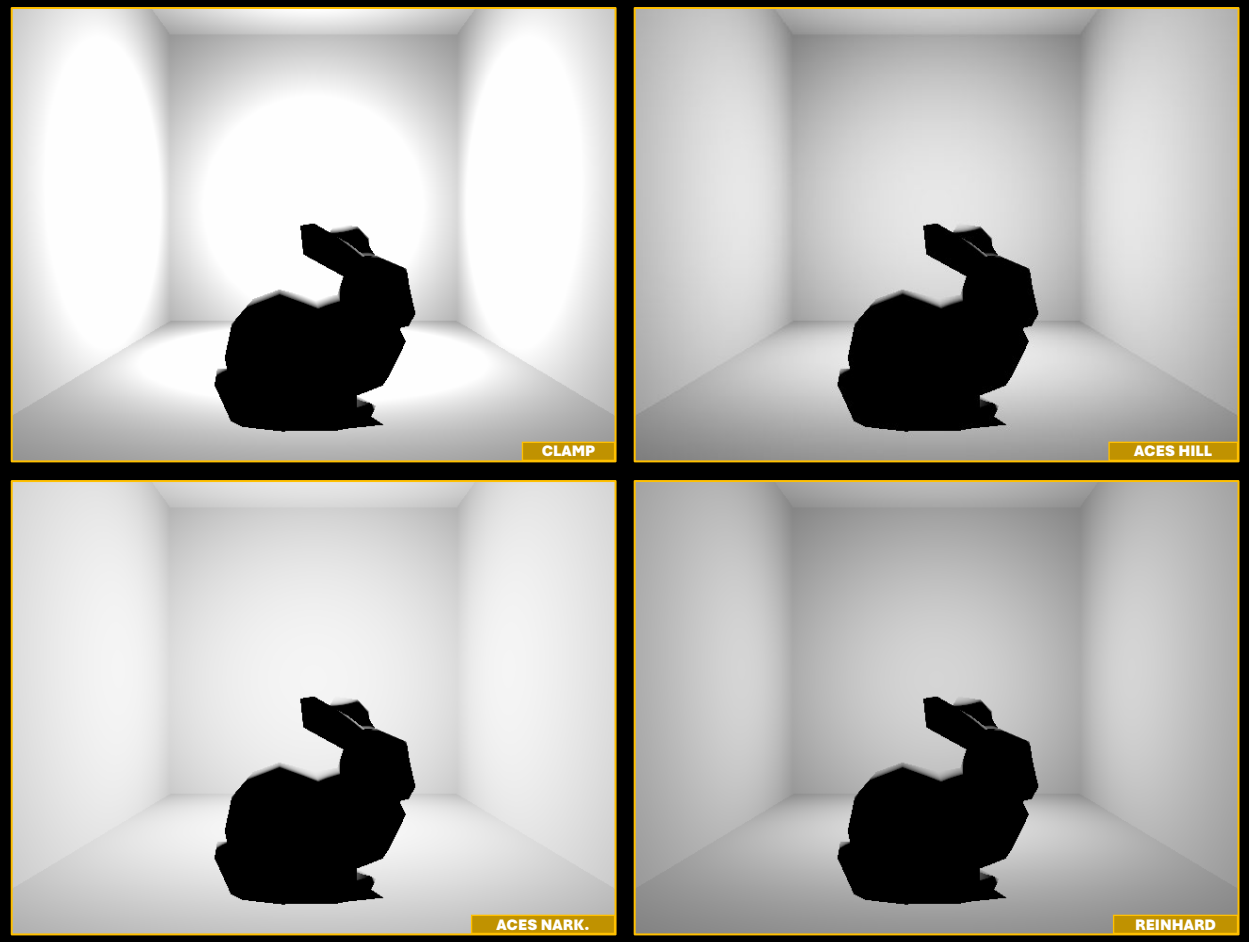
// Create a CalculateRadiance(.) function, which caluclates the radiance.
// Idealy this function accepts a Light* and const Vector3& with world position.
// See slides about Rendering Equation | Incidence for the formulas needed.
// Hint: SqrMagnitude() saves you a sqrt.
case LightType::kPoint: return ...;
case LightType::kDirectional: return ...;

// Use the returned radiance with the observerd area.
final_color += radiance * cos_theta;

// Radiance is unbounded, so values go past 1. Casting a float > 255 to uint8_t
// is undefined behaviour. Use Clamp() from the color.h at the bottom of the loop.
// Then render once more with ToSrgb() after it and compare the two.
final_color.Clamp();
```

Step 01 | Tone Mapping

Fix it using tone mapping. There are different operators, test them all. Important, all use sRGB remapping in the end!

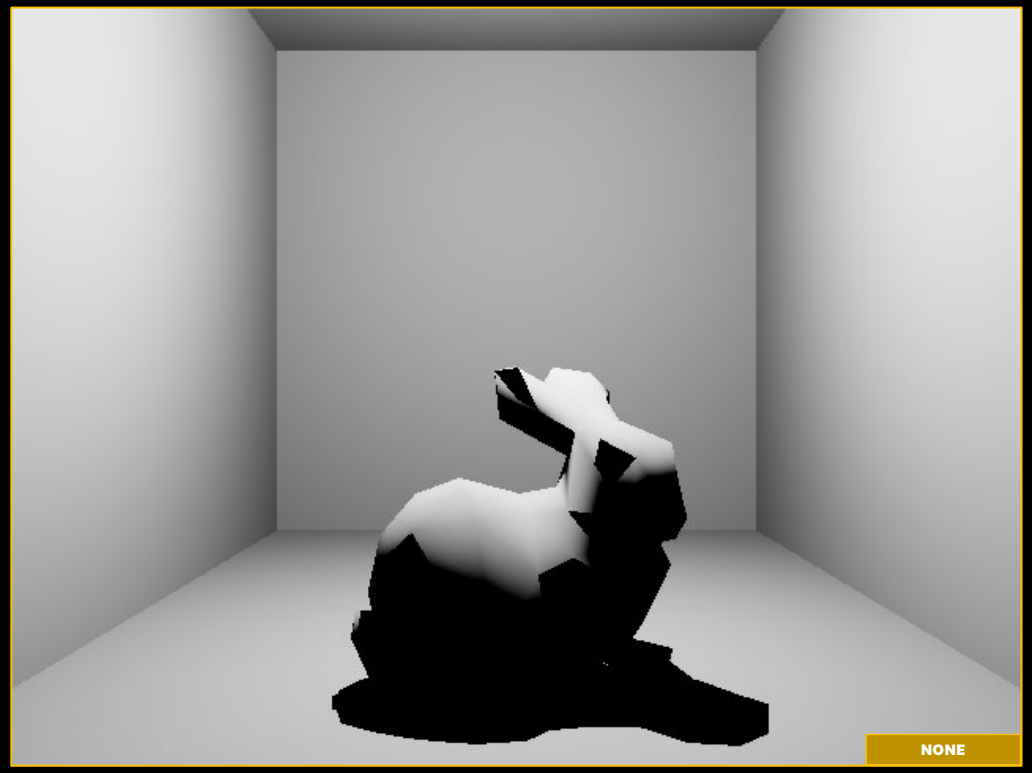



```
//============================================
// Step 01 - Tone Mapping
// Clamping discards everything above 1. Tone mapping compresses it instead.
// Operators also live in color.h (kNone, kReinhard, kACESNarkowicz, kACESHill)
// and are selected through debug_params. Only apply the display transform to
// kNone/kDirectOnly: the debug views are already display-referred.
// Important for renders, default (when not mentioned in name) is kACESHill,
// as seen in the debug_params.
final_color = final_color.Tonemap(context_->debug_params.tone_map_operator);
final_color = final_color.ToSrgb();

// Hint: keep the Clamp() before the write. kNone passes HDR straight through,
// and kReinhard maps luminance while holding the RGB ratios fixed, so a
// saturated channel can still land above 1. Only the ACES curves bound
// per channel.
```

Step 02 | Hard Shadows

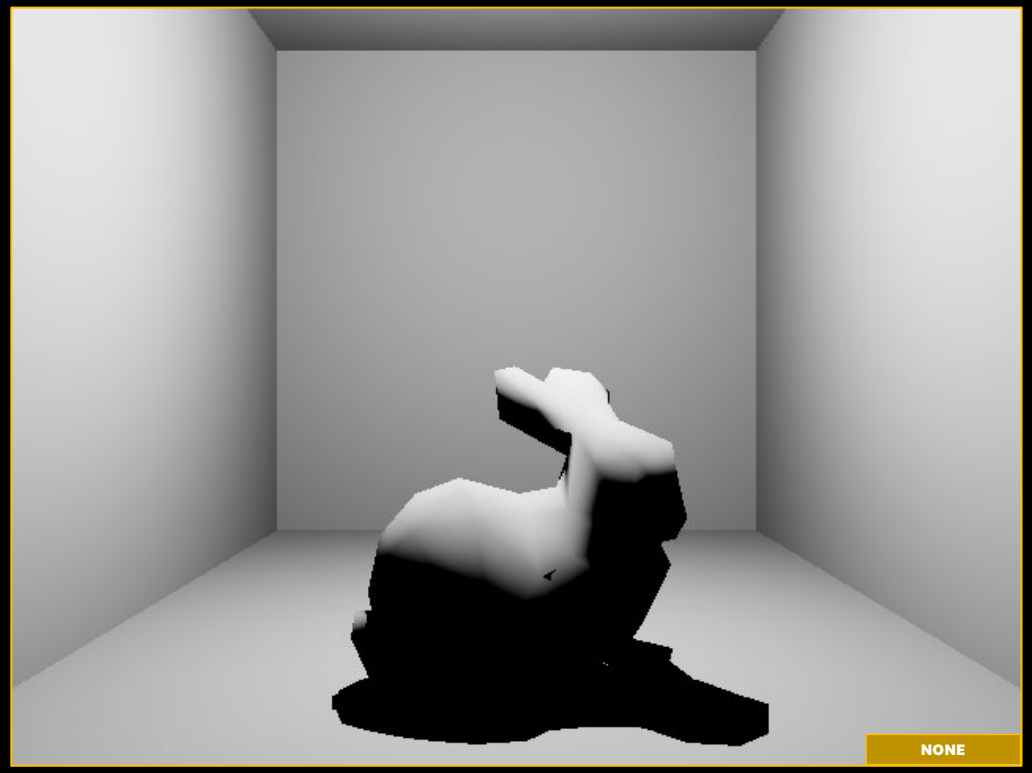
You will notice triangle artifacts, with smooth normals, when the light rotates! Screenshot below uses rotation of $\frac{3\pi}{2}$.




```cpp
//============================================
// Step 02 - Shadow Rays
// Trace from the hit point toward the light. Pass ignore_record = true:
// you only need to know whether something blocks it, which lets the test
// early-out.
// Cap .max at the light distance so geometry behind the light doesn't shadow it.
// Directional lights have no position, so they reach to infinity.
const Ray shadow_ray = {
    .origin = shading_input.world_position,
    .direction = light_dir,
    .min = 0.001f,
    .max = light->type == LightType::kDirectional
        ? std::numeric_limits<float>::max() : light_distance
};
RayHitRecord shadow_hit = {};
if (SceneClosestHitTest(shadow_ray, shadow_hit, true))
    continue;
```

Step 03 | Shadow Terminator Fix

Fix the previously seen issue based on technique by Johannes Hanika.




```
//=============================================
// Step 03 - Shadow Terminator
// With smooth normals the shading normal and the geometric normal disagree,
// so a shadow ray leaving the hit point can start below the interpolated surface.
// You will notice the artifacts in the previous renders. Let's now instead use the
// field shadow_origin in ShadingInput. Use it for the shadow_ray.origin, instead of
// shading_input.world_position.
struct ShadingInput final
{
    //...
    Vector3 shadow_origin{};
    //...
};
//...
shadow_ray.origin = shading_input.shadow_origin,
//...

// To calculate it, use the method from Johannes Hanika as mentioned in
// the slides. This happens in the ConstructShadingInput(.). The fallback is a
// small push along the normal. Do not to forget to transform it when there is
// a instance_transformation. Hint: all world positions and vectors need
// to be transformed into world space!
//...
const Vector3 offset =
  n0.value( ) * (w * d0) +
  n1.value( ) * (u * d1) +
  n2.value( ) * (v * d2);
shading_input.shadow_origin = p + offset + face_normal * 0.001f;
//...
```

Step 04 | Lambert BRDF

Add the Lambert BRDF and use it in the associated material.




```
//============================================
// Step 04 - Lambert BRDF
// Until now every surface reflects identically. Fetch the material for the hit
// object and ask it to describe the surface, then to shade it.
const SceneObject& obj = active_scene->objects.at(closest_hit_record.object_index);
const Material* material = active_scene->materials_factory.Get(obj.material_index);
const SurfaceProperties surface = material->GetSurfaceProperties(
    shading_input, active_scene->textures_factory);

// Complete GetSurfaceProperties(.) and Shade(.) for LambertMaterial.
// Remember, GetSurfaceProperties(.) takes the calculated ShadingInput,
// and for now just generates the surface properties using parameters
// defined for the entire surface, see params_ for those.
// The BRDF itself also needs to be completed and is in brdfs.h.
surface.albedo = params_.diffuse_color;
surface.shading_normal = input.world_normal;

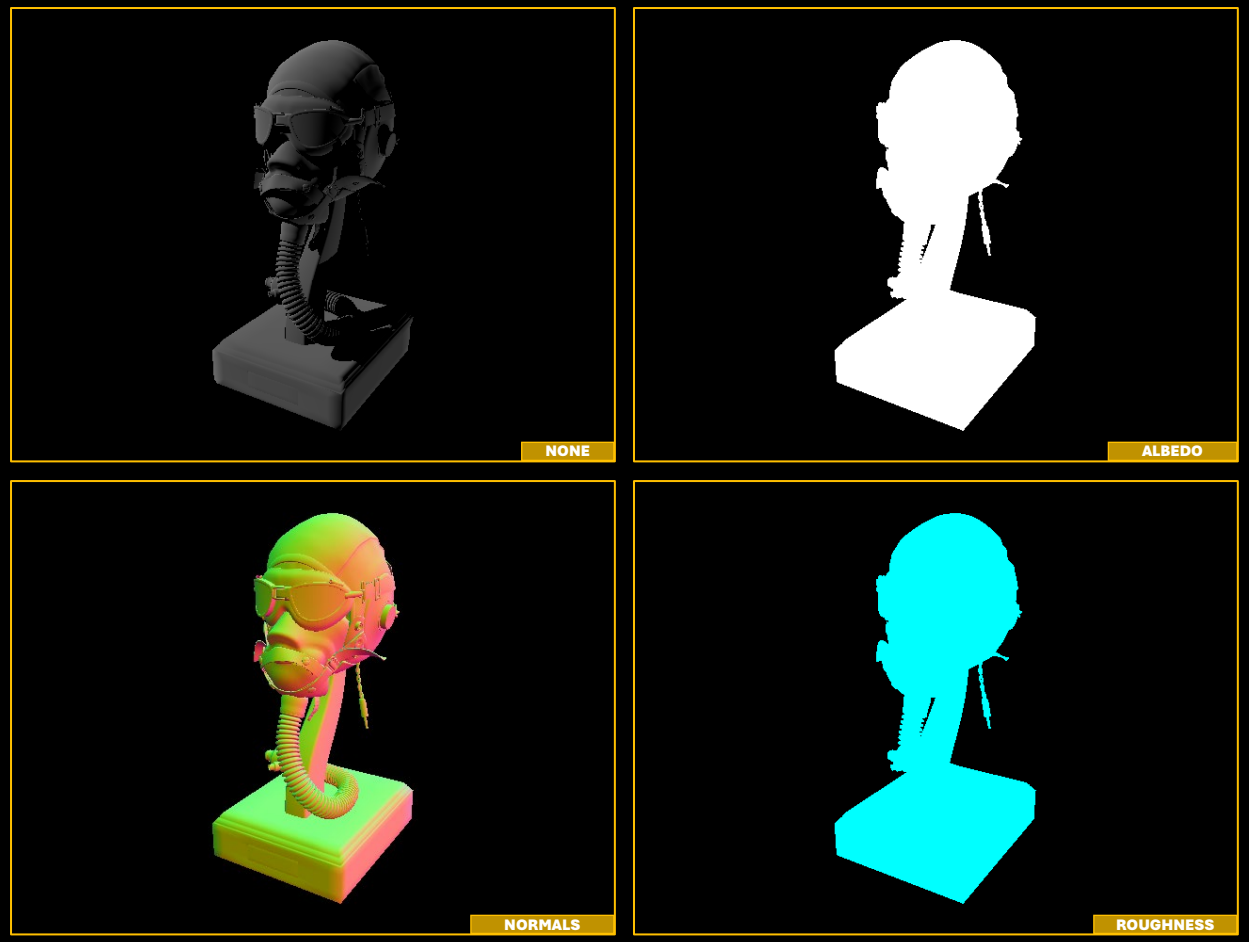
// Shade (.) uses the SurfaceProperties and actually evaluates the BRDF,
// also using the ShadingInput. Implement the Lambert BRDF itself in brdfs.h,
// and use it in Shade(.). Notice that here the ShadingInput is not needed
// as Lambert does not rely on any ShadingInput data, which is usually not the case.
return Lambert(params_.diffuse_reflectance, surface.albedo);

// Do not forget to set the light direction before you call Shade(.).
shading_input.light_direction = light_dir;

// This completes the current render equation implementation for now,
// which sets the final_color.
const ColorRgba brdf = material->Shade(shading_input, surface);
final_color += radiance * brdf * cos_theta;
```

Step 05 | Cook-Torrance BRDF

The biggest code completion, add the Cook-Torrance BRDF and use in associated material. Test on new scene!




```
//=============================================
// Step 05 - PBR BRDF
// In main.cpp:
context.scene_manager->CreateScene<CornellBoxScene>();
context.scene_manager->CreateScene<DamagedHelmetScene>();

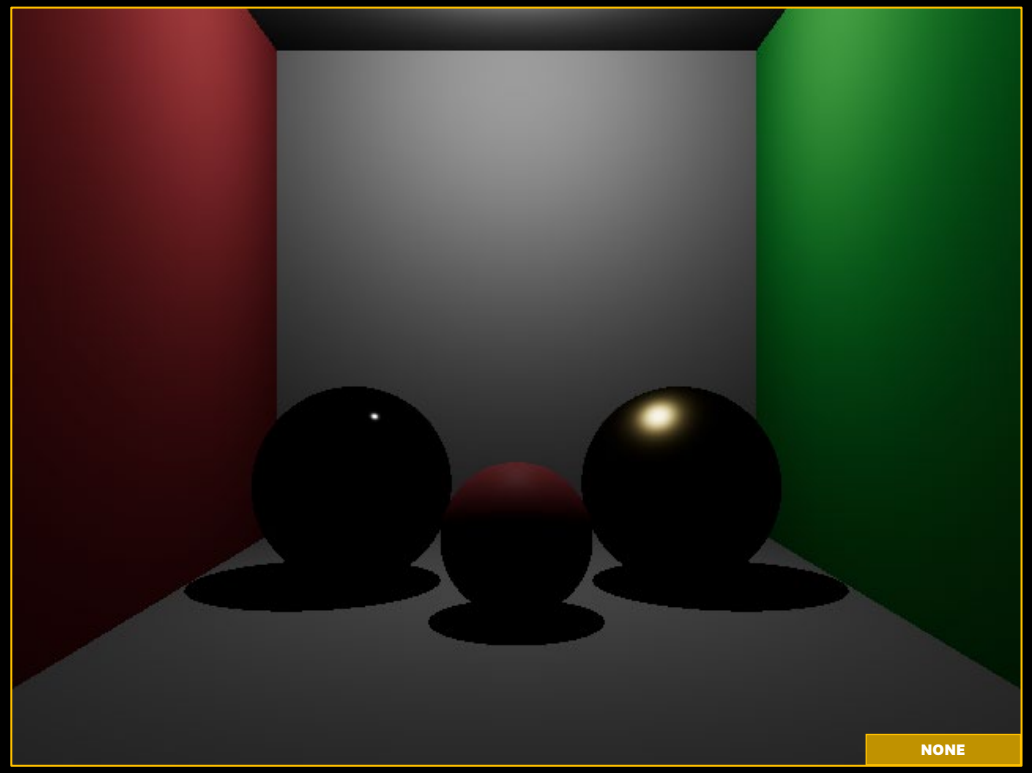
// Cook-Torrance = a diffuse lobe plus a microfacet specular lobe.
// All three specular terms are in brdfs.h; you combine them.
// In GetSurfaceProperties(.) of PbrMaterial, derive the base reflectivity F0
// from the IOR (see slides), then tint it with the albedo for metals:
const float f0_dielectric = ((ior - 1.f) / (ior + 1.f)) * ...;
surface.f0 = ColorRgba::Lerp(surface.f0, surface.albedo, surface.metallic);

// In Shade(.):
// Create the half vector
const Vector3 half_vector = (input.view_direction + input.light_direction).Normalized();
// Implement Cook-Torrance, and complete the BRDF code in brdf.h:
// F = FresnelFunction_Schlick(.),
// D = NormalDistribution_GGX(.),
// G = GeometryFunction_Smith(.)
// specular = (F * D * G) / (4 * max(Dot(n,v),0) * max(Dot(n,l),0))
// That denominator goes to zero at grazing angles, so guard it with
// std::max(denominator, 0.0001f).
// Metals have no diffuse lobe, and light that reflects cannot also scatter:
ColorRgba kd = ColorRgba(1.f, 1.f, 1.f) - F;
kd *= 1.f - surface.metallic;
return Lambert(kd, surface.albedo) + specular;

// Hint: the kAlbedo, kMetallicRoughness and kUV debug views become
// useful here: check them before trusting the lit result!
```

Step 05 | Cook-Torrance BRDF

The biggest code completion, add the Cook-Torrance BRDF and use in associated material. Test on new scene!




```
//===========================================
// Step 05 - PBR BRDF
// In main.cpp:
context.scene_manager->CreateScene<CornellBoxScene>();
context.scene_manager->CreateScene<DamagedHelmetScene>();

// Cook-Torrance = a diffuse lobe plus a microfacet specular lobe.
// All three specular terms are in brdfs.h; you combine them.
// In GetSurfaceProperties(.) of PbrMaterial, derive the base reflectivity F0
// from the IOR (see slides), then tint it with the albedo for metals:
const float f0_dielectric = ((ior - 1.f) / (ior + 1.f)) * ...;
surface.f0 = ColorRgba::Lerp(surface.f0, surface.albedo, surface.metallic);

// In Shade(.):
// Create the half vector
const Vector3 half_vector = (input.view_direction + input.light_direction).Normalized();
// Implement Cook-Torrance, and complete the BRDF code in brdf.h:
// F = FresnelFunction_Schlick(.),
// D = NormalDistribution_GGX(.),
// G = GeometryFunction_Smith(.)
// specular = (F * D * G) / (4 * max(Dot(n,v),0) * max(Dot(n,l),0))
// That denominator goes to zero at grazing angles, so guard it with
// std::max(denominator, 0.0001f).
// Metals have no diffuse lobe, and light that reflects cannot also scatter:
ColorRgba kd = ColorRgba(1.f, 1.f, 1.f) - F;
kd *= 1.f - surface.metallic;
return Lambert(kd, surface.albedo) + specular;

// Hint: the kAlbedo, kMetallicRoughness and kUV debug views become
// useful here: check them before trusting the lit result!
```

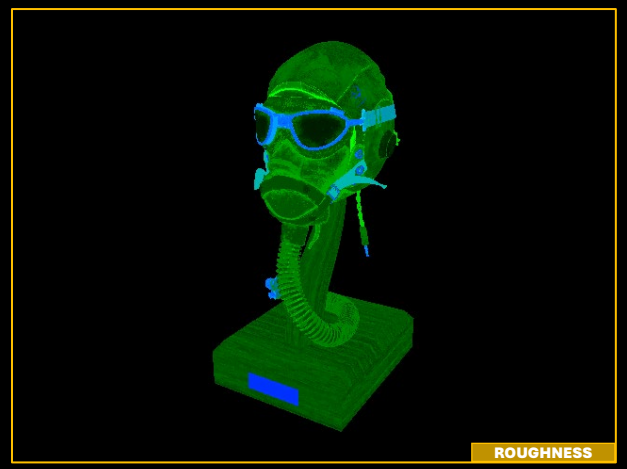
Step 06 | Textures

Add texture support in the GetSurfaceProperties. Make sure to check if the texture is available.





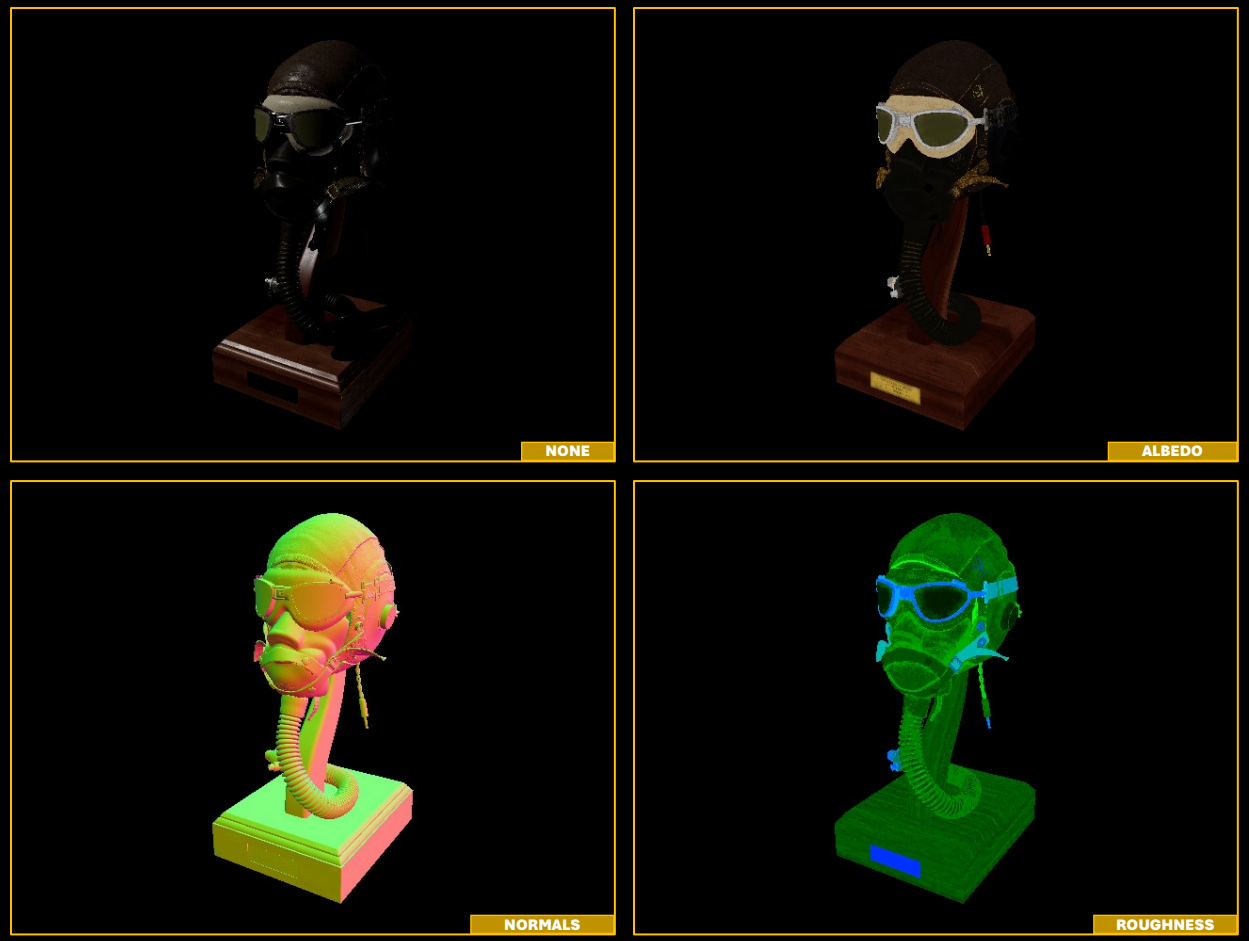





```
//=============================================
// Step 06 - Textures
// Drive the surface properties from maps, sampled with the
// UVs you interpolated in week 02. Use the textures in GetSurfaceProperties(.)
// with the Sample(.) function of the texture class itself. In other words,
// if a albedo texture is available, it overwrites the shared param_ value,
// so surface.albedo = the sampled texture value instead of params_ value.
// IMPORTANT: color textures (albedo) are authored in sRGB and must be
// decoded before any lighting maths. Do not decode non-color textures
// (e.g. normals, roughness, etc.). You can check using IsSrgb(), and
// can convert using ToLinear().
// Hint: no other code should change because of the use of SurfaceProperties!
const ColorRgba sample = tex->Sample(input.texture_coordinates.value());
surface.albedo = (tex->IsSrgb() ? sample.ToLinear() : sample);
```

Step 07 | Normal Mapping

Finally, complete the SampleNormal in the template material.




```
//=============================================
// Step 07 - Normal Mapping
// Surface detail without extra geometry. Complete the
// SampleNormal(.) template helper in materials.h. Texture::
// SampleNormal(.) handles the [0,1] -> [-1,1] remap and
// reconstructs Z from the unit-length constraint, so no sRGB
// decode is involved.
const Vector3 tangent_normal = tex->SampleNormal(input.texture_coordinates.value());

// Transform tangent space -> world space with the TBN basis.
// The TBN vectors can be taken from the ShadingInput.
const Matrix tbn{ T, B, N, Vector3{} };
return tbn.TransformVector(tangent_normal).Normalized();

// Store it, then shade with it instead of the geometric normal:
surface.shading_normal = SampleNormal(params_, input, textures);
// in Render(.):
shading_input.world_normal = surface.shading_normal;
```

Step 07 | Normal Mapping

Finally, complete the SampleNormal in the template material.



GOOD LUCK!